IBM Rhapsody Systems Engineering

Contents

Overview of IBM Rhapsody Systems Engineering

IBM® Rhapsody® Systems Engineering is a web-based solution designed for systems engineering teams, enabling them to create smarter, more sophisticated, and more competitive solutions for their end users. It transforms increasing design complexities into a competitive advantage.

The inherent nature of systems, whether in their initial design phase or during subsequent upgrades, is characterized by a rapid accumulation of new capabilities outpacing the removal of obsolete ones. This inevitably leads to an escalation in complexity over time. As systems grow more intricate, they become increasingly challenging to comprehend, resulting in heightened unpredictability. Consequently, new behaviors emerge, often accompanied by unintended and potentially catastrophic side effects. However, this complexity is not without its merits. By adeptly harnessing and managing the intricacies of complex systems, organizations can develop smarter products, offer unique and differentiated solutions, and therefore improve their competitive edge in their respective market.

Rhapsody Systems Engineering is based on several key technologies briefly described below that allow customers to leverage the benefits on complexity while mitigating its associated risks.

Key features of Rhapsody Systems Engineering

Built to support SysML V2

Rhapsody Systems Engineering supports the new SysML V2 standard, helping practitioners design complex systems graphically, with dedicated graphic editors, customizable browsers and a configurable set of completeness and correctness checks. It also supports working with SysML V2 textual representations of the models.

To provide a consistent experience and modeling guidelines within and across projects in an organization, tools and methods experts can customize the tool to support the specific processes and workflows needed. This customization includes user-defined extensions, by using JavaScript or Python, for example that use the various APIs that are provided with Rhapsody Systems Engineering, extending well beyond the APIs defined by the SysML V2 standard.

Web-based solution

Rhapsody Systems Engineering offers modern and intuitive user experiences and workflows for everyone in systems engineering teams. This comprehensive support includes systems engineering practitioners and reviewers, domain architects, compliance and security officers, design partners and suppliers. A web browser and a URL is all you need to access the tool. Administrators benefit from it too, since the product is delivered as containers, easing the speed and simplicity of installation and product configuration.

Harmony model-based engineering

Harmony model-based engineering (HarmonyMBE) is an iteration of the Harmony workflow that simplifies model development and analysis workflows, providing a prescriptive process empowered by automations that reduce the amount of manual work required to create correct and consistent models that are based on the SysML V2 languages and at the size and complexity of real-word systems and systems-of-systems. HarmonyMBE is designed to simplify the use of SysML V2 and streamline complex model building tasks. For example, on creating a new project, the HarmonyMBE automation automatically builds a predefined model structure and layers, which helps you to quickly set up your projects with the main steps and placeholders of the Harmony process.

Part of the IBM Engineering Lifecycle Management solution

Rhapsody Systems Engineering integrates with IBM Engineering Lifecycle Management's open, federated, versioned digital thread for engineering artifacts, including its support for Global Configurations. It integrates through open services for lifecycle collaboration (OSLC) or other APIs with other downstream engineering domains, such as software design with production code generation, using IBM Rhapsody Developer and Rhapsody Designer, allowing customers to manage cross domain digital threads while bridging the gap between systems engineering and downstream

software engineering using the unified modeling language (UML) or AUTOSAR Classic or Adaptive for automotive applications.

For more information, see the following video.

Related information

Overview of IBM Engineering Systems Design Rhapsody

What's new in Rhapsody Systems Engineering 1.5.0

Learn more about what's new and changed in Rhapsody Systems Engineering 1.5.0.

General usability

Exporting diagrams

You can download your diagrams in the SVG format by using **Download SVG** option available in the right toolbar of the view area. For more information, see "Exporting diagrams" on page 121.

Populating to a new view

The Populate to a new view dialog box includes various options to customize your view.

- You can define the view name, owner, view type, and select model-specific elements.
- Choose whether to include recursive members and specify element and relationship types.
- Control which relationships are displayed using new filtering options for direction and scope.

For more information, see "Generating views using the Browser view panel" on page 127.

Quick actions in a view

You can now perform additional actions directly in the view area using the Additional operations menu.

- Hide or show ports using the **Ports** option.
- Hide or show Action parameters using the Parameters option.
- Modify the element type using **Change type** option.
- Create a definition with **Create definition**, which moves all features from usage to the new definition, creates a typing relationship between them. The usage inherits all features like attributes and ports from the new definition.

The ports or parameters option appears only when the element has ports or parameters. The Create definition menu option appears only when the usage is not already typed.

Focusing on an element in a view

You can now quickly focus on a selected element in a view by using the **Auto Fit to View on Selection**  icon. To focus on an element in a view, click **Auto Fit to View on Selection** () icon to enable it, and then select the element from the Browser panel. The diagram automatically pans and centers on the selected element.

Views

Table view

- Assign element references by dragging elements from the Browser panel to the reference field.
- View the hierarchy of elements by enabling Hierarchy mode.
- Sort columns alphabetically or numerically based on their content.

Matrix view

To display your model elements and their relationships in a structured manner, you can use matrix views in Rhapsody Systems Engineering. For more information, see "Matrix view" on page 120.

Configuration management

Committing changes to public branch
You can commit your changes directly to a selected public branch.

Merging latest changes
You can keep your project branch up to date by merging the latest changes from its source branch, typically `main`.

Searching by commits
A search bar is added to the Commit box enabling you to search through your commits.

For more information, see “Managing changes” on page 93.

Language extension

Metadata support
You can add information to elements without changing their core semantics or structure. This is done by using metadata definition elements, which you can create and manage in the language extension view. For more information, see “Metadata support” on page 134.

Managing extensions

To improve the efficiency of your projects, you can directly enable extensions from the user interface of Rhapsody Systems Engineering. For more information, see “Enabling extensions” on page 137.

Monitoring the licenses in use

As a server administrator, you can review the licenses that are checked out in the License Monitor page. For more information, see “Monitoring the licenses in use” on page 84.

IBM Common Licensing server

The IBM Common Licensing Server version is upgraded to version 9.0.0.2. Starting from Rhapsody Systems Engineering version 1.6.0, upgrade the IBM Common Licensing Server to version 9.0.0.2. For more information, see Upgrading IBM Common Licensing server.

What's new in Rhapsody Systems Engineering 1.8.0

Learn more about what’s new and changed in Rhapsody Systems Engineering 1.8.0.

Configuration management

When you use your current project as a reusable template and create a new project, the new project is automatically initialized with predefined content, including all project usages from the original source project.

Adaptive ports

Ports act as intelligent endpoints, automatically adjusting when elements are moved or resized. Connections stay aligned and no longer remain fixed in place. Consider the following enhancements.

- Ports connected to multiple edges are positioned based on overall layout context, not just a single connection.
- Locking is applied at the port level. A single, clear indicator represents the lock state, and changes affect all connected edges together.
- Port-connected edges support the same editing interactions as other connections, including manual routing, lane spacing, and bend point adjustments.

For more information, see “Adaptive ports and connections” on page 123.

Access control

As a server or project administrator, you can assign reviewers to projects in your organization. For more information, see “Reviewing projects” on page 78.

General usability

Additional operations for ports or parameters

When you select ports or parameters, the **Additional operations** menu provides the following options.

- You get the option to view only connected ports or parameters in a diagram.
- You can create definitions for ports or parameters.

For more information, see “Additional operations of ports or parameters” on page 126.

Improved placement of ports

You can add ports by selecting them from the palette and then clicking on the node. Ports are automatically positioned based on cursor location:

- Left side → Input ports
- Top side → Unassigned ports
- Right side → Output ports
- Bottom side → InOut ports

You can also update the direction of port after its creation. Select the port and choose the desired direction from the **Port Direction** menu. The port will automatically move to the appropriate side of the node.

Creating ports with direction

You can directly create ports with direction from the contextual toolbar of the diagram. For more information, see Creating ports.

Appearance settings for ports

You can update the properties of the port displayed in the Properties panel. For example, by using **Declared short name** field, you can provide a short name for the port that would be displayed in the diagram.

You can choose to customize the appearance of ports by customizing their color, label content, and label positioning. For more information, see “Customizing the appearance of ports ” on page 127.

Improved routing in shared ports

The ports connected to multiple edges are placed based on all connections resulting in more stable layouts. Instead of following a single connection, the port considers all connected edges and selects the side that best fits the overall layout.

Improved routing in edges

The edges automatically avoid overlapping with nodes during routing. When an automatically routed edge encounters a blocking node, it creates a clean orthogonal detour around the obstacle while maintaining existing layout behavior such as lane spacing and user adjustments.

Improved search capability

In the project editor, the search capability is enhanced to include both model elements and library elements. You can now search using:

- Library element names
- Partial keywords (for example: `break`, BRE)
- Element IDs

Consider the following search enhancements.

- Search supports multiple imported libraries simultaneously.
- Matching results are highlighted across all relevant elements.

- Selecting a search result automatically navigates to the corresponding model or library view.
- Pagination support is available for navigating large result sets.

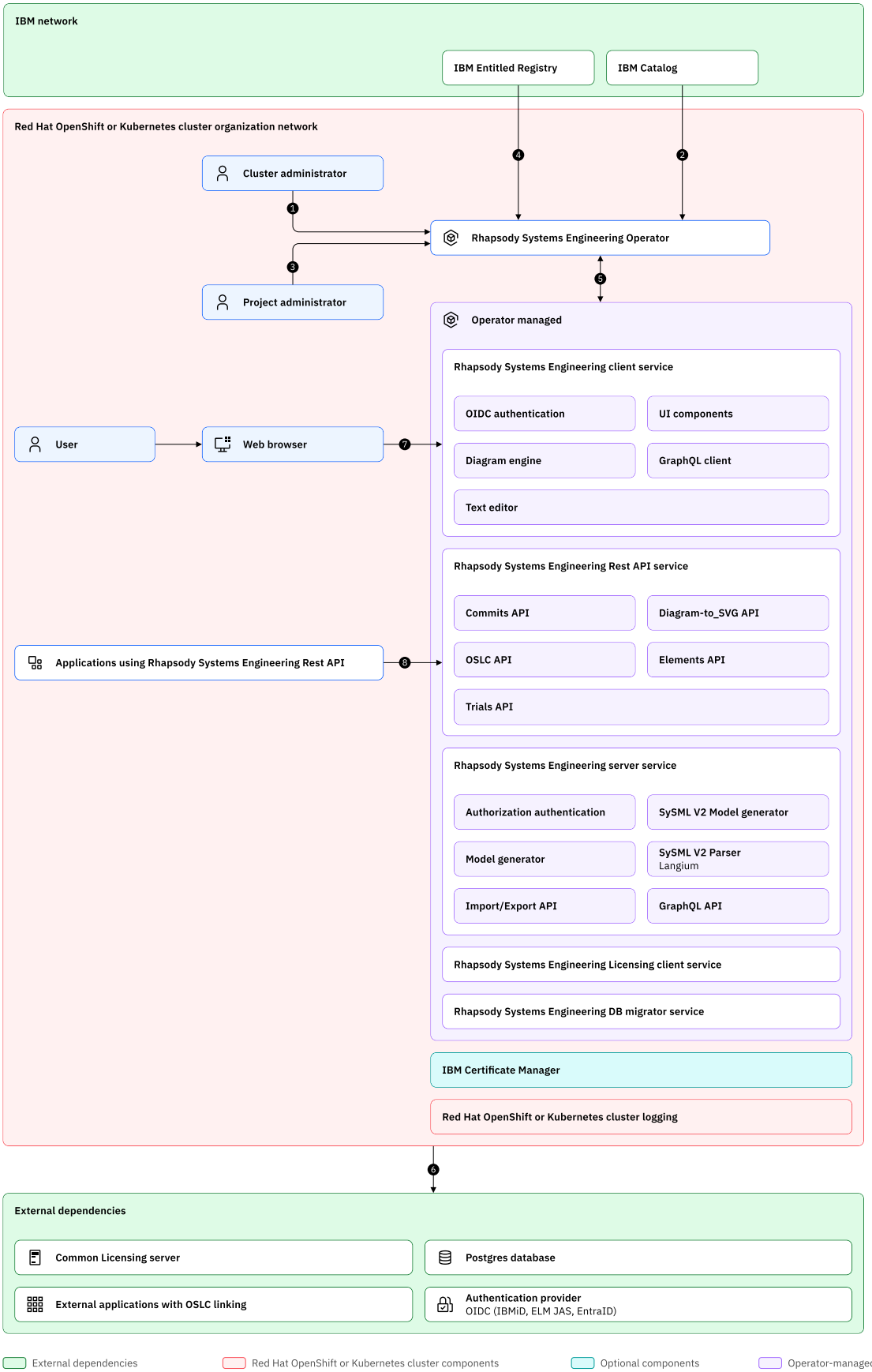
Selecting and placing elements

You can quickly add elements to your view by selecting and clicking them on the palette. For more information, see “Selecting and placing the elements” on page 98.

Architecture of Rhapsody Systems Engineering

Rhapsody Systems Engineering is deployed on a Red Hat OpenShift cluster or Kubernetes cluster. It can be deployed in the cloud or on-premises.

The following is the detailed workflow that outlines the flow in the deployment of Rhapsody Systems Engineering. The cluster administrator installs the Rhapsody Systems Engineering operator, and the project administrator deploys the instance, and configures dependencies with integrations including with OIDC authentication. You can then access Rhapsody Systems Engineering.

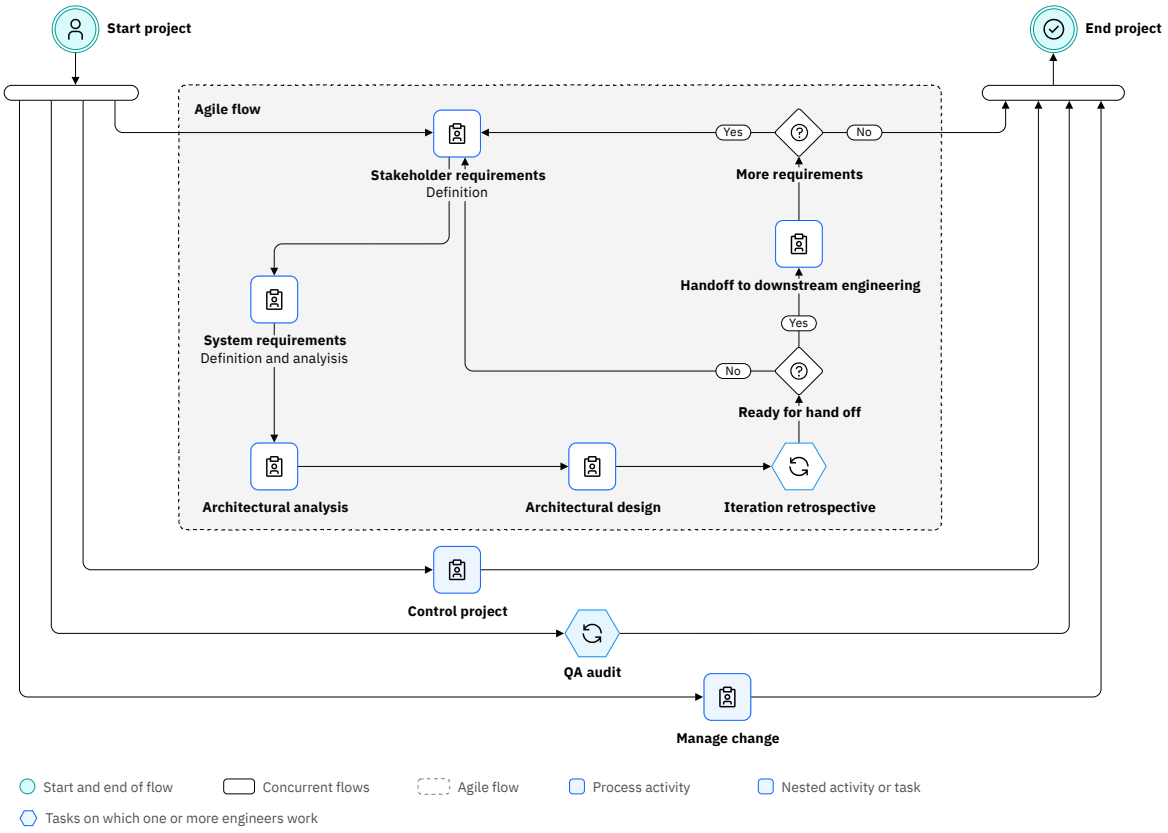

IBM network
IBM Entitled Registry
IBM Catalog
Red Hat OpenShift or Kubernetes cluster organization network
Cluster administrator
Rhapsody Systems Engineering Operator
Project administrator
Operator managed
Rhapsody Systems Engineering client service
OIDC authentication
UI components
Diagram engine
GraphQL client
Text editor
User
Web browser
Rhapsody Systems Engineering Rest API service
Commits API
Diagram-to_SVG API
Applications using Rhapsody Systems Engineering Rest API
OSLC API
Elements API
Trials API
Rhapsody Systems Engineering server service
Authorization authentication
SysML V2 Model generator
Model generator
SysML V2 Parser
Langium
Import/Export API
GraphQL API
Rhapsody Systems Engineering Licensing client service
Rhapsody Systems Engineering DB migrator service
IBM Certificate Manager
Red Hat OpenShift or Kubernetes cluster logging
External dependencies
Common Licensing server
Postgres database
External applications with OSLC linking
Authentication provider
OIDC (IBMiD, ELM JAS, EntraID)
External dependencies
Red Hat OpenShift or Kubernetes cluster components
Optional components
Operator-managed

1. **Cluster administrator installs the Rhapsody Systems Engineering operator**
   The cluster administrator installs the Rhapsody Systems Engineering operator within the Kubernetes or Red Hat OpenShift to manage and operate the Rhapsody Systems Engineering application lifecycle.
2. **Automation pulls the Rhapsody Systems Engineering operator image**
   The Rhapsody Systems Engineering operator image is automatically pulled from the operator catalog or a predefined source. The Rhapsody Systems Engineering operator image is fetched from the catalog source, which can be an external or internal registry, ensuring that the operator image is signed and verified.
3. **Project administrator deploys the Rhapsody Systems Engineering instance**
   The project administrator deploys the Rhapsody Systems Engineering instance by using the installed Rhapsody Systems Engineering operator. It creates a custom resource (CR) definition here for the Rhapsody Systems Engineering instance by using YAML or the `oc or kubectl` commands. It also provides the configuration settings, such as the namespace, replicas, and resource limits for the deployment. At a later stage the Rhapsody Systems Engineering operator automatically picks up the CR and configures the required resources for the application.
4. **Rhapsody Systems Engineering operator pulls Rhapsody Systems Engineering application images from the IBM Entitled Registry**
   The Rhapsody Systems Engineering operator pulls the necessary application images from the IBM Entitled Registry or any other authorized container registries by using Docker or Red Hat OpenShift image pulls. The Rhapsody Systems Engineering operator is configured with credentials for the IBM Entitled Registry and the operator pulls the container images for the Rhapsody Systems Engineering applications.
5. **Rhapsody Systems Engineering operator creates and configures the Rhapsody Systems Engineering application pods**
   The Rhapsody Systems Engineering operator orchestrates the creation and configuration of the Rhapsody Systems Engineering application pods in the Kubernetes or Red Hat OpenShift. Rhapsody Systems Engineering operator then creates the required pods, services, deployments, and secrets that are required for the Rhapsody Systems Engineering to run. The operator ensures that the application containers are deployed correctly with the environment variables and settings that are applied for each pod.
6. **Organization uses OIDC as the authentication provider**
   The organization integrates an external authentication provider, OIDC to manage user access to the Rhapsody Systems Engineering application. It helps in configuring Rhapsody Systems Engineering to authenticate users by using OIDC.
7. **Rhapsody Systems Engineering user accesses Rhapsody Systems Engineeringby using the web clients or rich clients**
   Users can access the Rhapsody Systems Engineering by using web-based clients or rich client applications by using the web browser or a desktop application. The Rhapsody Systems Engineering shows a web interface that can be accessed by using a browser. Users can also interact with the Rhapsody Systems Engineering application to manage licenses, view reports, and so on.
8. **Users can access Rhapsody Systems Engineering by using Rhapsody Systems Engineering Rest API**
   Rhapsody Systems Engineering APIs allow you to interact with the application. You can query and alter your projects, elements, and configurations by using these APIs.

Harmony model-based engineering

Harmony model-based engineering focusses on the development of model-based system engineering work products, such as requirements, architecture, interfaces, trade studies, and analysis such as safety, reliability, and security. It does this in an agile fashion by incorporating incremental development of engineering data, early and continuous verification of the correctness of that information, and continuous integration of the work of collaborating engineers. The following figure shows the overall process flow. It

also enables rapid prototyping and testing of embedded systems using model-driven development (MDD) techniques.



Each of the rectangular boxes in the figure represents a process activity, which, in turn, is defined by a set of nested activities or tasks. The diamonds represent decision points (at which only a single flow is taken at a time), while the horizontal bars are either forks or joins, which represent concurrent flows. The labeled pentagons are tasks on which one or more engineers work. Each task is defined with inputs and outputs, a purpose, description, the set of steps necessary to complete the task, and optional guidance material.

The following are the activities and tasks of the Harmony model-based engineering process:

Initialize project
: Identify and prioritize stakeholder use cases, create the engineering team structure, create first cut schedule, risk management plan, and the system engineering management plan.

Define stakeholder requirements
: Identify stakeholders of interest, stakeholder needs as requirements, allocate these to use cases, and perform rudimentary requirements analysis, normally limited to scenario elaboration.

System requirements definition and analysis
: Identify system use cases (normally 1:1 match for the stakeholder use cases), derive system requirements, allocate them to use cases, analyze the use cases with computable models, create logical flow data and flow schema, analyze dependability, and create the initial system verification plan.

Architectural analysis
: Identify and analyze system trades and make technological and/or architectural choices based on that analysis.

Architectural design
Identify subsystems, allocate system requirements to subsystems, create subsystem requirements, create and allocate use cases to subsystems, update the logical data schema, develop control laws, and update dependability analyses.

Control project
Perform project management activities, maintain risk management plan, and use daily meetings to enhance engineer collaboration

Perform QA audit (task)
Perform quality assurance audits to ensure process compliance.

Manage changes
For work products under configuration management, control the change request process including review, assignment, resolution, and verification of each change request.

Perform iteration retrospective (task)
Ascertain the project's adherence to the project plan and look for opportunities to improve; also replan as necessary and appropriate.

Handoff to downstream engineering
Develop materials necessary for downstream engineering, including physical interface specification, creation of subsystem models, creation of a deployment (interdisciplinary) model, allocate requirements to engineering disciplines and define the interdisciplinary interfaces.

The following are the goals of Harmony model-based engineering:

- Reduce the amount of merging necessary. Both previous iterations had one thing in common – use cases were modelled separately (either in separate Rhapsody models or in separate packages in the same model) and then had to be merged. This merge process inevitably resulted in conflicts as the use case teams were independent. Harmony model-based engineering works with a common pool of functions at the functional level allowing for shared modelling and less conflict resolution.
- Promote reuse. Each architectural level might be reused so engineers are not starting with a blank slate.
- Make the workflow repeatable to multiple levels of architecture.
- Use few extensions to the standard systems modeling language V2 (SysML V2).
- Provide more guidance, flexibility, and automation for the user.

Harmony automation for Rhapsody Systems Engineering

Harmony automation for Rhapsody Systems Engineering is designed to streamline complex model building tasks. Creating a new project by using Harmony template automatically builds a predefined model structure, which helps you to quickly set up your projects with the main steps and placeholders of the Harmony process. This automation reduces manual effort and ensures consistency, making it easier to manage and develop complex models.

The project created by using the Harmony template is organized into four top-level packages representing the main steps of the Harmony process: 1_RequirementsAnalysisPkg, 2_FunctionalAnalysisPkg, 3_LogicalArchitecturePkg, and 4_PhysicalArchitecture. There are two indicators available for use cases, which helps you to visually track their progress across the architecture layers:

- : Indicates that the use case is propagated to the next architecture layer.
- : Indicates that the use case is not propagated to the next architecture layer. You can move them to next layer by using the Generate <next-leyer> arttifacts" option from the contextual menu of the use case.

Each top-level package includes the following components:

ActorPkg, ItemsPkg, PortsPkg, RequirementsPkg, UseCasePkg
These packages contain the most frequently used elements at each level.

System parts

- system represents the system at the current level.

- system_Functional represents the functional aspects of system.
- system_Logical represents the logical structure of system.
- Functional and logical system parts have a subsetting relationship with the system part of the previous level.

System context view
An interconnection view available at each level.

Use case view
This view is located under **RequirementsAnalysisPkg** > **UseCasePkg**. It is used to visualize use case analysis.

You can rename the elements created automatically during project setup. However, they retain their original functionality within Harmony automations.

Architectural design – Functional analysis

Functional analysis is done by examining individual use cases. Each use case can be analyzed independently, which allows for a detailed and focused examination of specific components of the system. The use cases contribute to the overall functional architecture.

Generating functional analysis artifacts

You can generate functional analysis artifacts by using the contextual menu of use cases inside 1_RequirementsAnalysisPkg. It populates the functional analysis layer with the artifacts related to the selected use case.

This operation creates or updates the following artifacts under 2_FunctionalAnalysisPkg:

- A use case is created under `UseCasePkg` with a subsetting relationship to the use case in `1_RequirementsAnalysisPkg`.
  - It includes the internal structure of use case, which includes actors and subjects bounded to the corresponding system actors and `system_Function` part.
  - In the use case, `mainBehavior` view action flow view is added under `mainBehavior` action.
- The related system actor parts are created under ActorPkg.
  - The copies of system actor parts are created that are bounded to the selected use case in `1_RequirementsAnalysisPkg`.
  - Port `pSystem` is created with feature typing relationship to the conjugated port definition `~<actor-name>Port` that represents connection to the system.
- Port definitions are created for each created system actor part.
  - Port definition `<actor-name>Port` is created with the the conjugated port definition `~<actor-name>Port` under it.
- Ports are added to the `system_Function` part that represents connection to the related system actor parts.
  - Ports named `p<Actor-name>` with feature typing relationship are created with the port definition `<actor-name>Port`.
- Interface relationships between `system_Function` part and related system actor parts are created.
  - Interface between system actor parts `pSystem` port and `system_Functional` part's `<actor-name>Port` port is created.
- system context view in 2_FunctionalAnalysisPkg is populated with the related system actor parts and interfaces to the system.

Populating ports

You can add items from the action flow view to their corresponding port definitions by selecting the Populate ports option in the contextual menu of the selected use case.

- Items are added to port definitions in PortsPkg.
  - The item definitions used as payload in Accept action nodes and items used as payload in Send action nodes in action flow view are converted into items in port definition based on the information of a receiver or sender through parameter of the action.
  - Items in Accept actions added as in and items in Send actions added as out directed features.
  - The direction of ports for the system actors and system part is set to in or out.

Architectural design – Logical architecture

Building logical architecture analysis in a use case-based fashion follows a similar approach as functional analysis. Each use case can be analyzed independently, which allows for a detailed and focused examination of specific components of the system. The use cases contribute to the overall logical architecture.

Generating logical architecture artifacts

You can generate logical architecture artifacts by using the contextual menu of use cases inside 2_FunctionalAnalysisPkg. It populates the logical architecture layer with the artifacts related to the selected use case.

This operation creates or updates the following artifacts under 3_LogicalArchitecturePkg.

- Use cases are created under UseCasePkg with subsetting relationships to the use cases in 2_FunctionalAnalysisPkg.
  - It includes the internal structure of use case including actors and subjects bounded to the corresponded system actors and system_Logical part.
  - mainBehavior view action flow view is created under mainBehavior action copied to the use case.
- The related system actor parts are created under ActorPkg.
  - The copies of system actor parts are created that are bounded to the selected use case in FunctionalAnalysisPkg.
- Interface relationships between system_ Logical part and the related system actor parts are created.
- system context view in 3_LogicalArchitecturePkg is populated with the related system actor parts and interfaces to the system.

Creating ports and interfaces

You can create ports and interfaces for a sub-system, by selecting the "Create ports and interfaces" option in its contextual menu. It uses swimlanes in the mainBehavior action of the use cases, identifies interactions between subsystems, and adds ports to the selected subsystems connecting them with interfaces between the ports. The following artifacts are created or updated under under 3_LogicalArchitecturePkg:

Port definitions added for each interacting subsystem
Port definition <Subsystem1-name_ Subsystem2-name>port with the corresponding conjugated port definition under it.

Ports added to the interacting subsystem parts
Ports named p<Other Subsystem> with feature typing relationship to the port definition <Subsystem1-name_ Subsystem2-name>Port

Ports added to the subsystem parts that interacts with the system actors
Ports named p<Actor-name> with feature typing relationship to the port definition <actor-name>Port.

Interface relationships between ports of the interacting subsystem parts
Interface between port p<Subsystem2> subsystem subsystem1, and port p<Subsystem1> subsystem subsystem2.

Interface relationships
Interface relationships between system_Logical part with the related subsystems.

system context view
system context view in 3_LogicalArchitecturtePkg is populated with the created ports and interfaces.

Architectural design – Physical architecture

Building physical architecture analysis in a use case-based fashion follows a similar approach as functional analysis. Each use case can be analyzed independently, but it contributes to the overall logical architecture.

Generating physical architecture artifacts

You can generate physical architecture artifacts by using the contextual menu of use cases inside 3_LogicalArchitecturePkg. It populates the physical architecture layer with the artifacts related to the selected use case.

This operation creates or updates the following artifacts under 4_PhysicalArchitecturePkg.

- The related system actor parts are created under ActorPkg. The copies of system actor parts are created that are bounded to the selected use case in 3_LogicalArchitecturePkg.
- system context view in 4_PhysicalArchitecturePkg is populated with the related system actor parts.

Creating ports and interfaces

You can create ports and interfaces for a physical part or system_Physical part including all sub-parts, by selecting the "Create ports and interfaces" option in its contextual menu. It uses allocation relationships from logical subsystems to physical parts and from logical item definitions to physical item definitions and builds physical port definitions, corresponding ports the selected physical parts connecting them with interfaces between the ports.

The following artifacts are created or updated under under 4_PhysicalArchitecturePkg:

Port definitions added for each created system actor part
Port definition <actor-name>Port with the corresponding conjugated port definition <actor-name>Port under it.

Ports added to system_Physical part that represents connection to the related system actor parts
Ports named p<Actor-name with feature typing relationship to the port definition <actor-name>Port.

Interface relationships between system_Functional part with the related system actor parts
Interface between system actor parts pSystem port system_Functional part, and <actor-name>Port port.

Port definitions for each interacting physical parts
Port definition <PhysicalPart1_PhysicalPart2>Port with the corresponding conjugated port definition under it.

Ports added to the physical parts that interacts with the system actors
Ports named p<Actor-name> with feature typing relationship to the port definition <actor-name>Port.

Ports added to the interacting physical parts
Ports named p<Other PhysicalPart> with feature typing relationship are added to the port definition < PhysicalPart1_PhysicalPart2>Port.

Interface relationships
Interface between port p<Subsystem2> subsystem, subsystem1 and port p<Subsystem1>, subsystem subsystem2.

system context view
system context view in 4_PhysicalArchitecturtePkg is populated with the created ports and interfaces.

Handoff to downstream engineering

You can refine system engineering data for downstream teams by organizing it into separate models. You can also collaborate with engineering teams to design a deployment architecture for each subsystem, and allocate the refined data accordingly, ensuring it is in a usable format for effective implementation and integration.

Consider the following information for the purpose of the handoff to downstream engineering:

- Refine the system engineering data to a form usable by downstream engineers.
- Create separate models to hold the prepared engineering data in a convenient organizational format.
- For each subsystem, work with downstream engineering teams to create a deployment architecture and allocate system engineering data into that architecture.

Logical Handoff operation (subsystem to system)

Logical Handoff operation is available in the contextual menu of a selected logical subsystem part. It creates a separate project of the standard harmony structure, where the selected subsystem becomes the main system.

Based on the interfaces of the selected subsystem and its behavior, the content of the new project is populated with the following elements:

- Four top-level packages that represent the main steps of Harmony process:
  - 1_RequirementsAnalysisPkg
  - 2_FunctionalAnalysisPkg
  - 3_LogicalArchitecturePkg
  - 4_PhysicalArchirecture
- Use cases named <use-case-name> for the <selected-subsystem-name>
- System actors that include the system actors that interact with the subsystem and other subsystems that interact with the subsystem become system actors.
- Bindings of use cases and system actors.
- use case view, which is populated with the related use cases and system actors.
- system context view, which is populated with the related system actor parts and interfaces to the system.
- 2_FunctionalAnalysisPkg includes the following items:
  - Use cases and system actors as in 1_Requirement.
  - Use case behavior, which is the subset of the original use case action flow view associated with the subsystem.
  - Port definitions that are used in the subsystems' ports.
  - Item definitions that are used in the selected port definitions.
  - Ports of the system part and system actors.
  - Interfaces between system actors and the system part.
  - system context view populated with the related system actor parts and interfaces to the system.
- 3_LogicalArchitecturePkg includes the following:

– System part <subsystem-name>_logicalSystem
– Subsystem behavior state transition view

Requirement tracing in subsystems

Subsystem handoff projects include only those requirements that are traced to the selected subsystem from the parent system model. The relevant system and stakeholder requirements are only included. You can define trace relationships by using `custom`, `trace` and `satisfy` links from the Harmony library. All external subsystems are automatically modeled as system actors. The use cases that apply to the subsystem are also traced directly to the corresponding requirements. These requirements are further linked by using `Satisfied By` relationships to specific functions, actions, and inputs, providing full traceability from high-level requirements to implementable model elements.

Using the Harmony library

When you create a project using the Harmony template, Harmony library is automatically included.

Procedure

1. Create a new project by using Harmony template. For more information, see "Creating and archiving projects" on page 78.
   A new project is created and the Harmony library is automatically included.
2. In the Browser panel, locate the Harmony library.
3. Expand the library to review its predefined modeling elements.
   - System actors
   - Trace relationships
   - General requirements
   - Stakeholder requirements
   - System requirements
4. Work with the system context view.
   a) In your newly created project, go to the system context view. It is a custom diagram created by using the Harmony library.
   b) Observe that system actors are preloaded into the view based on the template.
5. Create and trace requirements.
   a) Create a new requirement.
   b) Select a requirement type from the library, for example, stakeholder requirement. A Priority field is automatically available.
   c) Assign a value to the priority.
   d) Use **Trace** to create links between:
      - Requirements and use cases
      - Requirements and system elements

Accessibility

Accessibility features assist users who have a disability, such as restricted mobility or limited vision, to use information technology successfully.

Overview

Rhapsody Systems Engineering includes the following accessibility features:

- Users can use the product by using a keyboard.

- The Rhapsody Systems Engineering online product documentation is available in IBM Documentation, which is viewable in a standard web browser.

Keyboard navigation

Rhapsody Systems Engineering uses the following keyboard shortcuts.

Table 1. Keyboard shortcuts for projects

Action	Windows shortcuts	Mac Shortcuts
Save project	`Ctrl + s`	`Cmd + s`
Undo	`Ctrl + z`	`Cmd + z`
Redo	`Ctrl + y`	`Cmd + y`
Toggle properties panel	`p`	`p`
Toggle explorer panel	`m`	`m`
Add element	`a`	`a`

IBM and accessibility

For more information about IBM's commitment to accessibility and to view Accessibility Conformance Reports for IBM products, see IBM Accessibility.

Trial

Rhapsody Systems Engineering is available in a trial mode, allowing you to evaluate the features without a license for a limited period of time. During the trial, Rhapsody Systems Engineering operates without restrictions.

Getting started with trial mode

Using Rhapsody Systems Engineering in a trial mode involves requesting for a trial token from your sales representative, applying it by using a API, and restarting the application to begin the trial. Once the trial is over, you can also extend the trial contact and request for a token for a longer period of time.

Procedure

1. Contact your sales representative, and request a trial token.
2. As an administrator, apply the token to your running Rhapsody Systems Engineering installation by using the `add-trial-token` API. If the token is accepted, the response will contain the field `"active": true`.
3. Restart Rhapsody Systems Engineering.
4. To verify that Rhapsody Systems Engineering is in trial mode, use your browser to log in to Rhapsody Systems Engineering and click the information icon in the top toolbar. If you are in trial mode, the message "Trial ends on YYYY/MM/DD" will be displayed next to the Rhapsody Systems Engineering build version number.

Once the trial is over the home page displays the following error message: "The Rhapsody Systems Engineering trial has ended. It expired on YYYY/MM/DD". The Rhapsody Systems Engineering APIs will no longer work and will return a similar error message.

5. To extend a trial contact, request a trial token for a longer period of time. Repeat the above steps to apply the token. You can extend your trial before it expires. Once the trial is extended, all projects will be accessible and you can continue from where you have left off.

Ending your trial

You can end your trial of Rhapsody Systems Engineering and apply for a license to continue using it.

Procedure

1. As an administrator , use the `remove-trial-token` API to end your trial. All tokens are removed and Rhapsody Systems Engineering checks for a license and operates normally if one is found.
2. Restart Rhapsody Systems Engineering.

   All projects data is preserved allowing users to continue from where they left off.

Getting started

Before you can use Rhapsody Systems Engineering, learn how to log in to the application and perform initial tasks as an administrator or a user.

Getting started for administrators

As an administrator, after you plan your deployment and install Rhapsody Systems Engineering on your system, you can get started by creating projects, inviting users to work on them, setting up the Open Services for Lifecycle Collaboration (OSLC) integrations, and establishing project associations. You can also set up project usages, if you want to share libraries between projects.

Before you begin

To plan your deployment and install Rhapsody Systems Engineering, see “Planning” on page 17 and “Installing” on page 26.

Procedure

- To create projects, see “Creating and archiving projects” on page 78.
- To add users to projects, see “Managing users” on page 77.
- To set up OSLC integrations, see “Managing open services for lifecycle collaboration integrations” on page 81.
- To associate projects of Rhapsody Systems Engineering and the projects of other friend applications, see “Creating associations between projects” on page 79.
- To set up project usages, see “Creating project usages” on page 80.

Related concepts

“Roles” on page 77
A user role is defined by a set of tasks that a specific role is given permission to perform. In Rhapsody Systems Engineering, the three predefined roles are: server administrator, project administrator, and practitioner.

“Installing” on page 26
Rhapsody Systems Engineering offers a supported image and operator that allows you to create and configure a production-ready Rhapsody Systems Engineering instance on Red Hat OpenShift Container Platform or Kubernetes cluster. The certified containers of Rhapsody Systems Engineering deliver curated deployment patterns through operator and related tooling that you can configure and deploy on container orchestration platforms.

Getting started for users

You can start working on Rhapsody Systems Engineering projects after your project or server administrator adds you as a new user. You are either added as a project administrator or as a practitioner.

Procedure

1. Share your email address used for your OIDC authentication setup with your project or server administrator.
2. Log in to Rhapsody Systems Engineering by using your OIDC credentials.
3. After you are invited to a project by your project or server administrator, you can start working on it. For more information, see "Managing projects" on page 90.

Related concepts

"Overview of IBM Rhapsody Systems Engineering" on page 1
IBM® Rhapsody® Systems Engineering is a web-based solution designed for systems engineering teams, enabling them to create smarter, more sophisticated, and more competitive solutions for their end users. It transforms increasing design complexities into a competitive advantage.

"Roles" on page 77
A user role is defined by a set of tasks that a specific role is given permission to perform. In Rhapsody Systems Engineering, the three predefined roles are: server administrator, project administrator, and practitioner.

Planning

Before you install Rhapsody Systems Engineering, prepare your environment by installing command line tools for Red Hat OpenShift or Kubernetes cluster, setting up the environment for the supported cluster, and then configuring the catalog source.

About this task

- The connection between the client and the ingress must always use `https` instead of `http`.
- Supported architecture is Intel or AMD.
- A cluster administrator role is required.

For more information about prerequisites, see Software Product Compatability Reports.

You can deploy Rhapsody Systems Engineering on any of the following environments:

- Red Hat OpenShift Container Platform (on-premises)
- Red Hat OpenShift on IBM Cloud
- IBM Cloud Kubernetes Services
- Other OpenShift or Kubernetes services in private clouds, or on-premises

The Rhapsody Systems Engineering operator is a cluster-scoped operator, which deploys applications to the installed namespace. External and optional dependencies used by the Rhapsody Systems Engineering application is managed by the end user. The operator only manages the Rhapsody Systems Engineering application workload and related Kubernetes resources.

Review the following information to plan for the multiple zone clusters for your Rhapsody Systems Engineering instance.

With multiple zone clusters, Red Hat OpenShift and Kubernetes automatically spreads the pods in a deployment or service across zones to reduce the impact of failures. For multiple zone clusters, well-known labels are applied to nodes to help indicate the topology to Red Hat OpenShift and Kubernetes. Rhapsody Systems Engineering relies on Red Hat OpenShift or Kubernetes capabilities to schedule the

pods across the zones. For more information about Kubernetes guidance on applying well-known labels for multiple zone clusters, go to Well-known Labels, Annotations, and Taints.

Preparing to install Rhapsody Systems Engineering operator

Before you install Rhapsody Systems Engineering operator, install the command line for the clusters. If you use Red Hat OpenShift platform cluster, create projects in the Red Hat OpenShift Container Platform and if you use Kubernetes cluster, install operator lifecycle manager and create namespaces.

Preparing to install Rhapsody Systems Engineering operator by using Red Hat OpenShift

Before you use Red Hat OpenShift cluster, create projects in the Red Hat OpenShift console.

Procedure

1. Install the Red Hat OpenShift command line (oc). For more information, see Installing the Red Hat OpenShift CLI by downloading the binary.
2. Create projects by using Red Hat OpenShift console or by command line.
   - If you use Red Hat OpenShift console, complete the following steps:
     a. Log in to the Red Hat OpenShift Container Platform by using the Red Hat OpenShift administrator credentials.
     b. Choose one of the following ways to create a project.
        – Go to **Projects**, and click **Create project**.
        – Go to **Operators** > **OperatorHub**, and then click **Create Project** from the **Project** list.
     c. In the **Create Project** dialog box, enter the name of the project in the **Name** field.
     d. In the **Display Name** and **Description** fields, enter the project display name and description.

        Note: Follow the naming conventions while specifying the project name. Use lowercase and do not use special symbols in the name.
     e. Click **Create**.
   - If you use command line, use the following command to create projects.

```
Create a new project with a display name and description
  oc new-project web-team-dev --display-name="Web Team Development" --
description="Development project for the web
team."

Usage:
  oc new-project NAME [--display-name=DISPLAYNAME] [--description=DESCRIPTION] [flags]
[options]
```

Preparing to install Rhapsody Systems Engineering operator by using Kubernetes cluster

Before you install Rhapsody Systems Engineering operator in the Kubernetes cluster, install operator lifecycle manager and create namespaces.

Procedure

1. Install Kubernetes command line (kubectl). For more information, see kubectl.
2. Install operator lifecycle manager.
   a) Install the Operator SDK.
   b) Log in to your Kubernetes environment.

c) Run the following command to install operator lifecycle manager in your cluster:

```
operator-sdk olm install
```

For more information about the output, see Installing Operator Lifecycle Manager in your cluster.

3. Create a new namespace to install the operator by using the following command:

```
kubectl create namespace <operator-namespace>
```

Preparing and configuring the catalog sources

Before you install Rhapsody Systems Engineering, the administrator must create the catalog source for Red Hat OpenShift cluster or IBM Kubernetes service cluster that registers the Rhapsody Systems Engineering operator. Configuring the Rhapsody Systems Engineering operator catalog source includes creating and applying a YAML file with the appropriate image and namespace details, using the command line or Red Hat OpenShift console to set up the catalog source, and verifying its installation.

Preparing and configuring the catalog source for Red Hat OpenShift

Configuring the Rhapsody Systems Engineering operator catalog source by using Red Hat OpenShift console includes setting up the catalog source and verifying its installation.

Procedure

1. Prepare the catalog source manifest.

a) From your local computer, create a `catalog_source.yaml` file with the following content.

```
apiVersion: operators.coreos.com/v1alpha1
kind: CatalogSource
metadata:
  name: ibm-rse-catalog
  namespace: openshift-marketplace
spec:
  displayName: IBM RSE Catalog
  # For the image name, see the following catalog source image names table and use the
appropriate value.
  image: '<image_name>'
  publisher: IBM
  sourceType: grpc
  updateStrategy:
    registryPoll:
      interval: 10m
```

b) Use the image names from the following table to update <image_name> in your YAML file.

Edition	Image name
Rhapsody Systems Engineering software	`cp.icr.io/cpopen/ibm-rse-operator-catalog:latest`

2. Configure the catalog source manifest by Red Hat OpenShift console or command line.

- To configure the catalog source by using Red Hat OpenShift console, complete the following steps:
  a. Log in to the Red Hat OpenShift console.
  b. Click the **Import YAML** icon.
  c. In the text area, paste the contents of the catalog source manifest prepared in “1.a” on page 19
  d. Update the namespace to `openshift-marketplace`.
  e. Click **Create**.
- To configure the catalog source by command line, complete the following steps.

a. Log in to the Red Hat OpenShiftContainer Platform. Run the **`oc cluster-info`** sub-command to verify that the correct cluster information is displayed.

b. Run the following command:

```
oc apply -f catalog_source.yaml -n <catalog_source_namespace>
```

c. Verify the installation by running this command:

```
oc get CatalogSources ibm-rse-catalog -n <catalog_source_namespace>
```

– If the installation was successful, the output appears similar to this.

Name	Display	Type	Publisher	Age
ibm-rse-catalog	IBM RSE Catalog	grpc	IBM	50s

Note: If the installation fails, see "Fixing the issues in preparing and configuring the catalog sources by using Red Hat OpenShift console " on page 149 for resolution.

The YAML file is imported, and it might take some time for the catalog to connect and reach a ready state.

What to do next

You can start installing the operator on either Red Hat OpenShift cluster or on Kubernetes Service. For more information, see "Installing" on page 26.

Preparing and configuring the catalog source for Kubernetes cluster

To set up Kubernetes Service cluster and support installation of operators, connect a command line environment to the cluster. The IBM Kubernetes administrator must create the catalog source that registers the Rhapsody Systems Engineering operator. After the registration, you can install the operator by using the command line.

About this task

Note: The `olm` namespace is created and managed by the operator lifecycle manager to run core OLM components such as:

- `olm-operator`
- `catalog-operator`
- `packageserver`

Installing Rhapsody Systems Engineering operator-related resources, such as `CatalogSource`, `Subscription`, or `OperatorGroup` into the `olm` namespace can interfere with OLM functionality, cause installation failures, or lead to unpredictable behavior. Create a namespace to perform the following procedure.

Procedure

1. Prepare the catalog source manifest. From your local computer, create `catalog_source.yaml` by using the following content.

```
apiVersion: operators.coreos.com/v1alpha1
kind: CatalogSource
metadata:
    name: ibm-rse-catalog
    namespace: <catalog_source_namespace>
spec:
    image: cp.icr.io/cpopen/ibm-rse-operator-catalog:latest
    sourceType: grpc
```

```
        updateStrategy:
            registryPoll:
                interval: 10m
```

Note: The updateStrategy must be adjusted to the desired frequency for the operator to check the registry for future updates. In the example, the operator checks for new updates every 10 minutes. To minimize the frequency of checks and reduce resource utilization, you can set the interval to a higher value if required.

2. Configure the catalog source by using command line.
   a) Run the following command. For *<catalog_source_namespace>*, enter the name of the catalog source namespace created for the Rhapsody Systems Engineering operator. If your catalog source namespace is not available, you can create a new namespace before you apply it.

```
kubectl apply -f catalog_source.yaml -n <catalog_source_namespace>
```

   b) Verify the installation by running the following command:

```
kubectl get CatalogSources ibm-rse-catalog -n <catalog_source_namespace>
```

   - If the installation was successful, the output is similar to this

Name	Display	Type	Publisher	Age
ibm-rse-catalog	IBM RSE Operator Catalog	grpc	IBM	50s

   - If the installation fails, see “Fixing the issues in preparing and configuring the catalog sources by using the Kubernetes cluster ” on page 150 for resolution.

The YAML file is imported, and it might take some time for the catalog to connect and reach a ready state.

What to do next

You can start installing the operator on either Red Hat OpenShift cluster or on Kubernetes Service. For more information, see “Installing” on page 26.

Planning for database configuration

Rhapsody Systems Engineering supports Postgres. Rhapsody Systems Engineering operator does not manage a database. It needs to be setup manually.

Procedure

1. Ensure that database and Rhapsody Systems Engineering operator cluster must be in the same network.
2. Create a database for Rhapsody Systems Engineering.

User roles mapping in Rhapsody Systems Engineering

Red Hat OpenShift cluster uses an identity provider to validate the usernames and passwords against an LDAPv3 server by using simple bind authentication.

You can set up the an identity provider by using Configuring an LDAP identity provider. In the Red Hat OpenShift cluster, you can define and apply the permissions at cluster and project levels and bind the roles by using Role-based access control (RBAC).

The following table lists various operations that are processed in the Red Hat OpenShift cluster and required user roles. In the table, the letter "X" indicates the required user role for the operation.

Table 2. Rhapsody Systems Engineering operations and required access roles						
	Cluster level			**Local level**		
	cluster-admin	admin	edit	admin	edit	view
Create the catalog source for the Rhapsody Systems Engineering operator	✔	X	X	X	X	X
Create a project in the Red Hat OpenShift cluster	✔	X	X	X	X	X
Install the Rhapsody Systems Engineering operator	✔	X	X	X	X	X
Uninstall the Rhapsody Systems Engineering operator	✔	X	X	X	X	X
Create the Rhapsody Systems Engineering instance	✔	✔	✔	✔	✔	X
Modify the Rhapsody Systems Engineering instance	✔	✔	✔	✔	✔	X
Delete the Rhapsody Systems Engineering instance	✔	✔	✔	✔	✔	X

Note:

- A user with an *admin* or *edit* role at the Red Hat OpenShift and Kubernetes project level can change the Rhapsody Systems Engineering operator subscription channel and manage the Rhapsody Systems Engineering operator version only by using the command line.
- A user with an *admin* or *edit* role at the project level can delete only the Rhapsody Systems Engineering operator subscription, not the operator group by using uninstall Rhapsody Systems Engineering operator or delete `ClusterServiceVersion` operations.

- A workaround is available to install or uninstall the Rhapsody Systems Engineering operator by a user with an *admin* role that is assigned to the project. For more information about the workaround, see Configure RBAC in Different Stages so End-users can Interact with OperatorHub in the Console.

Viewing the operator custom resource status

Common custom resource status allows clients to interact with custom resources both manually and programmatically. A clear and standard set of custom resource status allows a static and single location for understanding the endpoints that the operator is managing. Inconsistent custom resource status creates confusion and needs extra development time to programmatically interact with custom resources.

About this task

Common custom resource status allows clients to interact with custom resources both manually and programmatically. A clear and standard set of custom resource status allows a static and single location for understanding the endpoints that the operator manages. Inconsistent custom resource status creates confusion and requires extra development time to programmatically interact with custom resources.

- Rhapsody Systems Engineering apps instance
- Rhapsody Systems Engineering client instance
- Rhapsody Systems Engineering license client instance
- Rhapsody Systems Engineering Rest API server instance
- Rhapsody Systems Engineering server instance

Procedure

1. Get the status of the Engineering Lifecycle Management instance.
   - By using the Red Hat OpenShift Container Platform web console method.
     a. Go to the namespace where Rhapsody Systems Engineering is installed by using **Home** > **Projects** > **Your Project Name**.
     b. Click **Operators**, and select **Operators** > **Installed Operators** > **Provided APIs** > **RhapsodySEApps**.
   - By using the command line method
     – **Red Hat OpenShift**

```
oc get rhapsodyseapp <rhapsodyseapps_instance_name> -o custom-
columns="Name:.metadata.name, Status:.status.conditions[?(@.status=='True')].type"
-n <namespace>
```

     Kubernetes

```
kubectl get rhapsodyseapp <rhapsodyseapps_instance_name> -o custom-
columns="Name:.metadata.name, Status:.status.conditions[?(@.status=='True')].type"
-n <namespace>
```

   The following table explains different custom resource statuses for Engineering Lifecycle Management instance.

Condition	Type	Status scenario
SharedStorageReady	`True`	PersistentVolumeClaim is bound
ApplicationReady	`True`	The application has been initialized and is ready for service

2. Get the status of the Rhapsody Systems Engineering client instance.

By using the Red Hat OpenShift Container Platform web console method

a. Go to the namespace where Rhapsody Systems Engineering is installed by using **Home** > **Projects** > **Your Project Name**.

b. Click **Operators**, and select **Operators** > **Installed Operators** > **Provided APIs** > **RhapsodySEApps**.

By using command line

Consider the custom resource statuses for Rhapsody Systems Engineering client instance.

Red Hat OpenShift

```
oc get client <rhapsodyseapp_client_instance_name> -o custom-
columns="Name:.metadata.name, Status:.status.conditions[?(@.status=='True')].type" -n
<namespace>
```

Kubernetes

```
kubectl get client <rhapsodyseapp_client_instance_name> -o custom-
columns="Name:.metadata.name, Status:.status.conditions[?(@.status=='True')].type" -n
<namespace>
```

3. Consider the custom resource statuses for Rhapsody Systems Engineering license client instance.

Red Hat OpenShift command line

```
oc get licenseclient <rhapsodyseapp_licenseclient_instance_name> -o custom-
columns="Name:.metadata.name, Status:.status.conditions[?(@.status=='True')].type" -n
<namespace>
```

Kubernetes

```
kubectl get licenseclient <rhapsodyseapp_licenseclient_instance_name> -o custom-
columns="Name:.metadata.name, Status:.status.conditions[?(@.status=='True')].type" -n
<namespace>
```

4. Consider the custom resource statuses for Rhapsody Systems Engineering Rest API server instance.

Red Hat OpenShift command line

```
oc get restapiserver <rhapsodyseapp_restapiserver_instance_name> -o custom-
columns="Name:.metadata.name, Status:.status.conditions[?(@.status=='True')].type" -n
<namespace>
```

Kubernetes

```
kubectl get restapiserver <rhapsodyseapp_restapiserver_instance_name> -o custom-
columns="Name:.metadata.name, Status:.status.conditions[?(@.status=='True')].type" -n
<namespace>
```

5. Consider the custom resource statuses for Rhapsody Systems Engineering server instance.

Red Hat OpenShift command line

```
oc get server <rhapsodyseapp_server_instance_name> -o custom-
columns="Name:.metadata.name, Status:.status.conditions[?(@.status=='True')].type" -n
<namespace>
```

Kubernetes

```
kubectl get server <rhapsodyseapp_server_instance_name> -o custom-
columns="Name:.metadata.name, Status:.status.conditions[?(@.status=='True')].type" -n
<namespace>
```

Operator versioning

The operator versioning strategy outlines the Rhapsody Systems Engineering operator versions mapping to the Rhapsody Systems Engineering operand versions.

Operator version

A Kubernetes-native method to deploy, manage, and maintain a complex application. The Rhapsody Systems Engineering operator is responsible for orchestrating the deployment and lifecycle of the Rhapsody Systems Engineering application.

It is a specific version of the operator software. Operators are developed and maintained independently of the applications or by the services that they manage. Operator versions include new features, bug fixes, performance enhancements, or improvements to the operator with its operands.

The pattern that is used to define the versions of operator uses the Semantic Versioning model (Server). Version releases depend on their inclusion of new features or fixes. A parallel stream of Rhapsody Systems Engineering operators is added to cater to the current products and the previous releases.

Operand

The application managed by the operator, in this case, the Rhapsody Systems Engineering itself.

It is the specific version of the Rhapsody Systems Engineering, such as Rhapsody Systems Engineering, version 1.2.1.

Channels

Channels provide a structured mechanism for controlling the rollout and lifecycle of operator versions in Kubernetes environments. In the context of the Rhapsody Systems Engineering operator, a channel represents a defined upgrade path that groups specific operator versions which are compatible with a corresponding series ofRhapsody Systems Engineering (operand) versions.

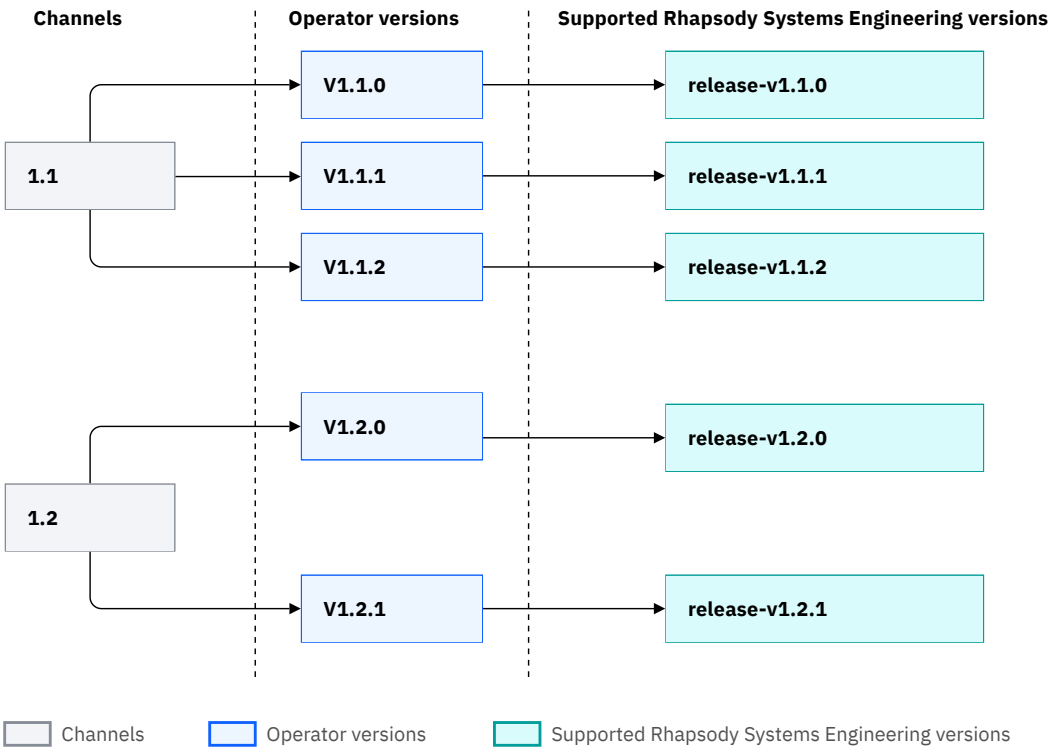
Each channel typically aligns with a `major.minor` version of the Rhapsody Systems Engineering product, for example, 1.1.0, 1.2.0. Within a channel, operator versions are updated as needed to include patches, enhancements, or security fixes, all while maintaining compatibility with a specific set of operand versions. This allows users to receive updates and bug fixes within a controlled compatibility range, without risking unintended changes from unrelated feature updates or breaking changes.

By selecting a particular channel during operator installation, cluster administrators can ensure the following:

- The operator supports only the Rhapsody Systems Engineering versions relevant to their deployment requirements.
- Upgrades follow a predictable and tested path.
- Stability and version alignment are maintained across environments.

Channels are especially useful in regulated or enterprise environments where consistency and control over application lifecycles are critical.

The following diagram represents the channels, operator versions, and their supported Rhapsody Systems Engineering versions for a scenario in which Rhapsody Systems Engineering 1.1.0 is released.



This structure ensures that each channel supports a compatible sequence of Rhapsody Systems Engineering versions, enabling safe and controlled upgrades.

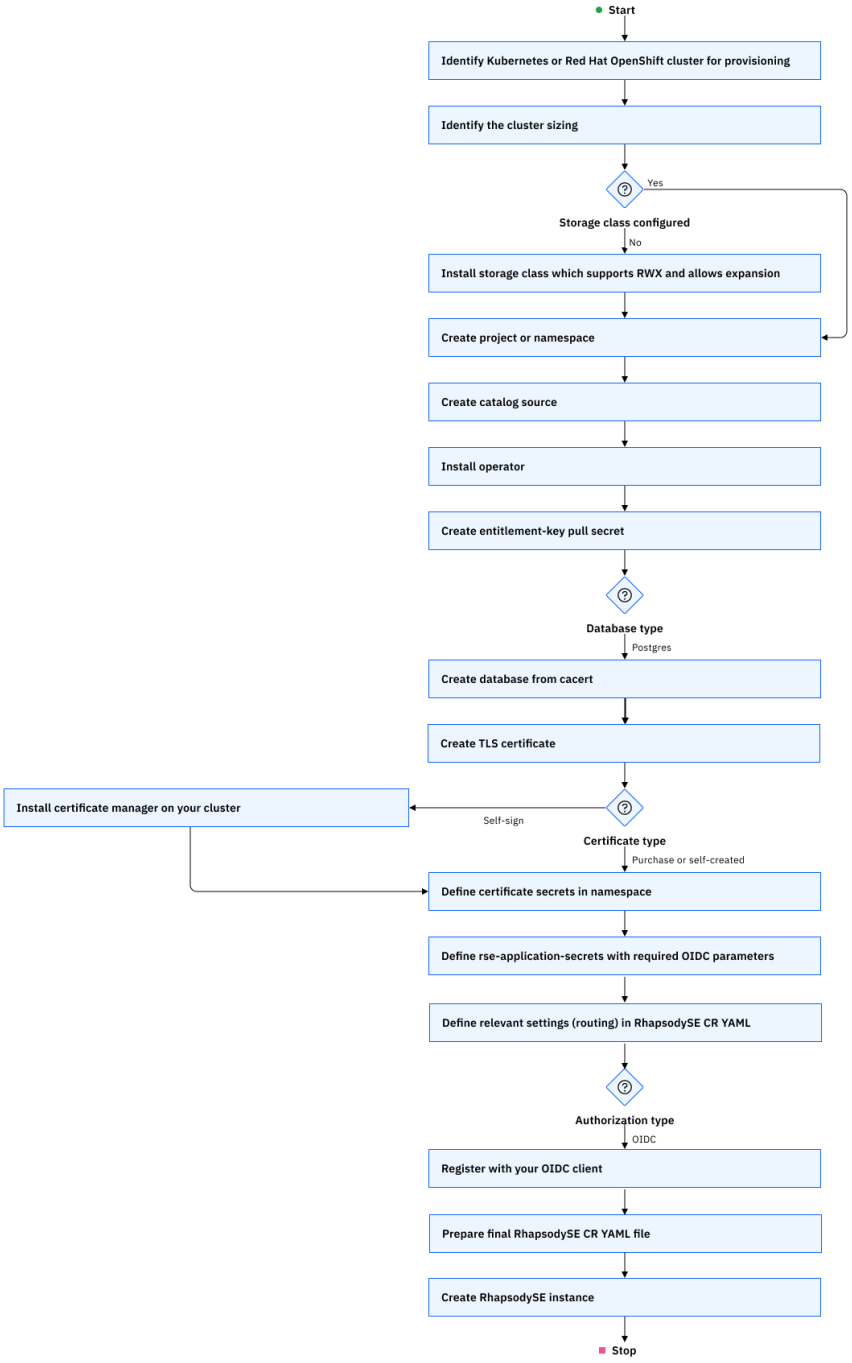
Installing

Rhapsody Systems Engineering offers a supported image and operator that allows you to create and configure a production-ready Rhapsody Systems Engineering instance on Red Hat OpenShift Container Platform or Kubernetes cluster. The certified containers of Rhapsody Systems Engineering deliver curated deployment patterns through operator and related tooling that you can configure and deploy on container orchestration platforms.

Note: Consider the following limitations:

- Air-gapped installations are not supported.
- Installation of Rhapsody Systems Engineering using Helm charts is not supported.

Consider the following installation flow diagram.



Installing Rhapsody Systems Engineering operator on a Red Hat OpenShift cluster

Installing Rhapsody Systems Engineering operator on a Red Hat OpenShift cluster includes installing the operator by using the Red Hat OpenShift console or command line.

Before you begin

Ensure that the Red Hat OpenShift cluster is configured. For more information, see “Preparing and configuring the catalog source for Red Hat OpenShift ” on page 19.

Installing the Rhapsody Systems Engineering operator using Red Hat OpenShift console

Installing Rhapsody Systems Engineering using the Red Hat OpenShift console includes configuring the catalog source, verifying the catalog source pod status, and then using the OperatorHub to install the operator.

Before you begin

Configure the catalog source. For more information, see “Preparing and configuring the catalog sources” on page 19.

Procedure

1. Log in to the Red Hat OpenShift Container platform console.
2. Verify that the catalog source pod is running by completing the following steps:
   a) Open your project.
   b) Go to **Workloads** > **Pods**. Check if the pod is running. The pod name is similar to the name that you provided in the input for catalog source. For example: `ibm-rse-catalog` or `ibm-operator-catalog`.
   c) Switch to **openshift-marketplace** project.
3. From the context selector in the navigation menu, click **Operators** > **OperatorHub**. The OperatorHub page opens.
4. To look for the Rhapsody Systems Engineering software operator, enter full or partial keywords into the search textbox. For example, enter Rhapsody Systems Engineering, and then click the tile.
5. Click **Install**. The Install Operator page opens.
   a) From the **Installed Namespace** list, select the required namespace.
   b) Select the **Update approval strategy** as manual or automatic.
6. Click **Install**. The Rhapsody Systems Engineering software operator is installed.
7. To view the details of Rhapsody Systems Engineering software operator, click **View Operator**.

   You can start creating the Rhapsody Systems Engineering instance. For more information, see “Creating Rhapsody Systems Engineering instance” on page 31.

Installing the Rhapsody Systems Engineering operator using command line

Installing Rhapsody Systems Engineering operator by using command line includes configuring the catalog source, creating an OperatorGroup and subscription, and reviewing the installation status of the operator.

Procedure

1. Configure the catalog source. For more information, see “Preparing and configuring the catalog source for Red Hat OpenShift ” on page 19.
2. Log in to the Red Hat OpenShift container platform or Kubernetes cluster. Ensure to run the `oc cluster-info` sub-command to verify that the correct cluster information is displayed.
3. Run the following commands to verify that the catalog source has an operator entry available to be installed.
   - `oc get packagemanifests -n openshift-marketplace | grep rhapsody`
   - `oc describe packagemanifests <operator name from previous command> -n openshift-marketplace`
4. If you are using a cluster other than Red Hat OpenShift Kubernetes cluster, create an OperatorGroup for the required scope (cluster or one or more namespaces), by completing the following steps.

a) Create an OperatorGroup yaml file for the required scope as shown in the following sample snippets.

```
# Operator group with cluster scope
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: rse-operator-global
  namespace: <namespace>
spec: {}
```

```
# Operator group with namespace scope
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: rse-operator-local
  namespace: <namespace>
spec:
  # Provide one or more namespaces which will be monitored by the RSE Operator.
  targetNamespaces:
  - <target_namespace1>
  - <target_namespace2>
```

b) Run the following command to create the OperatorGroup.

```
oc create -f <OperatorGroup>.yaml -n <namespace>
```

5. Create a subscription yaml as shown in the following sample snippet. Ensure that the source name should match the catalog source name that you set in “Preparing to install Rhapsody Systems Engineering operator by using Red Hat OpenShift ” on page 18.

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: rhapsody-se-operator
  namespace: <namespace>
spec:
  channel: "1.5"
  installPlanApproval: Automatic
  name: rhapsody-se-operator
  source: ibm-rse-catalog
  sourceNamespace: <CatalogSource_namespace>
```

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: rhapsody-se-operator
  namespace: <namespace>
spec:
  channel: "1.8"
  installPlanApproval: Automatic
  name: rhapsody-se-operator
  source: ibm-rse-catalog
  sourceNamespace: <CatalogSource_namespace>
```

6. Run the following command to run the subscription.

```
oc create -f <Subscription>.yaml -n <namespace>
```

7. View the installation status of operator by running the following command.

```
oc describe sub <subscription_name> -n <namespace>
```

8. Verify the version of the installed Rhapsody Systems Engineering operator by running the following command.

```
oc get sub <subscription_name> -o jsonpath="{.status.installedCSV}" -n <namespace>
```

You can start creating the Rhapsody Systems Engineering instance. For more information, see “Creating Rhapsody Systems Engineering instance” on page 31.

Installing Rhapsody Systems Engineering operator on Kubernetes Service

The Rhapsody Systems Engineering operator adheres to the Kubernetes operator pattern, which simplifies the management of Rhapsody Systems Engineering software environment through specialized custom resources (CRs). The primary CR, RhapsodySEApp serves as the main configuration point for setting up an Rhapsody Systems Engineering software environment.

Before you begin

Ensure that the Kubernetes cluster is configured. For more information, see "Preparing and configuring the catalog source for Kubernetes cluster" on page 20.

About this task

Installing Rhapsody Systems Engineering operator on IBM Kubernetes Service includes configuring the IBM Kubernetes Service, and installing the operator on IBM Kubernetes Service by using command line inputs.

The operator lifecycle manager (OLM) is used to install the operator to the IBM Kubernetes Service cluster. It helps the users to install, update, and manage the lifecycle of an operator and its associated services that run across the cluster.

Note: The `olm` namespace is created and managed by the operator lifecycle manager to run core OLM components such as:

- `olm-operator`
- `catalog-operator`
- `packageserver`

Installing the Rhapsody Systems Engineering operator-related resources, such as `CatalogSource`, `Subscription`, or `OperatorGroup` into the `olm` namespace can interfere with OLM functionality, cause installation failures, or lead to unpredictable behavior. Create a namespace to perform the following procedure.

Procedure

1. Switch to the namespace, where you want to install the operator by using the following command:

   ```
   kubectl config set-context --current –-namespace=<operator-namespace>
   ```

2. Create an OperatorGroup similar to the following example.

   ```
   apiVersion: operators.coreos.com/v1
   kind: OperatorGroup
   metadata:
     name: rhapsody-operator-group
     namespace: <namespace>
   spec: {}
   ```

   a) Save to a file called *`rse-operator-group.yaml`*, and run the following command to apply it.

   ```
   kubectl apply -f rse-operator-group.yaml
   ```

3. Create a subscription for the operator.

   a) Create a YAML file similar to the following example:

   ```
   apiVersion: operators.coreos.com/v1alpha1
   kind: Subscription
   metadata:
   ```

```
        name: rhapsody-se-operator
        namespace: <namespace>
    spec:
        channel: "1.5"
        installPlanApproval: Automatic
        name: rhapsody-se-operator
        source: ibm-rse-catalog
        sourceNamespace: <CatalogSource_namespace>
```

```
    apiVersion: operators.coreos.com/v1alpha1
    kind: Subscription
    metadata:
        name: rhapsody-se-operator
        namespace: <namespace>
    spec:
        channel: "1.8"
        installPlanApproval: Automatic
        name: rhapsody-se-operator
        source: ibm-rse-catalog
        sourceNamespace: <CatalogSource_namespace>
```

b) Save to a file called `subscription.yaml`, and run the following command to apply the change.

```
kubectl apply -f subscription.yaml
```

4. To verify that an operator is installed, run the following command:

```
kubectl get csv
```

The command returns a list similar to the following example.

NAME	DISPLAY	VERSION	REPLACES	PHASE
`rhapsody-se-operator.v1.5.0`	Rhapsody Systems Engineering	1.5.0		Succeeded

NAME	DISPLAY	VERSION	REPLACES	PHASE
`rhapsody-se-operator.v1.8.0`	Rhapsody Systems Engineering	1.8.0		Succeeded

Creating Rhapsody Systems Engineering instance

You can create Rhapsody Systems Engineering instance on the supported cluster.

Note: Do not create the RhapsodySE instance in the operator namespace. Create a dedicated namespace and use it to deploy the RhapsodySE instance to ensure better separation of concerns and easier management. Additionally, ensure that the application secret, image pull secret, and TLS secrets are created within this new namespace.

Preparing to create Rhapsody Systems Engineering instance

Before you create Rhapsody Systems Engineering instance, gather your OpenID Connect (OIDC) settings for authentication, install IBM Common Licensing server, install the supported database, and ensure that you have access to a supported cluster. You can create more namespaces to create Rhapsody Systems Engineering instance.

About this task

Note: Do not create the RhapsodySE instance in the operator namespace. Create a dedicated namespace and use it to deploy the RhapsodySE instance to ensure better separation of concerns and easier

management. Additionally, ensure that the application secret, image pull secret, and TLS secrets are created within this new namespace.

Related information

Software Product Comparability Reports

Authenticating with OIDC

Rhapsody Systems Engineering uses OpenID Connect (OIDC) for secure authentication. OIDC is a secure authentication protocol that uses OAuth 2.0 to provide an identity layer, allowing clients to verify a user's identity based on the authentication performed by an authorization server.

Before you begin

To configure authentication for the product, ensure that you have access to an OIDC-capable authentication provider. This might be an internal provider within your organization or a third-party service that supports OIDC. For more information about how to configure OIDC authentication in Rhapsody Systems Engineering, see “Creating and configuring the instance” on page 41.

Note: For IBM users, w3ID is the OIDC authentication provider used within IBM. If you deploy the product internally within IBM, configure your authentication settings to use w3ID.

Note: If you are planning to link Rhapsody Systems Engineering elements with artifacts of other applications, for example, Engineering Lifecycle Management, configure Rhapsody Systems Engineering to use an OIDC provider that generates access token less than 250 characters.

Related concepts

“OIDC authentication” on page 71
Rhapsody Systems Engineering supports OIDC authentication with providers, such as IBMid authentication, Microsoft Entra ID, Keycloak, and Jazz authorization server (JAS).

Configuring IBM Common Licensing server

Configure the IBM Common Licensing Server so that users can check out the tokens before they analyze the designs or models.

Before you begin

- Install IBM Common Licensing server. For more information, see Installing IBM Common Licensing and IBM Common Licensing overview. The supported IBM Common Licensing server versions are 8.1.6 releases or fix packs and 9.0.0.

Important: The IBM Common Licensing Server version is upgraded to 9.0.0.2. Starting from Rhapsody Systems Engineering version 1.6.0, upgrade the IBM Common Licensing Server to version 9.0.0.2. For more information, see Upgrading IBM Common Licensing server.

Note: Ensure that the required license server and vendor ports are opened if any firewall rules or organizational cluster egress policies exist between the Rhapsody Systems Engineering environment and the IBM Common Licensing Server environment.

Procedure

1. Choose disk serial number as the Host ID type. To retrieve the serial number on Windows, open the command prompt and run the following command:

   ```
   vol
   ```

2. Obtain license keys with IBM License Key Center.
3. Import a license key file in the license key administrator.

Results

You can acquire the Rhapsody Systems Engineering license from the available pool.

Checking out Rhapsody Systems Engineering license

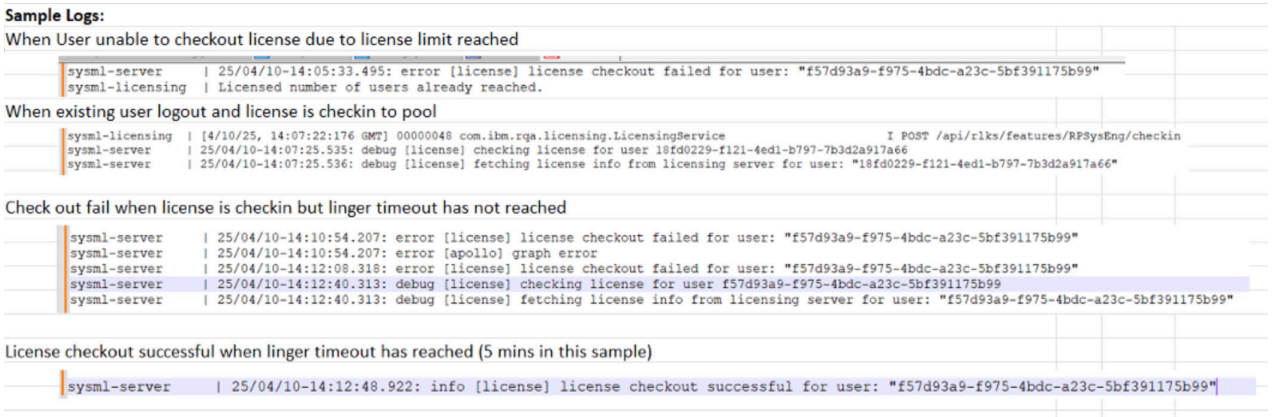
To return the license, log out of the session and then close browser.

Consider the following scenarios when a user checks out the Rhapsody Systems Engineering license:

- After a user logs out, the license is not immediately released back to the pool. It remains checked out until the `linger timeout period` expires. By default, the value of `linger timeout period` is set to 1800 seconds or 30 minutes.
- If the user session becomes inactive for a long period of time, the license is automatically released after the duration specified by `licensesExpirySecs`. By default, the value of `licensesExpirySecs` is set to 4 hours.

You can configure the parameters while creating and configuring the Rhapsody Systems Engineering instance. For more information, see “Creating and configuring the instance” on page 41.

Consider the following sample of logs for a floating license in use.



The following table displays the license parameter support for Rhapsody Systems Engineering.

Setting up watsonx.ai

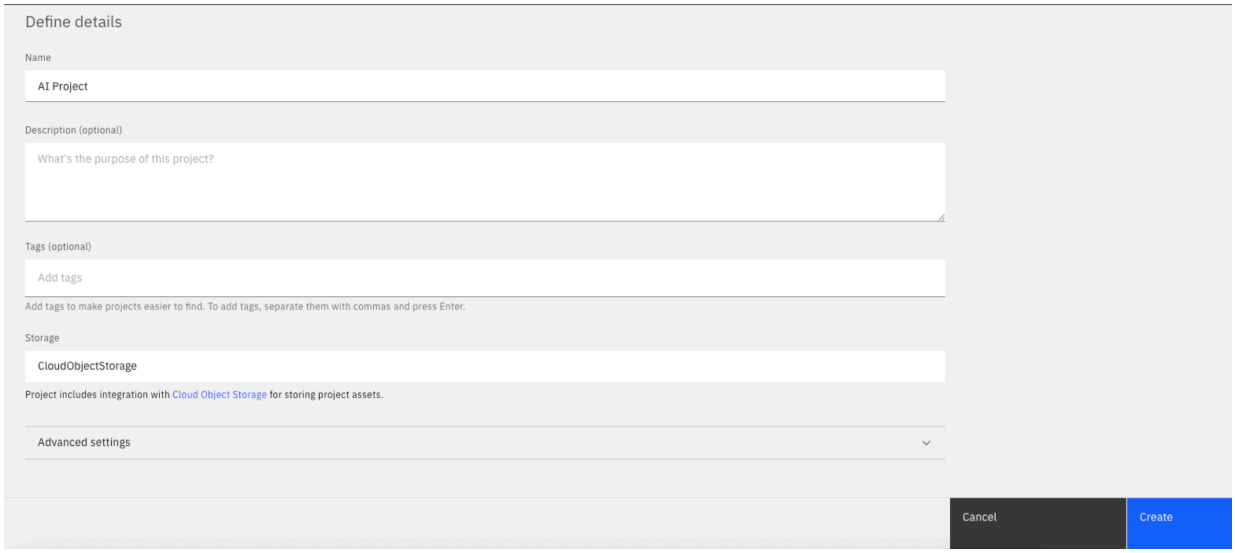
Learn how to set up IBM `watsonx.ai` to configure the project, runtime, and model settings that are needed to generate values for the `ai-services-settings` secret, which enables Rhapsody Systems Engineering pods to securely connect to AI services.

Procedure

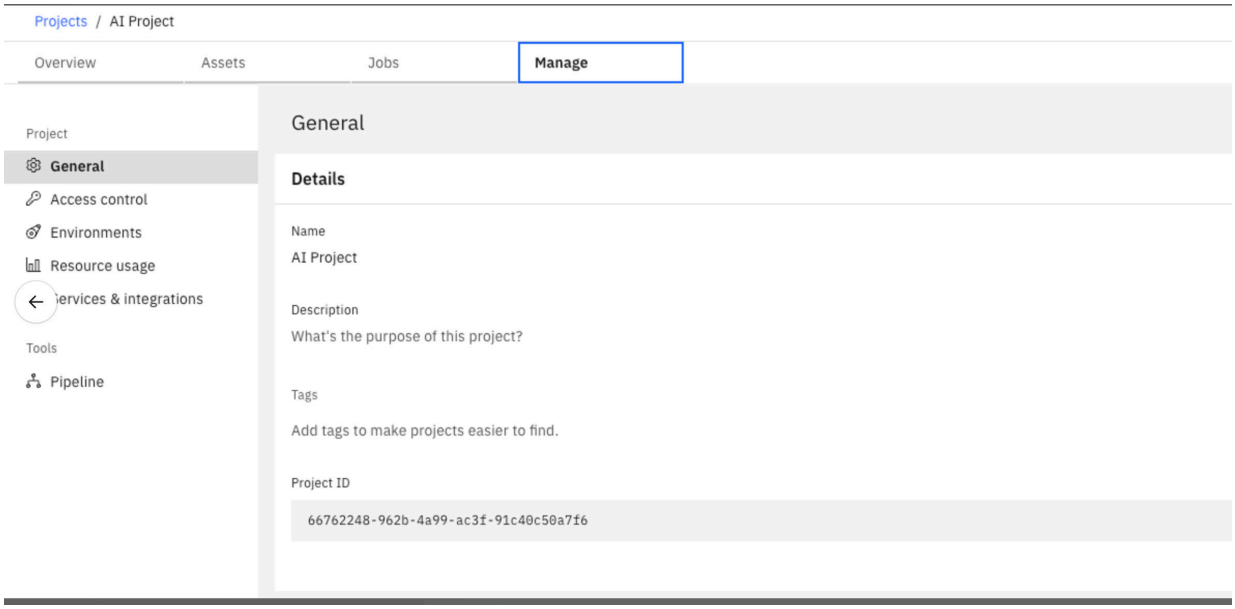
1. Go to https://eu-de.dataplatform.cloud.ibm.com/wx/home and log in to `watsonx.ai`.

   CAUTION: Do not select the `eu-gb` region. It does not support the required `ibm/granite-3-3-8b-instruct` model.

2. Create a project.
   a) From the navigation menu, choose **Projects** > **View all projects** and click **New project**.
   b) Enter a name.
   c) Enter a description.
   d) Add tags to find your projects. To add more tags, separate with commas and press `Enter`.
   e) Select any of the project configurations.
   f) Define project storage.
   g) Click **Create**. You can start adding resources to your project.

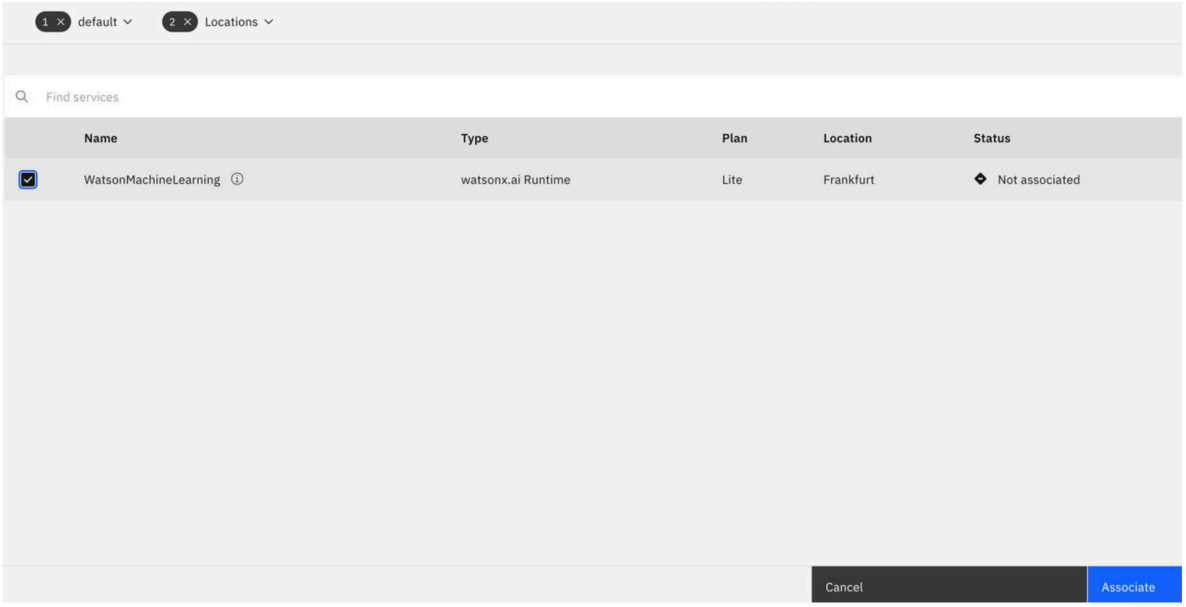


3. Go to the **Manage** tab.

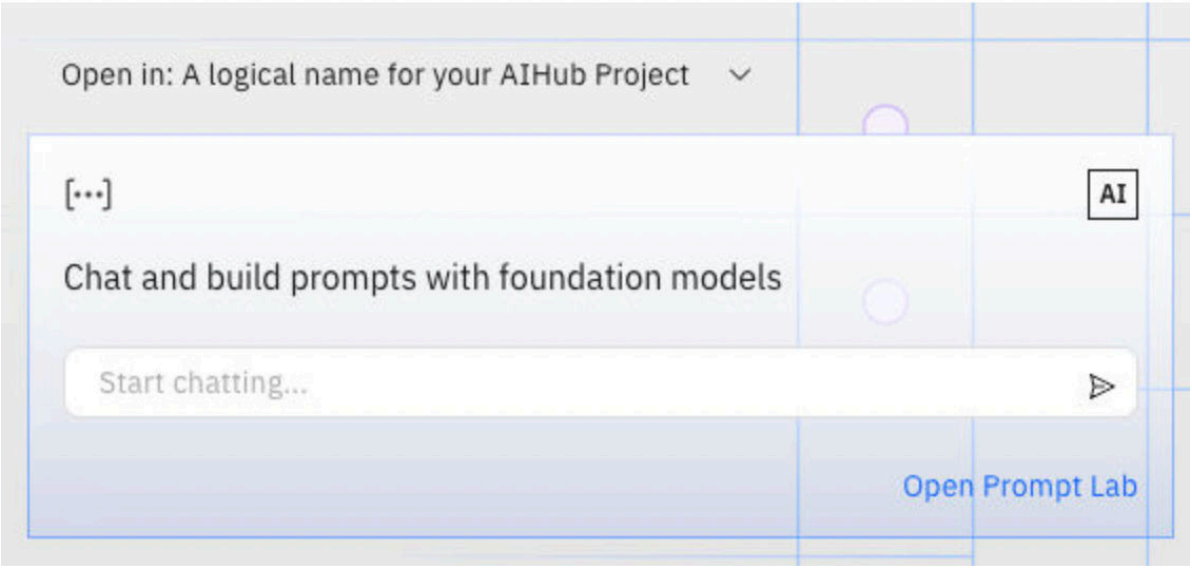


4. Copy your Project ID for your `rse-application-secrets` secret.
5. Associate the watsonx.ai runtime service to the project.
   a) On the project's **Manage** tab, select the Services and integrations page
   b) In the IBM Services section, click **Associate Service**.

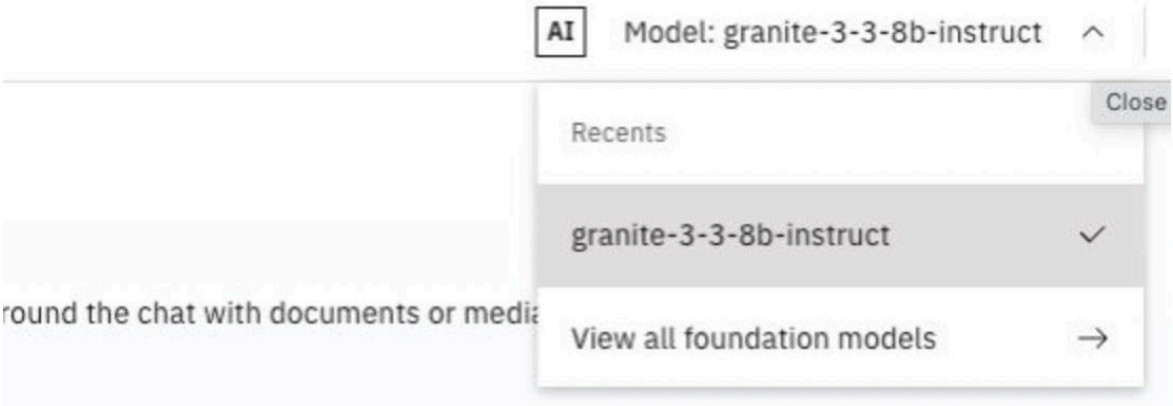
   c) Select your IBM watsonx.ai Runtime service instance and click **Associate**.



6. Set up the prompt lab.
   a) Click the **IBM watsonx** to return to the Home page.
   b) Click **Open Prompt Lab**.

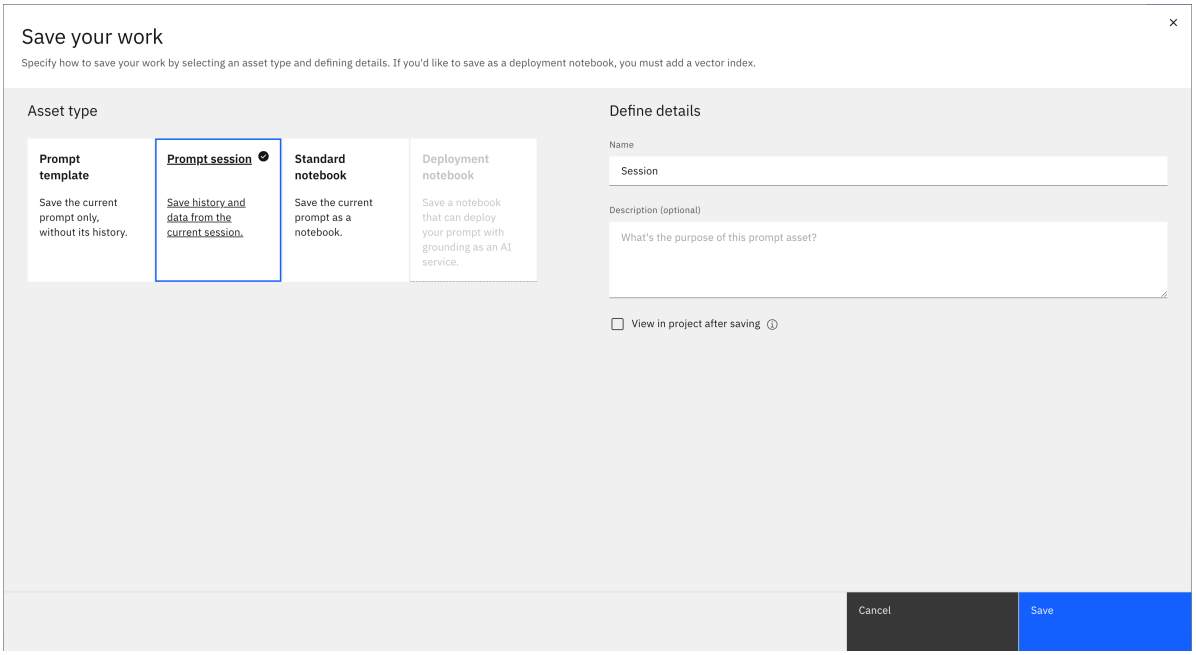


   c) Ensure that it is using **granite-3-3-8b-instruct**.



d) If a different model is selected, click **View all foundation models**, and choose `granite-3-3-8b-instruct`.

e) Save the changes.



i) Select the asset type.

ii) Specify the name and description.

iii) Click **Save**.

What to do next

Create an IBM Cloud API key. For more information, see Creating an API key.

Creating the image pull secret in the cluster

The Rhapsody Systems Engineering images are stored in IBM Entitled Registry. They are pulled from the registry and used to install the Rhapsody Systems Engineering application in the supported cluster.

About this task

An image pull secret must be created to pull the container image in the pod. The entitlement key that is obtained from the registry is used to create the image pull secret. The administrator can create a new image pull secret. It is used to add the image pull secret to grant access for pods to pull images from your registry.

Procedure

1. To obtain the entitlement key from the registry, complete the following substeps:
   a) Go to the IBM container software library link.
   b) Click **View library**. The Access your container software page opens.
   c) To copy the entitlement key in the clipboard, on the Entitlement key section, click **Copy key**.
   d) Paste the entitlement key in a text editor that you use. You are required to provide the entitlement key when you create the image pull secret.
2. To add Rhapsody Systems Engineering to the entitlement key, complete the following susbteps:
   a) Go to https://myibm.ibm.com/products-services/containerlibrary.
   b) Log in by using your IBM ID.

c) Click **Container software library**.

d) Click the **Self-Nominate form**, and then fill the form with required details.

e) Search for Rhapsody Systems Engineering and add it.

f) Select the entitlement keys.

g) Click **Add key**, and copy the key to use it for creating the secret.

3. You can create the image by using the supported cluster or by using command line inputs.

- To create the image pull secret in the Red Hat OpenShift cluster by using the console, complete the following substeps:

  a. Log in to the Red Hat OpenShift cluster platform by using the Red Hat OpenShift administrator credentials.

  b. Go to **Projects**, and select the project where you want to install the Rhapsody Systems Engineering instance.

  c. Go to **Workloads** > **Secrets**.

  d. Select **Image Pull Secret** from the **Create** list.

  e. On the **Create Image Pull Secret** page, enter the following details:

  i) In the **Secret Name** field, enter `ibm-entitlement-key`.

  ii) From the **Authentication Type** list, select **Image registry credentials**.

  iii) In the **Registry Server Address** field, enter `cp.icr.io/cp`.

  iv) In the **Username** field, enter `cp`.

  v) In the **Password** field, enter the `<entitlement-key>` from the text editor.

  vi) In the **Email** field, enter `<your-email-address>`.

- To create the secret by using command line, run the commands for Red Hat OpenShift or Kubernetes cluster:

  Red Hat OpenShift

  a. To select the project where you want to the Rhapsody Systems Engineering instance, run the following command:

  ```
  oc project <project name>
  ```

  b. Run the following command:

  ```
  oc create secret docker-registry ibm-entitlement-key --docker-server =
  <registry_server> --docker-username = <user_name>--docker-password = <password> --
  docker-email = <email>
  ```

  For more information, see Using image secrets in Red Hat OpenShift.

  Kubernetes

  ```
  kubectl create secret docker-registry ibm-entitlement-key --docker-server=<your-
  registry-server> --docker-username=<your-name>--docker-password=<your-password>  --
  docker-email=<your-email>
  ```

  where

  – *<your-name>* = username. Enter cp as username.

  – *<your-password>* = *<entitlement-key-token>*

  – *<your-email>* = *<your-email-address>*

  – *<your-registry-server>* = *<your-registry-server>*. By default, the registry server uses `cp.icr.io/cp`.

Creating TLS certificates for Rhapsody Systems Engineering

Create transport layer security (TLS) certificates secrets for your Rhapsody Systems Engineering instance.

Before you begin

- Ensure that you have valid certificates from the certificate authority of the Rhapsody Systems Engineering.
- Ensure that you have the certificate files and the private keys available before you update the Rhapsody Systems Engineering routes with the certificates.

As a cluster administrator, configure the TLS certificates with the required TLS certificate, private key and certificate authority (CA) certificate of the application hostname, so that routes or ingress are configured to validate the incoming SSL traffic.

Procedure

1. Log in to your cluster.
2. To secure Rhapsody Systems Engineering with TLS encryption, create an external TLS certificate as a secret by providing a certificate and key.

   Red Hat OpenShift

   ```
   oc create secret generic <rse-instance-tls-cert> --from-file=tls.crt=<path-to-cert-file>
   --from-file=tls.key=<path-to-key-file> -n <namespace
   ```

   Kubernetes

   ```
   kubectl create secret generic <rse-instance-tls-cert> --from-file=tls.crt=<path-to-cert-
   file> --from-file=tls.key=<path-to-key-file> -n <namespace>
   ```

3. To ensure a secure communication between services within the cluster, create an internal TLS certificate as a secret.

   Note: Internal TLS certificate with DNS names is relevant for internal service communication only. The DNS names should include `*.<rse-app-namespace>.svc.cluster.local`. For example, if Rhapsody Systems Engineering is deployed on `rse-app namespace` in a cluster, then the applicable DNS name is `*.rse-app.svc.cluster.local`.

   As a cluster administrator, there are many ways to manage the lifecycle of these certificates. One of the ways that you can manage them is by using cert-manager. For more information, go to Cloud native certificate management documentation.

   Red Hat OpenShift

   ```
   oc create secret generic tls-secret-rse-internal --from-file=tls.key==<path-to-server-
   key-file> --from-file=tls.crt=<path-to-server-crt-file> --from-file=ca.crt=<path-to-crt-
   file> -n <namespace>
   ```

   Kubernetes

   ```
   kubectl create secret generic tls-secret-rse-internal --from-file=tls.key==<path-to-
   server-key-file> --from-file=tls.crt=<path-to-server-crt-file> --from-file=ca.crt=<path-
   to-crt-file> -n <namespace>
   ```

   The operator looks for the default secret name, `tls-secret-rse-internal`. If the internal secret name is not provided, by default Rhapsody Systems Engineering uses the predefined name for the internal secret.
4. If you want to change the secret name of internal TLS certificate, update the routing section with `internalServiceTlsSecret : <name_of_the_secret>` before creating the instance. For more information, see “Creating and configuring the instance” on page 41.

Example

Consider the following example of generating a self-signed internal TLS certificate by using OpenSSL.

```
openssl genpkey -algorithm RSA -out server.key -pkeyopt rsa_keygen_bits:2048
openssl req -x509 -days 3650 -new -key server.key
-out server.crt -subj "/CN=rse.myrsecluster.com" -addext
"subjectAltName=DNS:localhost,DNS:'*.<namespace>.svc.cluster.local',IP:127.0.0.1"
```

Red Hat OpenShift

```
oc create secret generic tls-secret-rse-internal --from-file=tls.key=server.key --from-
file=tls.crt=server.crt --from-file=ca.crt=server.crt -n <namespace>
```

Kubernetes

```
kubectl create secret generic tls-secret-rse-internal --from-file=tls.key=server.key --from-
file=tls.crt=server.crt --from-file=ca.crt=server.crt -n <namespace>
```

Consider the following example of generating an internal TLS certificate signed by a Certificate Authority (CA) by using OpenSSL.

```
openssl genpkey -algorithm RSA -out ca.key -pkeyopt rsa_keygen_bits:2048
openssl req -x509 -new -key ca.key -days 3650 -out ca.crt -subj "/CN=rse.myrsecluster.com"
-addext "subjectAltName=DNS:localhost,DNS:'*.<namespace>.svc.cluster.local',IP:127.0.0.1"
openssl genpkey -algorithm RSA -out server.key -pkeyopt rsa_keygen_bits:2048
openssl req -new -key server.key -out server.csr -subj "/CN=server.myrsecluster.com" -addext
"subjectAltName=DNS:localhost,DNS:'*.<namespace>.svc.cluster.local',IP:127.0.0.1"
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -days
3650 -copy_extensions copy
```

Red Hat OpenShift

```
oc create secret generic tls-secret-rse-internal --from-file=tls.key=server.key --from-
file=tls.crt=server.crt --from-file=ca.crt=ca.crt -n <namespace>
```

Kubernetes

```
kubectl create secret generic tls-secret-rse-internal --from-file=tls.key=server.key --from-
file=tls.crt=server.crt --from-file=ca.crt=ca.crt -n <namespace>
```

Setting up ingress controllers

Ingress controllers manages the lifecycle of required networking routes or ingresses in Red Hat OpenShift or Kubernetes clusters, enabling the access to the application. The Rhapsody Systems Engineering operator allows the cluster administrators to choose the ingress controllers while deploying the application.

The Rhapsody Systems Engineering operator uses the default ingress controller available on the cluster. If multiple ingress controllers are available, the cluster administrator can use the `ingressClassName` property while creating and configuring the Rhapsody Systems Engineering instance. For more information, see “Creating and configuring the instance” on page 41.

Red Hat OpenShift cluster

To set up ingress controller on Red Hat OpenShift cluster, go to OpenShift Ingress Controller.

The Rhapsody Systems Engineering operator automatically adds the following default annotation to the route resources it creates.

Annotations	Value	Details
`haproxy.router.openshift.io/timeout`	120s	Custom timeout in seconds.

For more information about Red Hat OpenShift route annotations, go to Route-specific annotations on Red Hat OpenShift platform .

Kubernetes

To set up the NGINX-based ingress controller on a Kubernetes cluster, go to Ingress-Nginx Controller Installation documentation.

For example, to deploy ingress controller by using Helm, run the following commands:

```
helm upgrade --install ingress-nginx ingress-nginx \
  --repo https://kubernetes.github.io/ingress-nginx \
  --namespace ingress-nginx --create-namespace \
  --set controller.ingressClassResource.name=ingress-nginx
```

The Rhapsody Systems Engineering operator adds the following default annotations to the ingress resources it creates.

Annotation name	Value	Details
`nginx.ingress.kubernetes.io/backend-protocol`	HTTPS	Used for secure backend communication. You cannot change them.
`nginx.ingress.kubernetes.io/secure-backends`	True	Used for secure backend communication for older versions. You cannot change them.
`nginx.ingress.kubernetes.io/proxy-connect-timeout`	120s	Custom timeout in seconds.
`nginx.ingress.kubernetes.io/proxy-read-timeout`	120s	Custom timeout in seconds.
`nginx.ingress.kubernetes.io/proxy-send-timeout`	120s	Custom timeout in seconds.
`nginx.ingress.kubernetes.io/proxy-body-size`	0	Allows any size of client request body
`nginx.ingress.kubernetes.io/proxy-buffer-size`	256k	
`nginx.ingress.kubernetes.io/proxy-busy-buffer-size`	512k	

Cluster administrators can override the default annotations by using the `annotations` property under the routing section while creating and configuring the instance. For more information, see “Creating and configuring the instance” on page 41.

Setting up dynamic storage provisioning

Rhapsody Systems Engineering uses persistent volume as a temporary storage. You must set up the cluster with dynamic storage provisioning. When you create Rhapsody Systems Engineering instance, provide the storage class value. The Rhapsody Systems Engineering operator uses the available storage class to create the persistent volumes dynamically.

About this task

The following options are available to retain policy setup of dynamically provisioned persistent volume.

Delete
The default policy that is applied when the persistent volumes are created dynamically. When the persistent volume claim is deleted, the persistent volume object and real backing storage are deleted.

Retain
When the persistent volume claim for persistent volume is deleted, the real backing storage is not deleted and persistent volume is moved to the `Released` phase. The data on the backing storage is not deleted and you must clean up the data manually. The persistent volume in the `Released` phase is not considered for another persistent volume claim creation.

Recycle
When the persistent volume claim for persistent volume is deleted, the real backing storage is not deleted and the persistent volume is moved to the `Available` phase. The data on the backing storage might be scrubbed according to your defined policies.

Procedure

1. Provision dynamic storage. For detailed steps to set up dynamic storage, see Kubernetes storage classes.
2. If you are deploying Rhapsody Systems Engineering on IBM Cloud by using classic infrastructure and are using `ibmc` file storage class, set up custom storage class with `gid=1001` using the example listed in https://cloud.ibm.com/docs/openshift?topic=openshift-cs_storage_nonroot.
3. If you are deploying Rhapsody Systems Engineering on IBM Cloud by using VPC infrastructure and are using `ibmc` file storage class, set up custom storage class with `gid=1001` using the example listed in https://cloud.ibm.com/docs/openshift?topic=openshift-storage-file-vpc-apps.

Creating and configuring the instance

As an administrator, you can create the Rhapsody Systems Engineering instance from the supported cluster or by using the command line inputs. After you create the instance, you can configure the parameters.

About this task

The project is alternatively called namespace. You can follow the same steps to create multiple Rhapsody Systems Engineering instances in multiple namespaces.

You can create Rhapsody Systems Engineering instance by using Red Hat OpenShift console or command line inputs.

Procedure

1. Optional: For improved performance based on your hardware (CPU, RAM, and disk type), tune the PostgreSQL configuration settings. This can improve both read and write performance. Restart your PostgreSQL server after you make any changes to the following configuration settings:

   Memory and cache settings

   - `shared_buffers`: Increase the size from default 128MB or 25% of total memory to utilize more RAM for caching tables and indexes.
   - `wal_buffers`: Increase the size from default 4MB to recommended 16MB. It can improve write performance by reducing WAL flush frequency.

   WAL and checkpoints

   - `max_wal_size`: Increase the size to reduce the frequency of checkpoints during heavy write activity.
   - `checkpoint_timeout`: Increase from 5 minutes time interval to reduce checkpoint overhead and disk input or output spikes.

Query planning and read performance

- `random_page_cost`: Lower the value (from default 4.0) to encourage index usage by the planner.
- `effective_cache_size`: Set to a higher value (50% of total memory of machine or default value 4GB) reflecting available system memory to guide the planner towards more efficient query plans.

2. To create Rhapsody Systems Engineering instance by using the Red Hat OpenShift console, complete the following steps:
   a) Log in to the Red Hat OpenShift Container platform by using the Red Hat OpenShift administrator credentials.
   b) Go to **Operators** > **Installed Operators**.
   c) Select the project where you installed the Rhapsody Systems Engineering operator.
   d) On the **Installed Operators** page, select Rhapsody Systems Engineering from the **Name** column, and click the **Create instance** link.
   e) On the **Create** Rhapsody Systems Engineering page, use one of the following options to edit the custom resource to create a new Rhapsody Systems Engineering instance:

   Form view

   You can enter the details in a form. The following is a sample form.

Table 3. Creating Rhapsody Systems Engineering instance

Property	Description	Sample values
`metadata.name`	Name of Rhapsody Systems Engineering deployment instance.	*rhapsodyseapp-sample-1*
`version`	Provide the Rhapsody Systems Engineering version.	
`imagePullSecret`	Provide a secret to pull images. For more information, see "Creating the image pull secret in the cluster" on page 36.	*ibm-entitlement-key*
`imageRepo`	Provide image repository to pull the operand images from the given repository.	The default value is `cp.icr.io/cp/ibm-rse`
`routing.hostname`	Host name for configuring the routes or ingress resources for Rhapsody Systems Engineering applications.	`https://www.rse.example.com`
`routing.tlsSecret`	Creates the external TLS certificates as a secret. For more information, see "Creating TLS certificates for Rhapsody Systems Engineering" on page 38 .	`rse-instance-tls-secret`

Table 3. Creating Rhapsody Systems Engineering instance (continued)

Property	Description	Sample values
`routing.internalServiceTlsSecret`	Optional: Creates the `self-signed-internal-cert` secret which has a self-signed certificate, private key, and CA certificate of application service DNS hostnames. For more information, see "Creating TLS certificates for Rhapsody Systems Engineering" on page 38.	`tls-secret-rse-internal`
`routing.ingressClassName`	Optional: Provide the specific ingress class name to create ingress resources for Kubernetes platform only. For more information, see "Setting up ingress controllers" on page 39.	
`routing.annotations`	Optional: Provide the required annotations to be added into ingress resources for Kubernetes platform only. For more information, see "Setting up ingress controllers" on page 39.	
`logging.server`	Optional: Enables the log level of server.	Values are `trace`, `debug`, `info`, and `warn`.
`logging.client`	Optional: Enables the log level of client.	Values are `trace`, `debug`, `info`, and `warn`.
`logging.restApi`	Optional: Enables the log level of restAPI.	Values are `trace`, `debug`, `info`, and `warn`.
`logging.license`	Optional: Enables the log level of license.	Values are `trace`, `debug`, `info`, and `warn`.
`linkIndexProvider.ldxEndpoint`	Optional: Base URL for Link Index Provider (LDX) application. For example: `https://<elm_host_name>:<port_number>/ldx`.	
`linkIndexProvider.isJASEnabled`	Optional: It is `true` if LDX is using JAS as an OIDC provider, else it is `false`.	Default value is `false`.
`oidc.ClientId`	The client ID forms part of the credentials to use to communicate with your OIDC provider. This ID must be registered with your OIDC provider. Contact your OIDC provider to get this value.	`secret::rse-application-secrets::oidcClientId`

Table 3. Creating Rhapsody Systems Engineering instance (continued)		
Property	**Description**	**Sample values**
`oidc.clientSecret`	The client secret forms part of the credentials to communicate with your ODIC provider. Contact your OIDC provider to get this value.	`secret::rse-application-secrets::oidcClientSecret`
`oidc.signingKey`	The signing key must be a string, which is used as a passphrase when signing with JWT tokens. Consider the length of signing key as atleast 16 characters. Do not share the key as it is a secret.	`secret::rse-application-secrets::oidcSigningKey`
`tokenExpiry`	Allows users to set expiry duration in days for the JWT tokens created through RhapsodySE UI. The values must be between 7 days and 180 days.	
`oidc.wellKnownEndpoint`	A URL of your OIDC provider to reach the well-known-endpoint. Contact your OIDC provider to get this value.	`https://login.example.com/oidc/endpoint/default/.well-known/openid-configuration`
`oidc.introspectionEndpoint`	A URL of your OIDC provider to reach the introspection-endpoint. Contact your OIDC provider to get this value.	`https://login.example.com/oidc/endpoint/default/introspect`
`oidc.customIdentityClaim`	Optional: Specifies one of the fields returned by your OIDC provider. This field determines what the administrator should enter while adding user roles. email is recommended.	`email`
`oidc.scope`	Optional: Specify the access privileges that are requested for access tokens.	"openid profile email"
`oidc.rejectUnauthorized`	Optional: Allows self signed TLS certificates.	By default it is `false`.
`enableWalkMe`	Enables product tours for Rhapsody Systems Engineering.	By default it is `false`.
`database.server`	The database connection URL	`postgress://hostname:<port>`
`database.user`	The ID to connect to your database.	`Admin`

Table 3. Creating Rhapsody Systems Engineering instance (continued)

Property	Description	Sample values
database.password	The password to connect to your database.	secret::rse-application-secrets::dbPassword
database.cacert	Optional: Secure, encrypted TLS or SSL connections between a database client and server.	secret::rse-application-secrets::ca-certificate.pem
database.adminUser	The administrator ID to connect to your database.	ibm_cloud_user
database.dbConnOptions	Optional: Passes JDBC connection options.	Any JDBC connection options as key: value
database.adminPassword	The administrator password to connect to your database.	secret::rse-application-secrets::dbAdminPassword
appEnvs	Optional: Adds the custom environment variables for the operands.	Any environment variables as key: value under appEnvs
licensing.server.host	Host details of your IBM Common Licensing server.	myserver.ibm.com
licensing.server.id	Server ID of IBM Common Licensing server. For more information, see “Configuring IBM Common Licensing server ” on page 32.	0242ac91000211
licensing.server.port	Port number of your IBM Common Licensing server.	27000
licensing.vendor.id	Vendor ID of IBM Common Licensing server.	ibmratl
licensing.vendor.port	Vendor port of IBM Common Licensing server.	27002
licensing.lingerTimeDefault	Optional: LingerTimeout defines for how long licenses should not be released to the pool after user log out the active session.	The default value is 1800 (30 min).
licensing.licensesExpirySecs	Specifies timeout on idle browser to return the licenses.	Default value is 14,400 seconds or 4 hours.
graphQLUser	Provides functional user ID responsible to run background OSLC tasks.	By default, it is oslc.functional-user@no-reply.com.
serviceToken	Creates secret phrase used for OSLC APIs. It is maximum up to 250 characters.	By default, it is empty.

Table 3. Creating Rhapsody Systems Engineering instance (continued)		
Property	**Description**	**Sample values**
`Storage.storageClassName`	The storage class name to dynamically provision the persistent volumes for used by applications. For more information, see “Setting up dynamic storage provisioning ” on page 40.	`rookceph-fs`
`client`	Optional: By default, a `client` operand is configured with a single replica.	For values, refer the yaml file.
`server`	Optional: By default, a `server` operand is configured with a single replica.	For values, refer the yaml file.
`restAPIServer`	Optional: By default, a `restAPIServer` operand is configured with a single replica.	For values, refer the yaml file.
`licenseServer`	Optional: By default, a `licenseServer` operand is configured with a single replica.	For values, refer the yaml file.
`fsGroup`	File system group ID used for controlling access to the block storage.	The allowed characters are numerical values. For example: 1001.
`supplementalGroup`	Group IDs used to control access to the shared storage.	The allowed characters are an array of numbers. If more than one group ID is populated, the group IDs would be in array. For example: 1001.
`authProtocol`	Optional: The identity provider that you prefer to use.	The oidc or saml valid values.
`saml.authSecret`	The secret must be a string, which is used as a passphrase when signing with JWT tokens. Consider the length of secret as atleast 16 characters.	`secret::rse-application-secrets::samlAuthSecret`
`saml.tokenExpiry`	Allows users to set expiry duration in days for the JWT tokens created through RhapsodySE UI. The values must be between 7 days and 180 days.	

Table 3. Creating Rhapsody Systems Engineering instance (continued)		
Property	**Description**	**Sample values**
`saml.samlIdpMetadataFile`	The XML metadata file provided by the identity provider. • IBMId calls this as the IDP metadata file. • Microsoft Entra ID calls this as the federation metadata XML.	Download from the identity provider.

Table 3. Creating Rhapsody Systems Engineering instance (continued)

Property	Description	Sample values

`saml.samlSpMetadataFile`	The XML metadata file provided by your service provider.
	• A unique entity ID or provider ID.
	• One or more ACS (HTTP-POST) URLs where the IDP sends SAML assertions.
		Consider the following XML file. Create the XML file and provide the following values:
		• `<entityID>`
		• `< Location>`
		`<?xml version="1.0" encoding="UTF-8" standalone="yes"?>`
		`<md:EntityDescriptor xmlns:md="urn:oasis:names:tc:SAML:2.0:metadata" entityID="<PUT YOUR UNIQUE IDENTIFIER>">`
		`<md:SPSSODescriptor protocolSupportEnumeration="urn:oasis:names:tc:SAML:2.0:protocol">`
		`<md:NameIDFormat>urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress</md:NameIDFormat>`
		`<md:NameIDFormat>urn:oasis:names:tc:SAML:2.0:nameid-format:persistent</md:NameIDFormat>`
		`<md:NameIDFormat>urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified</md:NameIDFormat>`
		`<md:AssertionConsumerService Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST" Location="https://<PUT YOUR APPLICATION HOSTED URL>/api/auth/login/saml" index="0" isDefault="true"/>`
		`</md:SPSSODescriptor>`
		`<md:Organization>`
		`<md:OrganizationName xml:lang="en">SysML V2</md:OrganizationName>`
		`<md:OrganizationDisplayName xml:lang="en">SysML V2</md:OrganizationDisplayName>`
		`<md:OrganizationURL xml:lang="en">https://www.ibm.com/products/engineering-rhapsody</md:OrganizationURL>`
		`</md:Organization>`
		`</md:EntityDescriptor>`

Table 3. Creating Rhapsody Systems Engineering instance (continued)		
Property	**Description**	**Sample values**
`saml.attributeMapping`	Allows users to map attributes from identity provider to application attributes. This is the default mapping.	#id: <uniqueSecurityName> #email: <email> #name: <name> The attributes 'uniqueSecurityName', 'email', and 'name' are received from the identity provider.
`mbseAgent.elmAIHubUrl`	Optional: Engineering AI Hub API URL.	`https://<ai-hub_server.ibm.com>`

Table 4. Creating Rhapsody Systems Engineering instance		
Property	**Description**	**Sample values**
`metadata.name`	Name of Rhapsody Systems Engineering deployment instance.	*`rhapsodyseapp-sample-1`*
`version`	Provide the Rhapsody Systems Engineering version.	
`imagePullSecret`	Provide a secret to pull images. For more information, see "Creating the image pull secret in the cluster" on page 36.	*`ibm-entitlement-key`*
`imageRepo`	Provide image repository to pull the operand images from the given repository.	The default value is `cp.icr.io/cp/ibm-rse`
`routing.hostname`	Host name for configuring the routes or ingress resources for Rhapsody Systems Engineering applications.	`https://www.rse.example.com`
`routing.tlsSecret`	Creates the external TLS certificates as a secret. For more information, see "Creating TLS certificates for Rhapsody Systems Engineering" on page 38 .	`rse-instance-tls-secret`
`routing.internalServiceTlsSecret`	Optional: Creates the `self-signed-internal-cert` secret which has a self-signed certificate, private key, and CA certificate of application service DNS hostnames. For more information, see "Creating TLS certificates for Rhapsody Systems Engineering" on page 38.	`tls-secret-rse-internal`

Table 4. Creating Rhapsody Systems Engineering instance (continued)		
Property	**Description**	**Sample values**
`routing.ingressClassName`	Optional: Provide the specific ingress class name to create ingress resources for Kubernetes platform only. For more information, see "Setting up ingress controllers" on page 39.	
`routing.annotations`	Optional: Provide the required annotations to be added into ingress resources for Kubernetes platform only. For more information, see "Setting up ingress controllers" on page 39.	
`logging.server`	Optional: Enables the log level of server.	Values are `trace`, `debug`, `info`, and `warn`.
`logging.client`	Optional: Enables the log level of client.	Values are `trace`, `debug`, `info`, and `warn`.
`logging.restApi`	Optional: Enables the log level of restAPI.	Values are `trace`, `debug`, `info`, and `warn`.
`logging.license`	Optional: Enables the log level of license.	Values are `trace`, `debug`, `info`, and `warn`.
`linkIndexProvider.ldxEndpoint`	Optional: Base URL for Link Index Provider (LDX) application. For example: `https://<elm_host_name>:<port_number>/ldx`.	
`linkIndexProvider.isJASEnabled`	Optional: It is `true` if LDX is using JAS as an OIDC provider, else it is `false`.	Default value is `false`.
`oidc.ClientId`	The client ID forms part of the credentials to use to communicate with your OIDC provider. This ID must be registered with your OIDC provider. Contact your OIDC provider to get this value.	`secret::rse-application-secrets::oidcClientId`
`oidc.clientSecret`	The client secret forms part of the credentials to communicate with your ODIC provider. Contact your OIDC provider to get this value.	`secret::rse-application-secrets::oidcClientSecret`

Table 4. Creating Rhapsody Systems Engineering instance (continued)		
Property	**Description**	**Sample values**
`oidc.signingKey`	The signing key must be a string, which is used as a passphrase when sigining with JWT tokens. Consider the length of signing key as atleast 16 characters. Do not share the key as it is a secret.	`secret::rse-application-secrets::oidcSigningKey`
`oidc.wellKnownEndpoint`	A URL of your OIDC provider to reach the well-known-endpoint. Contact your OIDC provider to get this value.	`https://login.example.com/oidc/endpoint/default/.well-known/openid-configuration`
`oidc.introspectionEndpoint`	A URL of your OIDC provider to reach the introspection-endpoint. Contact your OIDC provider to get this value.	`https://login.example.com/oidc/endpoint/default/introspect`
`oidc.customIdentityClaim`	Optional: Specifies one of the fields returned by your OIDC provider. This field determines what the administrator should enter while adding user roles. email is recommended.	`email`
`oidc.scope`	Optional: Specify the access privileges that are requested for access tokens.	"openid profile email"
`oidc.rejectUnauthorized`	Optional: Allows self signed TLS certificates.	By default it is `false`.
`enableWalkMe`	Enables product tours for Rhapsody Systems Engineering.	By default it is `false`.
`database.server`	The database connection URL	`postgress://hostname:<port>`
`database.user`	The ID to connect to your database.	`Admin`
`database.password`	The password to connect to your database.	`secret::rse-application-secrets::dbPassword`
`database.cacert`	Optional: Secure, encrypted TLS or SSL connections between a database client and server.	`secret::rse-application-secrets::ca-certificate.pem`
`database.adminUser`	The administrator ID to connect to your database.	`ibm_cloud_user`
`database.dbConnOptions`	Optional: Passes JDBC connection options.	Any JDBC connection options as key: value

Table 4. Creating Rhapsody Systems Engineering instance (continued)		
Property	**Description**	**Sample values**
`database.adminPassword`	The administrator password to connect to your database.	`secret::rse-application-secrets::dbAdminPassword`
`appEnvs`	Optional: Adds the custom environment variables for the operands.	Any environment variables as key: value under appEnvs
`licensing.server.host`	Host details of your IBM Common Licensing server.	`myserver.ibm.com`
`licensing.server.id`	Server ID of IBM Common Licensing server. For more information, see “Configuring IBM Common Licensing server ” on page 32.	`0242ac91000211`
`licensing.server.port`	Port number of your IBM Common Licensing server.	`27000`
`licensing.vendor.id`	Vendor ID of IBM Common Licensing server.	`ibmratl`
`licensing.vendor.port`	Vendor port of IBM Common Licensing server.	`27002`
`licensing.lingerTimeDefault`	Optional: LingerTimeout defines for how long licenses should not be released to the pool after user log out the active session.	The default value is 1800 (30 min).
`licensing.licensesExpirySecs`	Specifies timeout on idle browser to return the licenses.	Default value is 14,400 seconds or 4 hours.
`graphQLUser`	Provides functional user ID responsible to run background OSLC tasks.	By default, it is `oslc.functional-user@no-reply.com`.
`serviceToken`	Creates secret phrase used for OSLC APIs. It is maximum up to 250 characters.	By default, it is empty.
`Storage.storageClassName`	The storage class name to dynamically provision the persistent volumes for used by applications. For more information, see “Setting up dynamic storage provisioning ” on page 40.	`rookceph-fs`
`client`	Optional: By default, a `client` operand is configured with a single replica.	For values, refer the yaml file.

Table 4. Creating Rhapsody Systems Engineering instance (continued)		
Property	**Description**	**Sample values**
`server`	Optional: By default, a `server` operand is configured with a single replica.	For values, refer the yaml file.
`restAPIServer`	Optional: By default, a `restAPIServer` operand is configured with a single replica.	For values, refer the yaml file.
`licenseServer`	Optional: By default, a `licenseServer` operand is configured with a single replica.	For values, refer the yaml file.
`fsGroup`	File system group ID used for controlling access to the block storage.	The allowed characters are numerical values. For example: 1001.
`supplementalGroup`	Group IDs used to control access to the shared storage.	The allowed characters are an array of numbers. If more than one group ID is populated, the group IDs would be in array. For example: 1001.
`mbseAgent.llmApiUrl`	Optional: Watsonx.ai URL used to call Watsonx.ai API.	`secret::rse-application-secrets::llmApiUrl`
`mbseAgent.llmApiKey`	Optional: IBM cloud API key for secure authorization to Watsonx.ai.	`secret::rse-application-secrets::llmApiKey`
`mbseAgent.llmProjectId`	Optional: Project ID for Watsonx.ai project.	`secret::rse-application-secrets::llmProjectId`

YAML view

You can create objects. Click the **YAML** tab and populate the Rhapsody Systems Engineering instance details in the Rhapsody Systems Engineering custom resource specification YAML.

f) Click **Create**. The Rhapsody Systems Engineering instance is created.

3. To create the Rhapsody Systems Engineering instance by using command line, complete the following steps:

   a) Create the YAML file.

   b) Generate the exact values for your deployment. Values defined in the following sample yaml is for illustration only. For more information, see “Preparing to create Rhapsody Systems Engineering instance” on page 31.

```
apiVersion: rhapsodyse.ibm.com/v1
kind: RhapsodySEApp
metadata:
  name: rhapsodyseapp-sample-1
  namespace: rse
spec:
  ## Accept IBM Rhapsody SE license by setting the value as true.
  acceptLicense: true

  ## (Optional) Version of IBM Rhapsody SE that need to be deployed. If omitted the
latest version supported will be installed
```

```
  version: <>

  ## Rhapsody SE image configuration
  imagePullSecret: all-icr-io
  imageRepo: cp.icr.io/cp/ibm-rse

  ## Defines the routing for the IBM Rhapsody SE apps
  routing:
    hostname: rse.mycluster.com
    tlsSecret: selfsigned-cert-tls
   #internalServiceTlsSecret: self-signed-internal-cert
    ingressClassName:
    annotations:

 #logging:
    #server: trace
    #client: trace
    #restApi: debug
    #license: info

  ## OIDC configuration
  oidc:
    clientId: secret::rse-application-secrets::oidcClientId
    clientSecret: secret::rse-application-secrets::oidcClientSecret
    signingKey: secret::rse-application-secrets::oidcSigningKey
    wellKnownEndpoint: https://login.example.com/oidc/endpoint/default/.well-known/openid-
configuration
    introspectionEndpoint: https://login.example.com/oidc/endpoint/default/introspect
    tokenExpiry:

  ## Database configuration
  database:
    server: https://pgdb.databases.appdomain.cloud:31018
    user: admin
    password: secret::rse-application-secrets::dbPassword
    adminUser: ibm_cloud_user
    adminPassword: secret::rse-application-secrets::dbAdminPassword
    # name: sysml
    # schema: sysml
    cacert: secret::rse-application-secrets::ca-certificate.pem
    #dbConnOptions:
      #connectTimeout: "100"
      #socketTimeout: "0"
      #tcpKeepAlive: "false"

 #appEnvs:
   #NODES: "0"
   #app: rse

  ## Licensing configuration
  licensing:
   #lingerTimeDefault: 1800
   #licensesExpirySecs: 4
    server:
      host: myserver.ibm.com
      id: 0242ac91000211
      port: 27000
    vendor:
      id: ibmratl
      port: 27002
    auth: false

## Link index provider
  #linkIndexProvider:
    #ldxEndpoint: ""
    #isJASEnabled: false

  ## List of server admin user names
  serverAdminUsernames:
  - sampleuser1@yourco.com
  - sampleuser2@yourco.com
  - john.doe@ibm.com

  ## Service token for adding OSLC associations
  serviceToken: ktf5hjkioliholhZyhgftk

  ## Storage configuration for storing template cache. By default it will use
ReadWriteMany as the access mode. Ensure you use a storage class that supports
ReadWriteMany
  storage:
    storageClassName: rookceph-fs
    # capacity: 10Gi
```

```
  ## Client application overrides
  client:
    replica: 1
    resources:
      requests:
        memory: "256Mi"
        cpu: "500m"
      limits:
        memory: "512Mi"
        cpu: "1"

  ## Server application overrides
  server:
    replica: 1
    resources:
      requests:
        memory: "1024Mi"
        cpu: "1"
      limits:
        memory: "1024Mi"
        cpu: "2"

  ## REST API Server application overrides
  restAPIServer:
    replica: 1
    resources:
      requests:
        memory: "512Mi"
        cpu: "1"
      limits:
        memory: "1024Mi"
        cpu: "2"

  ## License Server application overrides
  licenseServer:
    replica: 1
    resources:
      requests:
        memory: "512Mi"
        cpu: "500m"
      limits:
        memory: "1024Mi"
        cpu: "1"

 fsGroup: <file-system-group-id>
 supplementalGroups:
   <supplemental-group-id-1>
   <supplemental-group-id-2>

  ## Optional AI settings
 mbseAgent:
   elmAIHubUrl: https://<ai-hub_server.ibm.com>

#authProtocol: saml #default oidc
#saml:
  #authSecret: secret::rse-application-secrets::samlAuthSecret
  #tokenExpiry:
  #idpMetadataFile: secret::rse-application-secrets::samlIdpMetadataFile
  #spMetadataFile: secret::rse-application-secrets::samlSpMetadataFile
  #attributeMapping:
    #id: uniqueSecurityName
    #email: email
    #name: name
```

```
apiVersion: rhapsodyse.ibm.com/v1
kind: RhapsodySEApp
metadata:
  name: rhapsodyseapp-sample-1
  namespace: rse
spec:
  ## Accept IBM Rhapsody SE license by setting the value as true.
  acceptLicense: true

  ## (Optional) Version of IBM Rhapsody SE that need to be deployed. If omitted the
latest version supported will be installed
  version: <>

  ## Rhapsody SE image configuration
  imagePullSecret: all-icr-io
  imageRepo: cp.icr.io/cp/ibm-rse
```

```
  ## Defines the routing for the IBM Rhapsody SE apps
  routing:
    hostname: rse.mycluster.com
    tlsSecret: selfsigned-cert-tls
   #internalServiceTlsSecret: self-signed-internal-cert
    ingressClassName:
    annotations:

 #logging:
    #server: trace
    #client: trace
    #restApi: debug
    #license: info

  ## OIDC configuration
  oidc:
    clientId: secret::rse-application-secrets::oidcClientId
    clientSecret: secret::rse-application-secrets::oidcClientSecret
    signingKey: secret::rse-application-secrets::oidcSigningKey
    wellKnownEndpoint: https://login.example.com/oidc/endpoint/default/.well-known/openid-
configuration
    introspectionEndpoint: https://login.example.com/oidc/endpoint/default/introspect

  ## Database configuration
  database:
    server: https://pgdb.databases.appdomain.cloud:31018
    user: admin
    password: secret::rse-application-secrets::dbPassword
    adminUser: ibm_cloud_user
    adminPassword: secret::rse-application-secrets::dbAdminPassword
    # name: sysml
    # schema: sysml
    cacert: secret::rse-application-secrets::ca-certificate.pem
    #dbConnOptions:
      #connectTimeout: "100"
      #socketTimeout: "0"
      #tcpKeepAlive: "false"

 #appEnvs:
   #NODES: "0"
   #app: rse

  ## Licensing configuration
  licensing:
   #lingerTimeDefault: 1800
   #licensesExpirySecs: 4
    server:
      host: myserver.ibm.com
      id: 0242ac91000211
      port: 27000
    vendor:
      id: ibmratl
      port: 27002
    auth: false

## Link index provider
  #linkIndexProvider:
    #ldxEndpoint: ""
    #isJASEnabled: false

  ## List of server admin user names
  serverAdminUsernames:
  - sampleuser1@yourco.com
  - sampleuser2@yourco.com
  - john.doe@ibm.com

  ## Service token for adding OSLC associations
  serviceToken: ktf5hjkioliholhZyhgftk

  ## Storage configuration for storing template cache. By default it will use
ReadWriteMany as the access mode. Ensure you use a storage class that supports
ReadWriteMany
  storage:
    storageClassName: rookceph-fs
    # capacity: 10Gi

  ## Client application overrides
  client:
    replica: 1
    resources:
      requests:
```

```
        memory: "256Mi"
        cpu: "500m"
      limits:
        memory: "512Mi"
        cpu: "1"

## Server application overrides
server:
  replica: 1
  resources:
    requests:
      memory: "1024Mi"
      cpu: "1"
    limits:
      memory: "1024Mi"
      cpu: "2"

## REST API Server application overrides
restAPIServer:
  replica: 1
  resources:
    requests:
      memory: "512Mi"
      cpu: "1"
    limits:
      memory: "1024Mi"
      cpu: "2"

## License Server application overrides
licenseServer:
  replica: 1
  resources:
    requests:
      memory: "512Mi"
      cpu: "500m"
    limits:
      memory: "1024Mi"
      cpu: "1"

 fsGroup: <file-system-group-id>
 supplementalGroups:
  <supplemental-group-id-1>
  <supplemental-group-id-2>

## Optional AI settings
 mbseAgent:
   llmApiUrl: secret::rse-application-secrets::llmApiUrl
   llmApiKey: secret::rse-application-secrets::llmApiKey
   llmProjectId: secret::rse-application-secrets::llmProjectId
```

c) Create a RhapsodySEApp resource using the above YAML file by configuring the parameters.

Note: You can create only one instance of RhapsodySEApp in a namespace. If you want to create multiple environments for development, quality, staging, production, and more, create the respective namespaces and secrets in your cluster.

d) Run the following command to create the RhapsodySEApp in the namespace created previously.

Red Hat OpenShift

```
oc apply -f <RhapsodySEApp-yaml-file> -n <namespace>
```

Kubernetes

```
kubectl apply -f <RhapsodySEApp-yaml-file> -n <namespace>
```

4. After you create and configure the Rhapsody Systems Engineering instance, access the application by using the URL configured in `routing.hostname`. For example: `https://www.rse.example.com`.

Updating the instance

You can update the Rhapsody Systems Engineering instance by using Red Hat OpenShift console or by changing any parameters and reapplying the yaml file.

Procedure

Update the Rhapsody Systems Engineering instance by using Red Hat OpenShift console or by using command line.

- Using Red Hat OpenShift console, complete the following steps:
  a. In the Red Hat OpenShift console, go to **Operators** > **Installed Operators**.
  b. Select the project, where you installed the Rhapsody Systems Engineering operator.
  c. Go to the RhapsodySEApp section, and click **More actions** (⋮) icon.
  d. Click **Edit RhapsodySEApp**.
  e. Update the existing details that you gave in the form of YAML file, and click **Update**. For more information, see "Creating and configuring the instance" on page 41.
- Using command line, complete the following steps:
  a. Log in to Red Hat OpenShift container platform cluster.
  b. Update the RhapsodySE CR yaml file, and run the following command to apply your changes. To refer the YAML file, see "Creating and configuring the instance" on page 41.

  Red Hat OpenShift

  ```
  oc apply -f <RhapsodySEApp-yaml-file> -n <namespace>
  ```

  Kubernetes

  ```
  kubectl apply -f <RhapsodySEApp-yaml-file> -n <namespace>
  ```

Verifying the resources created after Rhapsody Systems Engineering instance creation

After the instance of Rhapsody Systems Engineering is created, verify that the resources were created.

Procedure

1. As a Red Hat OpenShift administrator or Red Hat OpenShift project administrator, log in to the Red Hat OpenShift console.
2. Run the following command to get a list of resources that are created after Rhapsody Systems Engineering is installed.

   OpenShift

   ```
   $ oc get <resource> -n <namespace>
   ```

   Kubernetes

   ```
   $ kubectl get <resource> -n <namespace>
   ```

   Where:

   <resource>
   is the resource category for which you want to identify the resources that are created.

   <namespace>
   is the namespace or the project, where Rhapsody Systems Engineering is installed.

The following table displays the resources that are created after the instance for Rhapsody Systems Engineering is created.

Note: `rhapsodyseapp-sample-1` is the name of the Rhapsody Systems Engineering application.

Resource category	Resources created
deployment	• **`rhapsodyseapp-sample-1-rse-client`** • **`rhapsodyseapp-sample-1-rse-license`** • **`rhapsodyseapp-sample-1-rse-rest`** • **`rhapsodyseapp-sample-1-rse-server`**
service	• **`rhapsodyseapp-sample-1-rse-client-service`** • **`rhapsodyseapp-sample-1-rse-license-service`** • **`rhapsodyseapp-sample-1-rse-rest-service`** • **`rhapsodyseapp-sample-1-rse-server-service`**
pods	• **`rhapsodyseapp-sample-1-rse-client-XXXX`** • **`rhapsodyseapp-sample-1-rse-license-XXXX`** • **`rhapsodyseapp-sample-1-rse-rest-XXXX`** • **`rhapsodyseapp-sample-1-rse-server-XXXX`**
networkpolicy	• **`rhapsodyseapp-sample-1-rse-policy-deny-cross-namespace-traffic`**
routes	• **`rhapsodyseapp-sample-1-rse-client-route`** • **`rhapsodyseapp-sample-1-rse-license-route`** • **`rhapsodyseapp-sample-1-rse-rest-route`** • **`rhapsodyseapp-sample-1-rse-server-route`** • **`rhapsodyseapp-sample-1-rse-server-socket-route`**
secrets	• **`rse-settings-config`**
configmap	• **`rse-license-client-config`**
Serviceaccount	• **`rhapsodyseapp-sample-1-rse-deploy`**

Uninstalling Rhapsody Systems Engineering operator

Uninstalling the Rhapsody Systems Engineering operator does not remove the Rhapsody Systems Engineering instance and associated resources. You can uninstall Rhapsody Systems Engineering operator by using the Red Hat OpenShift console or by using the command line. This process involves the removal of the operator from the Installed operators page.

Before you begin

- Ensure that the Rhapsody Systems Engineering instance is deleted. For more information, see "Deleting the Rhapsody Systems Engineering instance" on page 62.
- You must have the cluster-admin role to uninstall the RhapsodySE operator. For more information about the user roles in the Red Hat OpenShift cluster, see "User roles mapping in Rhapsody Systems Engineering" on page 21.

About this task

Use one of the following methods to uninstall the Rhapsody Systems Engineering operator.

- By using Red Hat OpenShift console
- By using Red Hat OpenShift command line
- By using Kubernetes

Procedure

1. To uninstall Rhapsody Systems Engineering by using the Red Hat OpenShift console, complete the following substeps:
   a) Log in to the Red Hat OpenShift Container Platform console.
   b) Click **Operators** > **Installed operators**. A page listing all the installed operators opens.
   c) From the **Project** list, select the select the project where you installed the Rhapsody Systems Engineering operator. The RhapsodySE operator is shown in a table on the Installed Operators page.
   d) Select the RhapsodySE operator and click **Uninstall Operator** from the overflow menu . A window opens to confirm the delete action.
   e) Select **Uninstall** to remove the RhapsodySE operator.
2. To uninstall Rhapsody Systems Engineering by using the Red Hat OpenShift cluster command line, complete the following substeps:
   a) Log in to the Red Hat OpenShift Container Platform command line.
   b) Log in to your cluster by using the following command.

   ```
   oc login --token=<token> --server=<openshift-server-url>
   ```

   where

   Item

   Description

   token
   Authentication token for the user to log in to the cluster.

   openshift-server-url
   Red Hat OpenShift Container Platform server URL.

   c) Verify the current version of the subscribed operator by using the following command.

   ```
   oc get subscription rhapsody-se-operator -n <namespace-name> -o yaml | grep currentCSV
   ```

where

current-csv-version
Current cluster service version of the operator. Use current CSV information for this parameter.

namespace-name
Project or namespace name where the RhapsodySE operator is installed.

d) Delete the subscription of the operator by using the following command.

```
oc delete subscription rhapsody-se-operator -n <namespace-name>
```

e) Delete the current subscription version of the operator in the target namespace by using the following command.

```
oc delete clusterserviceversion <current-csv-version> -n <namespace-name>
```

where

current-csv-version
Current cluster service version of the operator. Use current CSV information for this parameter.

namespace-name
Project or namespace name where the RhapsodySE operator is installed.

The RhapsodySE operator is uninstalled from the namespace.

3. To uninstall Rhapsody Systems Engineering by using the Kubernetes command line, complete the following substeps:

a) Log in to your Kubernetes cluster .

b) Verify the current version of the subscribed operator by using the following command.

```
kubectl get subscription rhapsody-se-operator -n <namespace-name> -o yaml | grep
currentCSV
```

where

current-csv-version
Current cluster service version of the operator. Use current CSV information for this parameter.

namespace-name
Project or namespace name where the RhapsodySE operator is installed.

The RhapsodySE operator is uninstalled from the namespace.

c) Delete the subscription of the operator by using the following command.

```
kubectl delete subscription rhapsody-se-operator -n <namespace-name>
```

d) Delete the current subscription version of the operator in the target namespace by using the following command.

```
kubectl delete clusterserviceversion <current-csv-version> -n <namespace-name>
```

where

current-csv-version
Current cluster service version of the operator. Use current CSV information for this parameter.

namespace-name
Project or namespace name where the RhapsodySE operator is installed.

The RhapsodySE operator is uninstalled from the namespace.

Deleting the Rhapsody Systems Engineering instance

The new Rhapsody Systems Engineering instances are created based on system requirements, and old ones are removed. Rhapsody Systems Engineering instance gets its own namespace, and the Rhapsody Systems Engineering operator is installed in it.

Deleting Rhapsody Systems Engineering instance by using Red Hat OpenShift console

Deleting the Rhapsody Systems Engineering instance by using Red Hat OpenShift console includes deleting the Rhapsody Systems Engineering operator, secret, persistent volume, persistent volume claims, service accounts, and deleting the secrets that are associated with service account.

Procedure

1. Log in to Red Hat OpenShift Container Platform console as a cluster admininstrator. For more information about user roles, see “User roles mapping in Rhapsody Systems Engineering” on page 21.
2. Go to **Operators** > **Installed Operators**. From the **Project** list, select the project where you installed the Rhapsody Systems Engineering operator.

   The Rhapsody Systems Engineering operator appears in a table on the Installed Operators page.
3. Click **RhapsodySE** to select the Rhapsody Systems Engineering operator.
4. On the Operator Details page, click **RhapsodySE** tab to view the Rhapsody Systems Engineering instance details.
5. On the Rhapsody Systems Engineering table in the RhapsodySE-instance row, click the overflow menu and select **Delete RhapsodySE**. A window appears to confirm the delete action.
6. Click **Delete** to remove the Rhapsody Systems Engineering instance and the associated resources.
7. Delete the secret *<secret name>*.
   a) Go to **Workloads** > **Secrets**.
   b) From the **Project** list, select the project where you installed the Rhapsody Systems Engineering operator.
   c) In the <secret name> row, click the overflow menu and select **Delete Secret**. A window opens to confirm the delete action.
   d) Click **Delete** to delete the Secret. Repeat this procedure until all secrets that contain the word token as part of the secret name are deleted.
8. Delete persistent volume claim *<PVC>*.
   a) Go to **Storage** > **Persistent Volume Claims**.
   b) From the **Project** list, select the project where you installed the Rhapsody Systems Engineering operator.
   c) On the persistent volume claims table in the <PVC> row, click the overflow menu and select **Delete Persistent Volume Claim**. A window is shown to confirm the delete action.
   d) Click **Delete** to delete the persistent volume claim.
9. Delete persistent volume *<PV>*.
   a) Go to **Storage** > **Persistent Volumes**.
   b) On the Persistent Volumes table in the <PV> row, click the overflow menu and select **Delete Persistent Volume Claim**. A window is shown to confirm the delete action.
   c) Click **Delete** to delete the persistent volume.
10. Delete service accounts.
    a) Go to **User Management** > **Service Accounts**.

b) Select the service accounts named *<SA>* from the table and click **Delete Service Account** from the overflow menu . A window is shown to confirm the delete action

c) Click **Delete** to delete the service accounts.

11. Delete the secrets that are associated with service account *<SA>*.

a) **Workloads** > **Secrets**. The secrets are listed in a table on the Secrets page and some of them contain the word token as part of the secret name.

Important: Delete all secrets manually that contain the word token as part of the secret name.

b) To select the secret from the table, click **Delete Secret** from the overflow menu. A window is shown to confirm the delete action

c) Click **Delete** to delete the secret.

What to do next

Uninstall the Rhapsody Systems Engineering operator. For more information, see "Uninstalling Rhapsody Systems Engineering operator" on page 60.

Deleting Rhapsody Systems Engineering instance by using Red Hat Open Shift command line

Deleting the Rhapsody Systems Engineering instance by using Red Hat OpenShift command line includes deleting the Rhapsody Systems Engineering operator, secret, persistent volume, persistent volume claims, service accounts, and deleting the secrets that are associated with service account by running the commands on Red Hat OpenShift command line.

About this task

The Rhapsody Systems Engineering instance gets its own namespace, and the Rhapsody Systems Engineering operator is installed on it. The deletion of instance enables the operator to remove the corresponding deployed resources, such as pods and services that are associated with the instance.

Procedure

1. Log in to Red Hat OpenShift cluster by using the command line. Log in by using the appropriate user role, which has the privilege to delete theRhapsody Systems Engineering instance. For more information about user roles, see "User roles mapping in Rhapsody Systems Engineering" on page 21.
2. Log in to your cluster by using the following command.

```
oc login --token=<token> --server=<openshift-server-url>
```

where

token
Authentication token for the user to log in to the cluster.

openshift-server-url
Red Hat OpenShift cluster cluster server URL

3. Delete the Rhapsody Systems Engineering instance by using the following command.

```
oc delete rhapsodyseapp/<rhapsodyse-instance-name> -n <namespace-name>
```

where

rhapsodyse-instance-name
Rhapsody Systems Engineering instance name that you want to delete.

namespace-name
Project name where the Rhapsody Systems Engineering instance is installed.

4. Delete the secret *<secret-name>* using the following command.

```
oc delete secret <secret-name> -n <namespace-name>
```

5. Delete the persistent volume claim *<pvc-name>*.

```
oc delete pvc <pvc-name> -n <namespace-name>
```

6. Delete the persistent volume *<pv-name>*.

```
oc delete pv <pv-name>
```

7. Delete service accounts *<sa-account-name>*.

```
oc delete serviceaccount <sa-account-name> -n <namespace>
```

Deleting Rhapsody Systems Engineering instance by using Kubernetes

Deleting Rhapsody Systems Engineering instance by using Kubernetes includes deleting the Rhapsody Systems Engineering operator, secret, persistent volume, persistent volume claims, service accounts, and deleting the secrets that are associated with service account by running the commands on Kubernetes.

About this task

All associated resources that are created by Rhapsody Systems Engineering operator are deleted when you delete the Rhapsody Systems Engineering instance.

Procedure

1. Display the Rhapsody Systems Engineering instance by using the following command.

```
kubectl get rhapsodyseapp  -n <namespace-name>
```

2. Delete the Rhapsody Systems Engineering instance by using the following command.

```
kubectl delete rhapsodyseapp/<rhapsodyse-instance-name> -n <namespace-name>
```

where

rhapsodyse-instance-name
Rhapsody Systems Engineering instance name that you want to delete.

namespace-name
Project name where the Rhapsody Systems Engineering instance is installed.

3. Delete the secret *<secret-name>* by using the following command.

```
kubectl delete secret <secret-name> -n <namespace-name>
```

4. Delete the persistent volume claim *<pvc-name>*.

```
kubectl delete pvc <pvc-name> -n <namespace-name>
```

5. Delete the persistent volume *<pv-name>*.

```
kubectl delete pv <pv-name>
```

6. Delete service accounts *<sa-account-name>*.

```
kubectl delete serviceaccount <sa-account-name> -n <namespace>
```

Upgrading

The upgrade of Rhapsody Systems Engineering is managed by the Rhapsody Systems Engineering operator.

Upgrade strategy for the Rhapsody Systems Engineering instance

The OLM manages the lifecycle of operators by using channels. Rhapsody Systems Engineering on Red Hat OpenShift facilitates the OLM and channels to support the product upgrades. When an interim fix, a mod, a release, or a version update is available, the OLM creates an update request. After the update request is approved according to the approval strategy, the Rhapsody Systems Engineering operator is upgraded to the new version by OLM.

When you install the Rhapsody Systems Engineering operator, you can choose the appropriate channel name. The Rhapsody Systems Engineering operator manages the version of the Rhapsody Systems Engineering application that is installed on the cluster.

Optionally, initiate the upgrade by modifying Rhapsody Systems Engineering instance custom resource and specifying the new version in the `spec.version`. The Rhapsody Systems Engineering operator then upgrades the Rhapsody Systems Engineering applications to the specified version.

The Rhapsody Systems Engineering upgrade supports the following scenarios:

- Interim fix updates take place within the same channel.
- Version or release or mod level updates take place when the Red Hat OpenShift administrator changes the channel.

Channel strategy

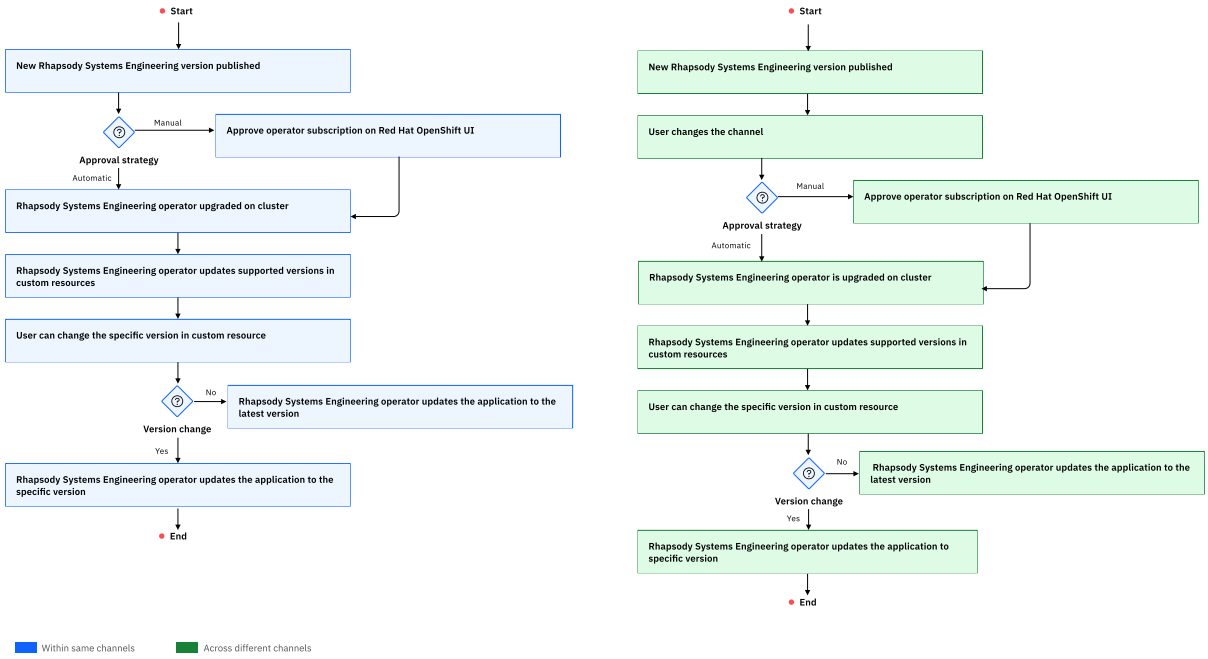
The Rhapsody Systems Engineering operator is available in the General Availability (GA) channel. The channel name convention is X.Y.Z.

Upgrade scenarios

The Rhapsody Systems Engineering operator and Rhapsody Systems Engineering application are always deployed within a namespace in the Red Hat OpenShift cluster.

When you require more than one Rhapsody Systems Engineering instance to cater to development, testing, and production scenarios, you can deploy separate Rhapsody Systems Engineering instances for each scenario in different namespaces. By using a separate namespace for each Rhapsody Systems Engineering instance, you can isolate the resources, configure role based access control (RBAC) permissions, and independently manage each Rhapsody Systems Engineering instance without affecting others. It also helps you to manage different release levels for each Rhapsody Systems Engineering instance and upgrade them independently. For example, you can upgrade the Rhapsody Systems Engineering instance that is deployed in the development namespace independently without affecting the Rhapsody Systems Engineering instance that is deployed in the testing namespace.

The following flow chart explain the upgrade steps within the same channel and across different channels.



Upgrading the Rhapsody Systems Engineering operator

You can apply the version or release or mod level updates to the Rhapsody Systems Engineering operator by upgrading the Rhapsody Systems Engineering operator that supports the new Rhapsody Systems Engineering version or release or mod level updates. Upgrade the Rhapsody Systems Engineering operator by using Red Hat OpenShift console or by using the command line inputs that include Red Hat OpenShift commands or Kubernetes commands.

Before you begin

Upgrading the Rhapsody Systems Engineering operator by using Red Hat Open Shift console

You can upgrade Rhapsody Systems Engineering operator by switching to the appropriate channel. The Rhapsody Systems Engineering operator upgrade process depends on the approval strategy that was selected during the Rhapsody Systems Engineering operator installation.

Before you begin

1. Verify that the X.Y.Z channel is available for upgrade, where X.Y.Z identifies the Rhapsody Systems Engineering version that you want to upgrade to.
2. Create an internal TLS certificate with DNS names suitable for internal service communication. For more information, see “Creating TLS certificates for Rhapsody Systems Engineering” on page 38.

Procedure

1. Log in to the Red Hat OpenShift Container platform.
2. Go to **Operators** > **Installed operators**
3. Click the Rhapsody Systems Engineering operator link available in the **Name** field.
4. For the subscription with an automatic approval strategy, complete the following step:

a) On the **Subscription** tab, under the **Update channel** column, click  to choose the channels that you want to update to.

5. For the subscription with a manual approval strategy, complete the following steps:
   a) On the **Subscription** tab, click the **1 requires approval** link.
   b) Click **Preview InstallPlan**. The manual installation plan details are shown.
   c) Click **Approve**. On the **Components** tab, you can see the status of various components that are getting updated with the Rhapsody Systems Engineering minor or major version. The Rhapsody Systems Engineering operator is upgraded.
6. To verify that the Rhapsody Systems Engineering operator channel is updated, go to **Operators** > **Installed operators**.

Upgrading the Rhapsody Systems Engineering operator by using command line inputs

You can upgrade Rhapsody Systems Engineering operator by switching to the appropriate channel. The Rhapsody Systems Engineering operator upgrade process depends on the approval strategy that was selected during the Rhapsody Systems Engineering operator installation. Upgrading the Rhapsody Systems Engineering operator by using command line inputs includes upgrading the operator by either Red Hat OpenShift or Kubernetes commands.

Before you begin

1. Ensure that the existing Rhapsody Systems Engineering setup is functional without any errors and all Rhapsody Systems Engineering applications are accessible.
2. Ensure that you have the appropriate permissions to upgrade the Rhapsody Systems Engineering operator.
3. Create an internal TLS certificate with DNS names suitable for internal service communication. For more information, see “Creating TLS certificates for Rhapsody Systems Engineering” on page 38.

About this task

Upgrading between the major release versions of Rhapsody Systems Engineering requires a change of operator channel, which is always a manual action for the user.

Procedure

1. Log in to the Red Hat OpenShift container platform or Kubernetes cluster by running the following commands.

   Red Hat OpenShift container platform

   ```
   oc login --username=<cluster admin user> --password=<cluster admin password>
   ```

   Kubernetes cluster

   ```
   kubectl login --username=<cluster admin user> --password=<cluster admin password>
   ```

2. Set the default project or namespace as the one that contains the operator to be upgraded.

   Red Hat OpenShift container platform

   ```
   oc project <namespace>
   ```

   Kubernetes cluster

   ```
   kubectl config set-context --current --namespace=<namespace>
   ```

3. Get a list of all the subscriptions in this project or namespace.

Red Hat OpenShift container platform

```
oc get subscriptions
```

Kubernetes cluster

```
kubectl get subscriptions
```

This returns the list of subscriptions in the namespace.

For the information that is returned, identify the subscription name for the operator that you want to upgrade from the NAME column, and the corresponding package name from the PACKAGE column.

4. For the subscription with an automatic approval strategy, complete the following steps:
   a) Verify that the channel to which you want to upgrade is available.
   b) Replace <package-name> in the following command by using the information returned in the previous step:

Red Hat OpenShift container platform

```
oc get packagemanifest <package-name> -o jsonpath='{.status.channels[*].name}'
```

Kubernetes cluster

```
kubectl get packagemanifest <package-name> -o jsonpath='{.status.channels[*].name}'
```

5. For the subscription with a manual approval strategy, complete the following steps. Replace the <subscription-name> in the following commands by using the information in Step “3” on page 67.
   a) Run the following command to check if your subscription is pending.

Red Hat OpenShift container platform

```
oc get subscription <subscription-name> -o yaml
```

Kubernetes cluster

```
kubectl get subscription <subscription-name> -o yaml
```

   b) Preview the installation plan by using the following command. The manual installation plan details are shown.

Red Hat OpenShift container platform

```
oc get installplan -n <namespace>
```

Kubernetes cluster

```
kubectl get installplan -n <namespace>
```

   c) Verify if the state of the subscription is pending.

```
state: UpgradePending
```

   d) Approve the upgrade by using the following command.

Red Hat OpenShift container platform

```
oc patch installplan <installplan-name> --type merge -p '{"spec":{"approved":true}}'
```

Kubernetes cluster

```
kubectl patch installplan <installplan-name> --type merge -p '{"spec":
{"approved":true}}'
```

6. Optional: Patch the subscription to move to the desired upgrade channel, replacing <subscription-name> and <channel> in the following commands.

Red Hat OpenShift container platform

```
oc patch subscription <subcription-name> --patch '{"spec":{"channel":"<channel>"}}' --
type=merge
```

Kubernetes cluster

```
kubectl patch subscription <subcription-name> --patch '{"spec":{"channel":"<channel>"}}'
--type=merge
```

7. To confirm whether the upgrade is successful, run the following command and check that the channel is correct.

Red Hat OpenShift container platform

```
oc get subscriptions
```

Kubernetes cluster

```
kubectl get subscriptions
```

What to do next

You can upgrade the instances after the operators that are used are upgraded.

Upgrading the Rhapsody Systems Engineering instance

Learn how to upgrade Rhapsody Systems Engineering on Red Hat OpenShift Container Platform console or by using command line inputs. It involves backing up of your database, running scripts, and validating that everything works as expected after the upgrade.

Before you begin

1. Ensure that the Rhapsody Systems Engineering operator is upgraded. For more information, see “Upgrading the Rhapsody Systems Engineering operator ” on page 66.
2. Verify that you have the appropriate permissions to upgrade the Rhapsody Systems Engineering instance.

Upgrading the Rhapsody Systems Engineering instance by using Red Hat OpenShift console

Learn how to upgrade Rhapsody Systems Engineering on Red Hat OpenShift Container Platform console. It involves backing up of your database, running scripts, and validating that everything works as expected after the upgrade.

Procedure

1. On the **Installed Operators** page, click the Rhapsody Systems Engineering operator link available in the **Name** field.
2. Go to the **Rhapsody Systems Engineering** tab and click the Rhapsody Systems Engineering instance link available in the **Name** field.
3. Go to the YAML view of Rhapsody Systems Engineering custom resource specification and search for `version: X.Y.Z`, where `X.Y.Z` represents the installed Rhapsody Systems Engineering version.
4. Update the `version` property with the available Rhapsody Systems Engineering version and click **Save**. The upgrade of Rhapsody Systems Engineering instance to the new Rhapsody Systems Engineering version starts.
5. Verify that the Rhapsody Systems Engineering instance after the upgrade process is completed. For more information, see “Verifying the resources created after Rhapsody Systems Engineering instance creation” on page 58.

Upgrading the Rhapsody Systems Engineering instance by using command line inputs

Learn how to upgrade Rhapsody Systems Engineering by using Red Hat OpenShift command line inputs or on Kubernetes cluster. It involves backing up of your database, running scripts, and validating that everything works as expected after the upgrade.

Procedure

Run the following command.

Red Hat OpenShift

```
oc patch RhapsodySEApp rse-instance --type=merge -p '{“spec”:{“version”:“<version>”}}
```

Kubernetes

```
kubectl patch RhapsodySEApp rse-instance --type=merge -p '{“spec”:{“version”:“<version>”}}
```

Upgrading the Rhapsody Systems Engineering database from version 1.1.0 to the subsequent versions

Learn how to upgrade Rhapsody Systems Engineering from version 1.1.0 to the subsequent versions. It involves backing up of your database, running scripts, and validating that everything works as expected after the upgrade.

Procedure

1. As a system administrator, before making any changes, take a back up of your database so that you can restore it if anything goes wrong during the process.
2. Run the following script to update user roles by project. It retrieves the required data to ensure a smooth upgrade.

```
SELECT usr_rl.project_id, prj.docs->>'name' as project_name, usr_rl.email, usr_rl.role FROM
sysml.user_roles as usr_rl
JOIN sysml.project as prj
    ON usr_rl.project_id = prj.id
ORDER BY project_id;

/* Uncomment below query to clear the table once all the data is retrieved and saved to some
file (tsv, csv, excel)*/
/*
delete from sysml.user_roles;
```

3. Save the retrieved data in a file. You can export the retrieved data to a file, such as, TSV, CSV, or Excel for secure storage.
4. To clear the `user_roles` table after saving the data, uncomment and run the following query:

```
DELETE FROM sysml.user_roles;
```

5. Reconfigure the mapping of user roles after the upgrade is complete. This is a one-time task.

Administering

After you start Rhapsody Systems Engineering, you can begin performing administrative tasks, such as creating projects, managing users, managing open services for lifecycle collaboration integrations, creating links between projects, and creating project usages.

Related concepts

“Managing open services for lifecycle collaboration integrations” on page 81

Managing open services for lifecycle collaboration (OSLC) integrations for Rhapsody Systems Engineering includes integrating other products with Rhapsody Systems Engineering, integrating Rhapsody Systems Engineering with IBM Engineering Lifecycle Management products, and creating associations between the projects of Rhapsody Systems Engineering and the projects of other friend applications.

"Managing links" on page 138
Managing links in Rhapsody Systems Engineering involves several key actions: creating new links between model elements and artifacts of friend applications, viewing detailed information about existing links, and removing links when necessary. This functionality allows for comprehensive management of relationships between Rhapsody Systems Engineering model elements and artifacts in friend application, ensuring effective traceability and alignment throughout the development process.

Related tasks

"Creating and archiving projects" on page 78
Create your projects from the **Projects** page based on a template that includes general, interconnection, use case, and action flow diagrams. They serve as the starting points to create part interconnection, use case, or action flow diagrams. You can also choose to archive them when they are no longer in use.

"Creating project usages" on page 80
You can reuse the libraries from your existing projects by creating project usages.

"Managing project usages" on page 80
Managing project usages include updating or searching the project usages for a project.

OIDC authentication

Rhapsody Systems Engineering supports OIDC authentication with providers, such as IBMid authentication, Microsoft Entra ID, Keycloak, and Jazz authorization server (JAS).

Any compliant OIDC provider should work with Rhapsody Systems Engineering. The following providers are validated and recommended for use:

- IBMid
- Keycloak
- Microsoft Entra ID

Related tasks

"Authenticating with OIDC" on page 32
Rhapsody Systems Engineering uses OpenID Connect (OIDC) for secure authentication. OIDC is a secure authentication protocol that uses OAuth 2.0 to provide an identity layer, allowing clients to verify a user's identity based on the authentication performed by an authorization server.

IBMid authentication for Rhapsody Systems Engineering

Rhapsody Systems Engineering includes IBMid authentication, which allows the business users to use single sign-on. Since the OIDC account of a users is authenticated as a single sign-on, users can log in and use more than one tool or application at the same time, without invalidating the first session, when IBMid is enabled on the Rhapsody Systems Engineering environment. When a user logs out of an application or tool, the logout page displays the cause of logout.

- If the user logs out normally, the logout page displays the message, "You have been successfully logged out. Click here to log in".
- If the user is logged out because of some event such as session timeout and store user is not authorized, the logout page displays the appropriate error message.

What is IBMid?

IBMid is an external identity service to allow customers to engage digitally with IBM. It provides an authentication and registration service for IBMid. IBM Identity Service is the cloud-based identity access and management solution, which provides identity and single sign-on services for IBM applications. You can use IBMid for single sign-on (SSO) for Rhapsody Systems Engineering.

The following are the benefits of using IBMid authentication:

- You can log in to IBM applications and offerings to engage with IBM digitally.
- You get a tool to reset your password.
- You can avail email verification to ensure communications is directed to the correct location.
- You get the ability to create an IBMid and access products.

Note: For IBM users, w3ID is the OIDC authentication provider used within IBM. If you deploy the product internally within IBM, configure your authentication settings to use w3ID.

Microsoft Entra ID

Microsoft Entra ID service allows the users to log in to a Rhapsody Systems Engineering instance. Once configured, this service enables users who belong to specific organizational directories to access Rhapsody Systems Engineering. The system can be set to either include or exclude personal Microsoft accounts. Microsoft Entra ID is the new name for Azure Active Directory.

Setting up a Microsoft Entra ID service

As a server administrator, setting up a Microsoft Entra ID services involves creating a new registration for the application, authenticating it, and adding users to the application.

Before you begin

A Microsoft Entra ID service must be provisioned on Microsoft Azure.

About this task

If you already have an Entra ID service, you can skip this section.

Procedure

1. On the Microsoft Azure portal, search for Microsoft Entra ID and click it.
2. Once provisioned, go to its Administrator dashboard.
3. On the left panel, click **App registrations**, and then click **+ New registration**.
4. In the **Name** box, specify a description about Rhapsody Systems Engineering, such as "RSE local dsh".
5. Under **Supported account types**, select an appropriate option.

   Note: If you select **I've been selecting Accounts in this organizational directory only (Default Directory only - Single tenant)**, perform step “11” on page 73 to add the list of valid users to the default directory of the service.
6. In the **Redirect URI (optional)** list, select **Web**, and type your Rhapsody Systems Engineering redirect URL in the input box.

   Note:

   - Do not use the SPA option.
   - The redirect URL is specific to your Rhapsody Systems Engineering instance. It follows the following format: `https://YOUR-PUBLIC-URL-HERE/nextapi/auth/callback/oidc`. If you are running Rhapsody Systems Engineering locally, then it follows the following format: `https://localhost:3000/nextapi/auth/callback/oidc`, but real deployments requires to retrieve the value to use for themselves.
   - If your Rhapsody Systems Engineering public addressable URL is not using port 443, then add the same port in the redirect URL.
   - Your Entra ID service rejects requests that have a origin header that does not match this value, so this must match up with your installation.

7. Click **Register**.
8. Click **Authentication**.
9. In the Implicit grant and Hybrid flows section, select **Access tokens (used for implicit flows)** and **ID tokens (used for implicit and hybrid flows)**.
10. Click **Owners**, and then click **Add owners**.
    - Review the Microsoft account, which is used to create this Entra ID service, then click **Select**.
11. If you selected **Accounts** in this organizational directory (Default Directory only - Single tenant) in step "6" on page 72, then set the list of valid accounts that can log in.
12. On the left panel, click **Users**, and then click **New user** to add the Microsoft accounts that can log in to Rhapsody Systems Engineering.

Setting up a Rhapsody Systems Engineering CR to use Entra ID settings

As a server administrator, edit your custom resource (CR) to consume the Entra ID settings.

Procedure

1. Open the CR. Consider the following sample:

```
oidc:
 wellKnownEndpoint: https://login.microsoftonline.com/abcdef-abcd-abcd-abcd-652c0d22fe9a/v2
 introspectionEndpoint: https://graph.microsoft.com/oidc/userinfo
 clientId: secret::rse-application-secrets::oidcClientId
 clientSecret: secret::rse-application-secrets::oidcClientSecret
 signingKey: abcdefghijklmnoqrstuvwxyz
 scope: openid profile email offline_access
```

2. Get the wellKnownEndpoint setting for your Entra ID service. This link contains a tenant ID unique to you.
   a) Open your Entra ID admin portal.
   b) On the left panel, click **App registrations**.
   c) Click the **Endpoints** icon.
   d) Scroll down to find the OpenID Connect metadata document.
   e) Copy this value, and enter it in your CR for the value of the `oidc.wellKnownEndpoint` setting.
3. Set `introspectionEndpoint` setting. You can copy the following value: `https://graph.microsoft.com/oidc/userinfo`.
4. Create credentials, `clientId` and `clientSecret` for your application.
   a) Open your Entra ID admin portal.
   b) On the left panel, click **App registrations**.
   c) In the App registrations section, click **All applications** and search for the Rhapsody Systems Engineering application registration in step "6" on page 72, and click it.
   d) Click **Overview**, and find the text near Application (client) ID.
      i) Copy this value, and enter it in your CR for the value of the `oidc.clientId` setting.
      ii) In the Manage section, click **Certificates & secrets**.
      iii) Specify a description for your Rhapsody Systems Engineering application, and then click **+ New client secret**.
      iv) In the table, copy the text from the **Value** column. Do not leave the page until you have copied this value. You will not be able to read this value again.
      v) Paste the text it in your CR for the value of the `oidc.clientSecret` setting. Do not use the Secret ID text that you see in this table.

vi) For the signing key, specify a random string of characters with a length between 10 and 32 characters. The signingKey should be relatively unique as it is used to build user session tokens by the Rhapsody Systems Engineering client.

e) Edit the scope setting. Set the setting oidc.scope value to `openid profile email offline_access`.

5. Save your Rhapsody Systems Engineering deployment.

Keycloak

Keycloak is an open-source identity and access management (IAM) solution. It offers secure management of user identities across different systems and applications. Once configured, this service enables the users who are registered with it to access Rhapsody Systems Engineering securely.

Configuring Keycloak

The realm and client must be properly configured to enable authentication and authorization in the OIDC setup.

Before you begin

A keycloak server must be running.

Creating a realm

Creating a realm in Keycloak is required to configure and organize authentication and authorization settings.

Procedure

1. In the Keycloak Admin Console, on the left panel, click the **realm** menu.
2. Click **Create realm**.
3. In the **Realm name** box, specify a name.
4. Click **Create**.

Configuring a realm

Configure the realm in the keycloak by adding the publicly exposed base URL of the keycloak server.

Procedure

1. In the keycloak admin console, on the left panel, from the **realm** menu, select the realm that was created.
2. On the left panel, click **Realm settings**.
3. In the General section, in the **Frontend URL** field, enter the value of `<keycloak-url>`, replacing `<keycloak-url>` with the publicly exposed base URL of the keycloak server.
4. Click **Save**.

Creating a client

A client represents an application that keycloak authenticates and authorizes for the users.

Procedure

1. In the keycloak admin console, on the left panel, from the **realm** menu, select the realm that was created.
2. On the left panel, click **Clients**.
3. Under **Clients** list, click **Create Client**.

4. In the **General settings** section, in the **Client ID** box, specify a client-id, and then click **Next**.
5. Under **Capability configuration** section, enable **Client Authentication**, and then click **Next**.
6. In the **Login settings** section, complete the following substeps:
   a) Create a valid redirect URI entry by using the value `<frontend-url>/nextapi/auth/callback/oidc`, replacing `<frontend-url>` with the base frontend URL.
   b) Create a web origins entry with the value `<frontend-url>`, replacing `<frontend-url>` with the base frontend URL.
   c) Click **Save**.

Setting up a Rhapsody Systems Engineering CR to use keycloak settings

As a server administrator, edit your custom resource (CR) to consume the keycloak settings.

Procedure

1. Open the CR. Consider the following sample.

```
oidc:
  wellKnownEndpoint: <keycloak-url>/realms/<realm-name>/.well-known/openid-configuration
  introspectionEndpoint: <keycloak-url>/realms/<realm-name>/protocol/openid-connect/token/
introspect
  clientId: secret::rse-application-secrets::oidcClientId
  clientSecret: secret::rse-application-secrets::oidcClientSecret
```

2. Replace the placeholders in `wellKnownEndpoint` and `introspectionEndpoint` with the following information:

   <keycloak-url>
   The base url of the keycloak server.

   <realm-name>
   The realm name of the realm that was created in keycloak.
3. Replace the value of `clientId` with the `client-id` that was created in Step “4” on page 75
4. Replace the value of client secret with the client secret of the client that was created in Step “6” on page 75
   a) In the keycloak admin console, on the left panel, from the **realm** menu, select the realm that was created.
   b) Click **Clients**, and then click **Credentials**. The client secret of the configured client will be displayed under Client secret.

Jazz Authorization Server

Jazz Authorization Server can serve as an OIDC provider for single sign-on (SSO) for Rhapsody Systems Engineering. It enables web applications to delegate user authentication further to a third-Party identity provider through OIDC.

Registering Rhapsody Systems Engineering as a client

Registering Rhapsody Systems Engineering as a client includes submitting the JAS client registration request by using an appropriate REST client, such as postman.

Procedure

1. Select method as POST.
2. Enter JAS registration URL in following format. Replace the values in <> brackets with actual value.

```
https://<jas_host_name>:<port_number>/oidc/endpoint/jazzop/registration
```

3. In the **Authorization** tab, select **Auth Type** as **Basic Auth** and provide JAS user credentials.

4. On the **Headers** tab, add the following header.

```
Content-Type: application/json
```

5. On the **Body** tab, add the following as raw data.

   Replace the values in <> brackets with actual value.

```
{
    "client_name": "<rse_client_name>",
    "client_id": "<rse_client_id>",
    "client_secret": "<rse_client_secret>",
    "scope": "openid profile email general",
    "preauthorized_scope": "openid profile email general",
    "functional_user_id": "<jas_admin_user_id>",
    "functional_user_groupIds": ["JazzAdmins"],
    "introspect_tokens": true,
    "redirect_uris": ["https://<rse_host_name>:<port_number>", "https://
<rse_host_name>:<port_number>/nextapi/auth/callback/oidc"],
    "trusted_uri_prefixes": [],
    "grant_types": ["authorization_code", "client_credentials", "password", "refresh_token",
"implicit", "urn:ietf:params:oauth:grant-type:jwt-bearer"],
    "response_types": ["code", "token", "id_token token"]
}
```

6. Click **Send**. In the **Response** pane, the Rhapsody Systems Engineering client details are shown that includes details, such as `client_id` and `client_secret`.

Updating the configuration file

To start using as JAS as an OIDC provider for Rhapsody Systems Engineering, update or add attributes to operator configuration file.

Procedure

Update or add the following attributes to the operator configuration file. Replace the values in <> with actual values.

```
oidc:
    clientId: <rse_client_id>
    clientSecret: <rse_client_secret>
    wellKnownEndpoint: https://<jas_host_name>:<port_number>/oidc/endpoint/jazzop/.well-known/
openid-configuration
    introspectionEndpoint: https://<jas_host_name>:<port_number>/oidc/endpoint/jazzop/introspect
    customIdentityClaim: sub
    signingKey: alskjdfsdfjidufHHKHFD
    clientData:
        authorization_signed_response_alg: HS256
        id_token_signed_response_alg: HS256

cspHeaders:
    [
      "default-src 'self' *",
      "script-src 'self' 'unsafe-eval' 'unsafe-inline' https://cdn.jsdelivr.net/ https://
cdn.walkme.com/ https://playerserver.walkme.com/",
      "style-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net/ https://cdn.walkme.com/",
      "img-src 'self' https: blob: data:",
      "font-src 'self' data: *",
      "connect-src 'self' *",
      "frame-src 'self' https://<elm_host_name>:<port_number>/",
      "worker-src 'self' blob:",
    ]

serviceToken: <create secret phrase that would be used for OSLC APIs (250 chars max)>

linkIndexProvider:
    ldxEndpoint: https://<elm_host_name>:<port_number>/ldx
    isJASEnabled: <true if LDX is using JAS as OIDC provider, else false>
```

Note: For the signing key, specify a random string of characters with a length between 10 and 32 characters. The signingKey should be relatively unique as it is used to build user session tokens by the Rhapsody Systems Engineering client. Edit the scope setting.

Managing users

Manage users by adding the users or changing their administrative privileges. The **All users** page helps you to add and manage the users for the projects of Rhapsody Systems Engineering. You require the roles of project administrator or server administrator to add and manage users.

About this task

Note: Administrators can assign roles to the users who are registered in the system, or logged in at least once. After their initial login, which captures the user's information, the users appear in the system, allowing administrators to assign appropriate roles.

Procedure

1. In the **Projects** page, select the project for which you want to add or manage users.
2. In the project row, click the **Options** icon, and then click **Manage users**. In the **All users** page, the users assigned to that project are displayed.

   Note: To manage the users for multiple projects at a time, select the projects, and click **Add users**.
3. To add users, click **Add users**.
4. Optional: To revoke a user role, select the user role, and click the **Revoke** icon.
5. Select the user role that you want to assign.
6. To add multiple users to a project, specify the email IDs of the users separated by a comma.
7. Click **Add**.

Roles

A user role is defined by a set of tasks that a specific role is given permission to perform. In Rhapsody Systems Engineering, the three predefined roles are: server administrator, project administrator, and practitioner.

Server administrator

- Create and archive projects.
- Create and manage public branches and tags.
- Create and manage your project usages.
- Assign other users as project administrators or practitioners to the projects available on your system.
- Export and import projects.
- Create and manage outbound friend connections.
- Establish and manage project associations.
- Customize the appearance of the application by updating the configuration files.

Project administrator

- Create and manage public branches and tags.
- Assign other users as project administrators or practitioners to the projects that you have access to.
- Create and view your project usages.
- Establish and manage project associations.
- Export projects.

Practitioner

- Create, work, and manage public branches and tags.
- View the project usages for the projects that you have access to.

- Export projects.

Reviewer

- View projects, elements, branches, and tags
- View commits and configurations
- Search elements
- View associations and relationships

Reviewing projects

As a reviewer, you can access and review your team members' projects. You get the privileges to view project elements, branches, tags, commits, configurations, as well as associations and relationships.

Before you begin

Ensure that you are assigned as a reviewer for the project. For more information, see “Managing users” on page 77.

Procedure

1. Go to the Projects page.
2. Identify the projects marked with the Reviewer icon.
3. Select the project and choose to perform the following actions.
   - View projects, elements, branches, and tags.
   - View commits and configurations.
   - Search elements.
   - View associations and relationships.

Managing projects

As a server administrator or a project administrator, you can perform various tasks on your projects.

In the **My Projects** page, as a project administrator, you can see an indicator icon () next to the project name for projects where you have admin access. This icon serves as a visual indication, allowing you to quickly identify which projects you can manage.

Creating and archiving projects

Create your projects from the **Projects** page based on a template that includes general, interconnection, use case, and action flow diagrams. They serve as the starting points to create part interconnection, use case, or action flow diagrams. You can also choose to archive them when they are no longer in use.

Procedure

1. Log in to Rhapsody Systems Engineering as a server administrator.
2. In the **Projects** page, click **Create**.
3. In the **Name** field, specify an appropriate name of the project.
4. Optional: In the **Template** list, choose the template that you want to use for the project.
   - To get a project with views that you can quickly use, select **SimpleTemplate** .
   - To use standard and reusable elements within your system models, select **Harmony**.
   - To use an existing model as a base, customize, and extend it as a usable template, select **Car interior control sample**.
5. Optional: In the **Description** field, specify an appropriate description for the project.

6. Click **Create**.
7. To archive your projects that are no longer in use, in the project row, click the **Options** (⋮) icon, and then click **Archive project**.

   Note: If you want to archive multiple projects at a time, select the projects and click **Archive**.

Related concepts

"Roles" on page 77
A user role is defined by a set of tasks that a specific role is given permission to perform. In Rhapsody Systems Engineering, the three predefined roles are: server administrator, project administrator, and practitioner.

Related tasks

"Creating associations between projects" on page 79
As a server or project administrator, you can establish an association between the projects of Rhapsody Systems Engineering and the projects of other friend applications.

Importing projects

As a server administrator, you can import your projects from your local file system to the Projects page of Rhapsody Systems Engineering.

Procedure

1. In the **Projects** page, click the **Import project** ( ) icon.
2. Specify the project name, and then click **Click to select** to select the ZIP file that you want to import. The imported project displays in the **All projects** list.

Creating associations between projects

As a server or project administrator, you can establish an association between the projects of Rhapsody Systems Engineering and the projects of other friend applications.

Procedure

1. In the Projects listing page, click the **Options** icon for the project, and click **View association**.
2. In the **Associations** page, click **Add association**.
3. From the **Friend application** list, choose from one of the applications that are registered with Rhapsody Systems Engineering. To associate a local project area of Rhapsody Systems Engineering with the current project, select the local option from the list.
4. From the **Association type** list, select an appropriate type of association and click **Next**.
5. If you are prompted to authenticate, log in to the selected friend application by using your credentials.
6. In the **Select artifact container** section, select the target projects to associate with your current project.
7. Click **Add association**.
8. Optional: To edit an existing association, in the **Associations** page, click the **Edit** icon for the association.
9. Optional: To delete an existing association, in the **Associations** page, click the **Delete** icon for the association.

Creating project usages

You can reuse the libraries from your existing projects by creating project usages.

Before you begin

You require the server administrator or project administrator privileges to create project usages.

Procedure

1. In the **My Projects** page, select the project in which you want to reuse the libraries of existing projects that provides libraries for import.
2. Click the **Options** icon, and then click **Create project usages**.
3. In the **Projects** list, select the project that provides libraries for import.
4. In the **Branch or tag** field, select an appropriate branch or tag of that project and click **Create**. The project usage is created.
5. To change the package to a library package, complete the following substeps:
   a) Open the project that has libraries for import in the project editor and navigate to the **Browser view** panel.
   b) Click **Options** (⋮) icon, and then click **Change to library package**. The package is removed from the Browser view and it appears in the list of libraries in your Library view.
6. To import the library package into your target project, complete the following substeps:
   a) Go to the **My Projects** page, and open the target project in the project editor.
   b) In the project editor, navigate to the Browser view panel, and click the **Options** (⋮) icon.
   c) Click **Import library**.
   d) Select one or multiple libraries that you want to import.
   e) Click **Import**. The imported libraries appears in the library view of the current project.

Related concepts

“Roles” on page 77

A user role is defined by a set of tasks that a specific role is given permission to perform. In Rhapsody Systems Engineering, the three predefined roles are: server administrator, project administrator, and practitioner.

Managing project usages

Managing project usages include updating or searching the project usages for a project.

Before you begin

You require the server administrator privileges to manage project usages.

Procedure

1. In the **My Projects** page, for a project, click the **Options** icon and then click **Manage project usages**.
2. To update the project usage, click the branch of a project and then select the branch that you want to use. If you want to undo your changes, click the **Reset** icon.
3. To delete a project usage, select it and click the **Delete** icon. If you want to undo your changes, click the **Reset** icon.

   Note: Do not delete the project usages that contain system libraries.
4. To review your changes, click **Review** and to save your changes, click **Update**.

Related concepts

"Roles" on page 77

A user role is defined by a set of tasks that a specific role is given permission to perform. In Rhapsody Systems Engineering, the three predefined roles are: server administrator, project administrator, and practitioner.

Managing open services for lifecycle collaboration integrations

Managing open services for lifecycle collaboration (OSLC) integrations for Rhapsody Systems Engineering includes integrating other products with Rhapsody Systems Engineering, integrating Rhapsody Systems Engineering with IBM Engineering Lifecycle Management products, and creating associations between the projects of Rhapsody Systems Engineering and the projects of other friend applications.

Related information

Open Services for Lifecycle Collaboration integrations

Integrating other products with Rhapsody Systems Engineering

Learn how to integrate other products. For example: IBM Engineering Lifecycle Management which is OSLC compliant with Rhapsody Systems Engineering. You can then associate your projects with projects of other products.

About this task

The server administrator can use the **Administration** page to manage existing friends or add new friends to Rhapsody Systems Engineering.

Procedure

1. To open the **Administration** page, click the **Administration** icon under the main menu of the Rhapsody Systems Engineering page.
2. To view the list of friends configured for Rhapsody Systems Engineering, click **Connections** and then go to the **Friends (Outbound)** section.
3. You can choose to edit or delete an existing friend.
4. To add a friend, complete the following steps.
   a) Click **Add friend**.
   b) In the **Root service URL** field, specify the root services URL of the application that you want to add as a friend.
   c) In the **Name** field, specify the name of friend application, and click **Next**.
   d) In the **OAuth secret** field, enter a code to associate with the new OAuth consumer key of the application.
   e) Select **Trusted** to designate the friend as a trusted consumer. Trusted consumers can share authorization with other trusted consumers and do not require user approval to access data.
   f) Click **Create friend**. If the connection to the friend application is successful, a message confirms that the friend is added and a provisional key is generated. Click **Next**.
   g) Click the **Grant access for the provisional key** link.
   h) Log in to the friend application as an administrator.
   i) For Global Configuration, provide functional user ID to perform background tasks.
   j) Click **Approve** to approve the consumer key, and then click **Finish**.

Integrating Rhapsody Systems Engineering with IBM Engineering Lifecycle Management products

You can integrate Rhapsody Systems Engineering with IBM Engineering Lifecycle Management products by managing URL allowlisting and establishing friend relationships. URL allowlisting enables

communication with IBM Engineering Lifecycle Management products by adding its domains to a permitted list. Establishing friend relationships with IBM Engineering Lifecycle Management include linking the Rhapsody Systems Engineering with IBM Engineering Lifecycle Management application by providing its location and an OAuth secret, facilitating secure interactions and data exchange between them.

About this task

For more information, see:

- Managing the URL allowlist
- Establishing friend relationships

Managing the URL allowlist

The URL allowlist permits applications, clients, and web pages for the Rhapsody Systems Engineering to send requests to other sites.

Procedure

1. To open the Administration page, click the **Administration** icon under the main menu of the Rhapsody Systems Engineering page.
2. To view the list of outbound URLs configured for Rhapsody Systems Engineering, click **Connections**, and then go to the Allowlist (Outbound) section.
3. To add a URL to the allowlist, click **Add new URL**.
4. Type a base URL in the format `scheme://domain:port/`, and click the **Apply** icon located near the text area.
5. Optional: To edit a domain name in the URL allowlist, hover over the domain name in the URL allowlist section and click the **Edit** icon. After making changes, click the **Save** icon.
6. Optional: To delete a domain name in the URL allowlist , hover over the domain name in the URL allowlist section and click the **Delete** icon.

Configuring OAuth consumers

OAuth is a protocol that allows users to grant third-party applications access to their data without sharing their login credentials. It enables secure and controlled access to protected resources through APIs. You can create, approve, delete, and manage OAuth keys for consumers.

The OAuth protocol enables websites or applications (consumers) to access protected resources from a web service or service provider through an API without requiring users to disclose their service provider credentials to the consumers. For more information about OAuth, see OAuth documentation.

Registering consumer keys

The Consumers (Inbound) section on the **Administration** page shows the list of consumers of Rhapsody Systems Engineering. Registering the OAuth consumer's secret includes creating a new consumer key.

About this task

The server administrator can register consumer keys on Rhapsody Systems Engineering.

Procedure

1. To open the **Administration** page, click the **Administration** icon under the main menu of the Rhapsody Systems Engineering page.
2. To view the list of consumers for Rhapsody Systems Engineering, click **Connections** and then go to the **Consumers (Inbound)** section.

3. Click **Register consumer**. Complete the following sub steps:
   a) In the **Name** box, specify a consumer name.
   b) By default, the server generates a consumer key, but you can also use your own key.
   c) In the **Consumer Secret** and **Re-type Consumer Secret** fields, type the consumer secret.
   d) Select the **Trusted** check box.

      Note: Trusted consumers can share authorization with other trusted consumers and do not require user approval to access data.
   e) Click **Register consumer**. The new key displays in the Authorized Keys section of the page. Another consumer can use the key with the specified secret to communicate with the application.

Approving consumer keys

Consumers can request provisional OAuth keys that are not active until they are authorized by an administrator.

Procedure

1. On the **Administration** page, click **Connections**, and then go to the **Consumers (Inbound)** section.
2. View the provisional keys section for new requests.
3. To approve the consumer key, hover over the **Actions** column for the consumer name of the key that you want to approve and click the **Approve** icon. Complete the following substeps:
   a) Optional: To identify the server that uses the consumer key, update the **Consumer Name** field with the new server name.
   b) Optional: In the **Consumer Secret** and **Re-type Consumer Secret** fields, type the consumer secret.
   c) Optional: If the consumer shares authorization with other trusted consumers, select the **Trusted** check box.
   d) Click **Approve**. Once the key is approved, it is included in the Authorized Keys section of the page. The requesting consumer can now use the key to communicate with Rhapsody Systems Engineering.

Editing consumer keys

Editing an existing consumer key includes editing the values for the name, secret, or trusted values of the consumer key.

Procedure

1. On the Administration page, click **Connections**, and then go to the **Consumers (Inbound)** section.
2. In the **Authorized Keys** section, hover over the **Actions** column for the Consumer Name of the key that you want to edit and click the **Edit** icon.
3. In the **Edit Consumer** dialog box, edit the values for the name, consumer secret, or trusted values of the consumer key.
4. Click **Save**.

Deleting consumer keys

You can delete authorized consumer keys to discontinue communication between two applications.

Procedure

1. On the **Administration** page, click **Connections**, and then go to the **Consumers (Inbound)** section.
2. In the **Authorized Keys** or **Provisional Keys** section, review the consumer name of the key that you want to delete, and click the **Delete** icon.

3. In the **Delete consumer** dialog box, verify the values for the name and OAuth consumer key that you want to delete, and click **Delete**. After the key is deleted, the key is removed from the Authorized keys or Provisional keys section.

Analytics report

As a server administrator, the Analytics report page helps you to monitor the performance of various projects running for your organization. This page offers detailed insights related to projects and their configuration statistics.

Procedure

1. To open the **Analytics report** page, click the **System analytics** icon under the main menu of the Rhapsody Systems Engineering page.
2. To get a combined view of your projects, view the Cumulative statistics section. It includes the following information:

   Total number of projects
   Shows the total count of all projects.

   Total number of configurations
   Shows the combined number of configurations across all projects.

   Total number of configurations
   Shows the combined number of configurations across all projects.
3. Optional: To review the data about your projects, and make informed decisions based on the following metrics, click **Project statistics**. It includes the following information:

   Project name
   Shows the number of projects that are active.

   Users count
   Shows the number of practitioners assigned for a project.

   Tags count
   Shows the the total number of tags used across projects.

   Project usages count
   Shows how many times a project is used.

   Configuration count
   Shows the number of configurations available for a project.
4. Optional: To review the data related to configuration of your projects, click **Configuration statistics**. It includes the following information:

   Branch
   Shows the branching details of a project.

   Elements count
   Shows the number of elements created within a project.

Monitoring the licenses in use

As a server administrator, the License Monitor page allows you to review the licenses that are checked out.

Procedure

1. To open the **License Monitor** page, click the **Administration** icon from the main menu.
2. Click **License Monitor**. You can review details, such as which users have checked out licenses, along with the issue and expiration date and time for each license.

Extensions

The extension capabilities of Rhapsody Systems Engineering allow developers and system administrators to extend the functionality of the Rhapsody Systems Engineering platform through a JavaScript-based UI extension architecture. This capability provides a way to add custom features, UI components, and integrations with third-party applications seamlessly into the existing Rhapsody Systems Engineering interface.

How it works?

Extensions work by injecting custom JavaScript scripts into the Rhapsody Systems Engineering application. These scripts can perform the following actions:

Handling events
Listen to various client-side events, such as element loading, updates, and deletions and respond with custom actions.

Adding UI components
Introduces new action buttons and menu items within predefined UI placements, such as the global editor toolbar and browser overflow menu.

Displaying Indicators
Provides visual feedback (indicators) on model elements to show statuses like errors, successful syncs, or warnings.

Managing notifications
Triggers notifications within the Rhapsody Systems Engineering UI to inform users of important events or statuses.

Accessing models
Provides ability to view and modify model elements.

Extending the UI

You can extend the user interface (UI) of Rhapsody Systems Engineering by using the `embedExternalPage` property defined in the `config/extension.json config` file.

If the `embedExternalPage` property is defined in the `config/extension.json config` file, an additional button appears below the Browser view and Library view buttons in the project editor. This button uses the `data.iconURL` value to display the extension logo. Clicking the button opens the URL specified in `data.embedExternalPage` within an embedded iframe next to the browser view and the view area. The content displayed in the iframe is determined by the extension provider.

Extending Browser view menu

As a server administrator, you can customize the overflow menu of the items in the Browser view by adding additional menu actions, enhancing the functionality of items within the view.

In the Browser view panel, each item has an overflow menu which can be extended with additional menu actions defined in extension script. It is deployed by the extension provider and its location is set in the configuration file: `data.externalScript`.

Structure of the sample extension script

You can use the sample extension script to build your own extension script to understand how to interact with the Rhapsody Systems Engineering client application.

notificationKind
Enumeration for notification types.

indicators
A sample set of indicator icons with tooltips and descriptions, which is used to update the indicators displayed to the user. You can change the icons, tooltips, and descriptions as per your requirement.

getRandomIndicators
This is not required for the extension, however it helps you to generate a random set of indicators for testing purposes.

getIndicators
This is not required for the extension, however it helps to simulate fetching indicators for a set of element IDs in the MarkToSync action demonstration.

validatePayload
Ensures that the incoming data meets the required format for authorization and project IDs.

publish , markToSync , and unMarkToSync
Sample action functions that respond to user actions in the UI.

Event mechanism

The event mechanism extension uses a subscription model to listen for and handle events. The Rhapsody Systems Engineering host application provides a `subscribe` function to manage event listeners.

How events work?

Subscribe
When the extension wants to listen for a particular event, it calls the `subscribe` function, passing the event type and handler function. The `subscribe` function returns an `unsubscribe` function.

Event triggered
When a specified event occurs, the handler function is triggered with the event data.

Unsubscribe
If the extension no longer needs to listen for the event, it calls the `unsubscribe` function to stop receiving updates for that event.

Available events

`CLIENT_EVENTS`
These events are sent from the client UI and can be used by the extension to respond to client-side actions, such as loading model elements.

`CLIENT_EVENTS.ELEMENTS_LOADED`
This event is triggered when model elements are loaded in the client UI.

`CLIENT_EVENTS.ELEMENTS_ADDED`
This event is triggered when model elements are added in the client UI.

`CLIENT_EVENTS.ELEMENTS_UPDATED`
This event is triggered when model elements are updated in the client UI.

`CLIENT_EVENTS.ELEMENTS_DELETED`
This event is triggered when the model elements are deleted in the client UI.

`EXTENSION_EVENTS`
These events are sent from the extension to the Rhapsody Systems Engineering client application to trigger various actions or update the UI.

`EXTENSION_EVENTS.INIT`
Initializes the extension, defining custom actions and buttons in the UI.

`EXTENSION_EVENTS.UPDATE_ELEMENT_INDICATORS`
Updates indicators on individual elements in the model.

`EXTENSION_EVENTS.UPDATE_PROJECT_INDICATORS`
Updates indicators on the project level, such as project health status.

`EXTENSION_EVENTS.SHOW_NOTIFICATION`
Sends a notification to the client, following the NotificationData structure.

`EXTENSION_EVENTS.UPDATE_ACTIONS`
Updates a set of action buttons in the UI, such as show, hide, and enable or disable options.

LAYOUT_PLACEMENT
These events are predefined placements in the Rhapsody Systems Engineering client UI, where the extension can add custom buttons or actions.

LAYOUT_PLACEMENT.GLOBAL_EDITOR_TOOLBAR
The global toolbar is accessible from the main editor, where actions like "Publish" can be added.

LAYOUT_PLACEMENT.EXPLORER_OVERFLOW_MENU
Overflow menu in the explorer view, where actions like “Mark to Sync” and “Unmark to Sync” can be added.

Usages

This extension uses various events to define custom buttons, actions, and hide or disable actions.

Event initialization

The extension sends an INIT event to the client to define custom buttons and actions.

In the following example, the extension defines three actions in two different UI placements:

- Publish appears in the global editor toolbar.
- "Mark to sync" and "Unmark to sync" actions are available in the overflow menu of an element.

Consider the following sample:

```
window.RHAPSODYSE.sendEvent(EXTENSION_EVENTS.INIT, {
        actions: {
            [LAYOUT_PLACEMENT.GLOBAL_EDITOR_TOOLBAR]: [
                { id: 'publish', label: 'Publish', onAction: publish },
            ],
            [LAYOUT_PLACEMENT.EXPLORER_OVERFLOW_MENU]: [
                { id: 'markToSync', label: 'Mark to Sync', onAction: markToSync },
                {
                    id: 'unmarkToSync',
                    label: 'Unmark to Sync',
                    onAction: unMarkToSync,
                    actionInfo: {
                        enabled: false, // set the action to disabled by default in the UI
                        afterSeparator: true, // add a separator between the actions
                        isDanger: true, // add a red background color to the action button
                    }
                },
            ],
        },
    });
```

Event subscription

This event is triggered after model elements are loaded in the client to initialize indicators after elements are added. Consider the following example of subscribing to the ELEMENTS_ADDED event.

```
const unsubscribe = window.RHAPSODYSE.subscribe(CLIENT_EVENTS.ELEMENTS_ADDED, (payload) =>
{ ... });
```

Event payload structure

The payload of the client events as well as the first parameter of the action callbacks defined in the INIT event onAction property includes the following fields:

- config contains Authorization , ProjectId , and configuration ID properties
- data contains actual data specific to the event, such as array of element IDs.

Consider the following example of retrieving the configuration fields:

```
window.RHAPSODYSE.subscribe(CLIENT_EVENTS.ELEMENTS_LOADED, async ({ config, data }) => {
   const { Authorization, ProjectId, ConfigurationId } = config;
});
```

Action info usage

By using the `actionInfo` property, you can hide or disable an action button for a specific element. You can also add a separator between actions or add a red background color to the action button to indicate that the action is dangerous. Consider the following example of hiding and disabling an action button for a specific element:

```
window.RHAPSODYSE.sendEvent(EXTENSION_EVENTS.UPDATE_ACTIONS, [ {
    elementId: your-specific-element-id,
    actionId: 'markToSync',
    actionInfo: {
        enabled: false, // enable (true) / disable (false) action
        visible: true, // show (true) / hide (false) action
        afterSeparator: true, // add a separator between the actions
        isDanger: true, // add a red background color to the action button
    }
}]);
```

Notifications

NotificationData rules allow you to successfully trigger notifications in the Rhapsody Systems Engineering host application. This configuration defines the properties and format for notifications sent to the client.

Notification configuration options

`title`
Main message displayed in the notification.

`subtitle`
Additional information shown below the title.

`kind`
Specifies the notification type, controlling the icon and style. It should consist of an error, info, info-square, success, warning, or warning-alt.

`isActionable`
This is optional. When `true`, it displays an action button in the notification.

`actionButtonLabel`
This is optional. It is required if `isActionable` is `true`. Text displayed on the action button.

`timeout`
Duration in milliseconds before the notification dismisses automatically. It is set to 0 to disable auto-dismiss.

`onAction`
This is optional. This function is a callback function for action button click events.

`onClose`
This is optional. This function is a callback function for notification close events.

Sample notification usage

```
/**
 * Configuration object for notifications
 * @typedef {Object} NotificationData
 * @property {string} title - The main notification message
 * @property {string} subtitle - Additional details shown below the title
 * @property {('error'|'info'|'info-square'|'success'|'warning'|'warning-alt')} kind - The type
of notification, controls the icon and styling
 * @property {boolean} [isActionable] - Whether the notification shows an action button
 * @property {number} [timeout] - Time in milliseconds before the notification auto-dismisses
(0 for no auto-dismiss)
```

```
 * @property {string} [actionButtonLabel] - Label for the action button (required if
isActionable is true)
 * @property {Function} [onAction] - Callback function when action button is clicked
 */
const notificationData = {
    title: 'Published successfully',
    subtitle: `The content was published successfully for the project with id: $
{payload.ProjectId}`,
    kind: notificationKind.SUCCESS,
    isActionable: true,  // optional, required if actionButtonLabel is provided
    actionButtonLabel: 'Action after publish',  // optional, required if isActionable is true
    timeout: 10000,
    onAction: () => {
        console.log('Optionally, you can handle the clicking of the action button here');
    },
    onClose: () => {
        console.log('Optionally, you can handle the close event here');
    },
};

// then send the notification to the client
window.RHAPSODYSE.sendEvent(EXTENSION_EVENTS.SHOW_NOTIFICATION, notificationData);
```

Indicator data structure

Rhapsody Systems Engineering UI can handle up to four indicators per element. It displays the last indicator in the array, and if there are more, the Rhapsody Systems Engineering UI displays a number next to this indicator notifiying you that there are more indicators. If you click on this indicator, the Rhapsody Systems Engineering UI adds all the indicators in a tooltip. The following is a sample indicator data structure:

```
const indicators = [
        {
            indicatorURL: "http://localhost:3002/assets/diamond-exclamation.png",
            indicatorTooltip: "ExtensionError",
            indicatorDescription: "An error occurred during the publishing process. This could
be due to a variety of reasons such as network issues, server problems, or invalid data inputs.
Please check the system logs for more detailed information about the cause of the error, and
try to publish again after resolving any identified issues."
        },
        {
            indicatorURL: "http://localhost:3002/assets/check-circle.png",
            indicatorTooltip: "Extension - Published successfully",
            indicatorDescription: "The content was published successfully"
        },
        {
            indicatorURL: "http://localhost:3002/assets/rotate-square.png",
            indicatorTooltip: "Extension - Synced successfully",
            indicatorDescription: "Synchronization completed successfully"
        },
        {
            indicatorURL: "http://localhost:3002/assets/triangle-warning.png",
            indicatorTooltip: "Extension - Warning",
            indicatorDescription: "There is a warning that needs your attention"
        }
    ];
```

Deploying extension script

To deploy your extension script, the system administrator must configure the JavaScript file in the extension's config JSON file. If the script is hosted on a different domain, update the CSP headers of the host application.

Extend the CSP headers section in the operator CR file `config/default/settings.json` by adding your script domain to the `script-src` list. Otherwise, the browser blocks the loading of your external script.

Consider the following sample of extension configuration file:

```
{
  "id": "extension-123",
  "data": {
    "name": "Extension Display Name",
```

```
    "iconURL": "YOUR_ICON_URL",
    "provider": {
      "name": "Extension Provider Name",
      "website": "YOUR_WEBSITE_URL",
      "contact": "YOUR_CONTACT_EMAIL"
    },
    "embedExternalPage": "YOUR_EXTERNAL_EMBEDDABLE_PAGE_URL",
    "externalScript": {
      "url": "YOUR_EXTERNAL_SCRIPT_URL",
      "integrity": null,
      "crossorigin": "anonymous"
    },
    "version": "1.0.0"
  }
}
```

Managing

Learn how you can manage your projects by using configuration management capabilities, work on them by creating elements, relations, links, views and manage them, and create detailed visual models by using SysML V2 diagrams.

Projects

In Rhapsody Systems Engineering, a project serves as the fundamental organizational unit for managing and structuring system models. The Projects page includes the following views to manage and organize projects effectively: My projects, Active projects, and Archived projects.

My projects
: It displays the list of public projects that you can work on. By default, these projects are available in the main branch. When you make changes to the main or public branch of a project, a private branch is automatically created for you. Your changes in the private branch are visible only to you under the **Active projects** tab. The **Branch** column provides information about the branches associated with each project. If new commits are made to a branch, a **Get latest** icon ( ) appears next to the branch name. Hover over the icon, and click **Get latest** to update your local workspace with the latest changes from that branch.

Active projects
: It displays the projects that you are currently working on. Once you complete your changes and are ready to finalize them, you can commit them from the Project editor page and merge them with the appropriate branch.

Archived projects
: It displays the projects that you have archived.

Managing projects

Managing projects involves editing, searching, archiving, exporting, importing them, and many more.

About this task

After you create your projects, you can choose to perform the following actions on your projects.

Procedure

1. To open your project, in the project row, click the **Options** icon, and then click **Open** .
2. To view and manage the details of a project, such as managing users and project associations, in the project row, click the **Options** icon, and then click **View project details**.

3. To edit the basic properties of the project, such as name and description, in the project row, click the **Options** (⋮) icon, and then click **Edit project**. You cannot update the template.
4. To sort the list of your projects in an alphabetical order, hover over the column headers, and click the arrow icon.
5. To find projects that match specific keywords found in name and description, type your search text in the **Search** box.
6. To select the number of items to display on a page, in the **Items per page** list, select the number.

Related tasks

"Managing users" on page 77
Manage users by adding the users or changing their administrative privileges. The **All users** page helps you to add and manage the users for the projects of Rhapsody Systems Engineering. You require the roles of project administrator or server administrator to add and manage users.

Creating a public branch

To manage work, you can create public branches for your projects and add practitioners for them.

Procedure

1. Select the project for which you want to create a branch.
2. In the project row, under the **Options** menu, click **Create branch**.
3. In the **Branch source** list, select the branch that you want to use to create a new branch.
4. In the **Branch name** field, specify a name of the branch.
5. Click **Create**.
6. Optional: To remove a branch, select that branch, and under the **Options** menu, click **Discard all changes**.

Related concepts

"Roles" on page 77
A user role is defined by a set of tasks that a specific role is given permission to perform. In Rhapsody Systems Engineering, the three predefined roles are: server administrator, project administrator, and practitioner.

"Configuration management" on page 92
Configuration management capabilities in Rhapsody Systems Engineering allow you to work privately or publicly by creating branches for your projects. You can also create tags to mark specific points of progress within a branch. Using the configuration context feature, you can seamlessly switch between branches or tags in projects by using local configuration or global configuration. You can also create projects by using libraries from other projects, and manage their utilization effectively.

Creating tags

A tag is a read-only set of artifacts. Create tags to capture the state of a branch of a project at a specific point in time, and to define a starting point for new work. For example, create a tag for a branch of a project and use it as the base of a new branch for new work.

Before you begin

You require server administrator or project administrator privileges to create tags for your projects.

Procedure

1. Select the project for which you want to create a tag.
2. In the project row, under the **Options** menu, click **Create tag**.
3. In the **Tag source** list, select the branch that you want to use to create a tag.

4. Click **Create**. You can switch to other tagged data from the project editor by using the **Current configuration** button. After tags are created, the tagged data becomes read-only and you can switch to another branch to resume working on the project.

Related concepts

"Roles" on page 77
A user role is defined by a set of tasks that a specific role is given permission to perform. In Rhapsody Systems Engineering, the three predefined roles are: server administrator, project administrator, and practitioner.

Exporting projects

You can save your projects in your local file system. You can also export them while you work in the project editor.

Procedure

1. In the **Projects** page, select the project that you want to export.
2. Click **Options**, and then click **Export project**. The project is downloaded on your local system as a ZIP file.
3. If you want to export your projects from the project editor page, click the **Export project** ( ) icon in the header.

Renaming branches

You can rename the branches that you have access to and provide an appropriate name for them.

Procedure

1. In the **My projects** or **Active projects** page, select the project whose branch you want to rename.
2. In the branch row, click the **Options** ( ) icon, and then click **Rename branch**.
3. In the **Branch name** field, specify the new name of the branch.
4. Click **Submit**.

Configuration management

Configuration management capabilities in Rhapsody Systems Engineering allow you to work privately or publicly by creating branches for your projects. You can also create tags to mark specific points of progress within a branch. Using the configuration context feature, you can seamlessly switch between branches or tags in projects by using local configuration or global configuration. You can also create projects by using libraries from other projects, and manage their utilization effectively.

When you work directly on the main or public branch of a project, a private branch is created for you and your changes are visible under the Active projects tab. After your changes are complete and saved, you can commit them from the project editor by using the **Manage changes** ( ) option. For more information, see "Managing changes" on page 93.

Remember: To resume working on a branch of a project, you can either go to Active projects page to select the branch you were working on, or you can switch to the specific branch from the project editor using the Current configuration button.

To visually differentiate between the different branches or tags, refer the following color-coded and labeled pills:

- Tag : Tags of a project.
- Main branch : Main branches of a project.

- Public branch: Public branches of a project.
- Private branch: Private branches of a project.

Managing changes

After you complete and save your changes in a project, you can choose to commit changes, and merge them with your default branch. Conflicts might arise when incoming and outgoing changes exist in your project, and both changes affect the same project. Learn how to manage changes and resolve conflicts if they occur.

Procedure

1. Optional: If you are working on the main or a public branch and if changes are available on that branch, you can click **Refresh** to get the latest changes.

   If you made changes to the default branch, save the changes. This creates a private branch for you.

2. In the project toolbar, click the **Manage changes** ( ) icon.

   A menu appears with options. The options are visible and enabled based on the type of branch you are working on.

   Public branches

   - Update from source: Update with latest changes from source branch. The unsaved changes are lost.
   - Commit: Commit changes to another selected public branch that is not up-to-date with the current branch.
   - Discard all changes

   Private branches

   - Commit to source: Commit changes to the source branch, which could be main or a public branch. The unsaved changes are lost.
   - Update from source: Update with latest changes from source branch.
   - Discard all changes.

   Main

   The options to commit and discard all changes are disabled as you cannot perform these actions on the main branch.

3. To commit your changes to the main branch, complete the following substeps.

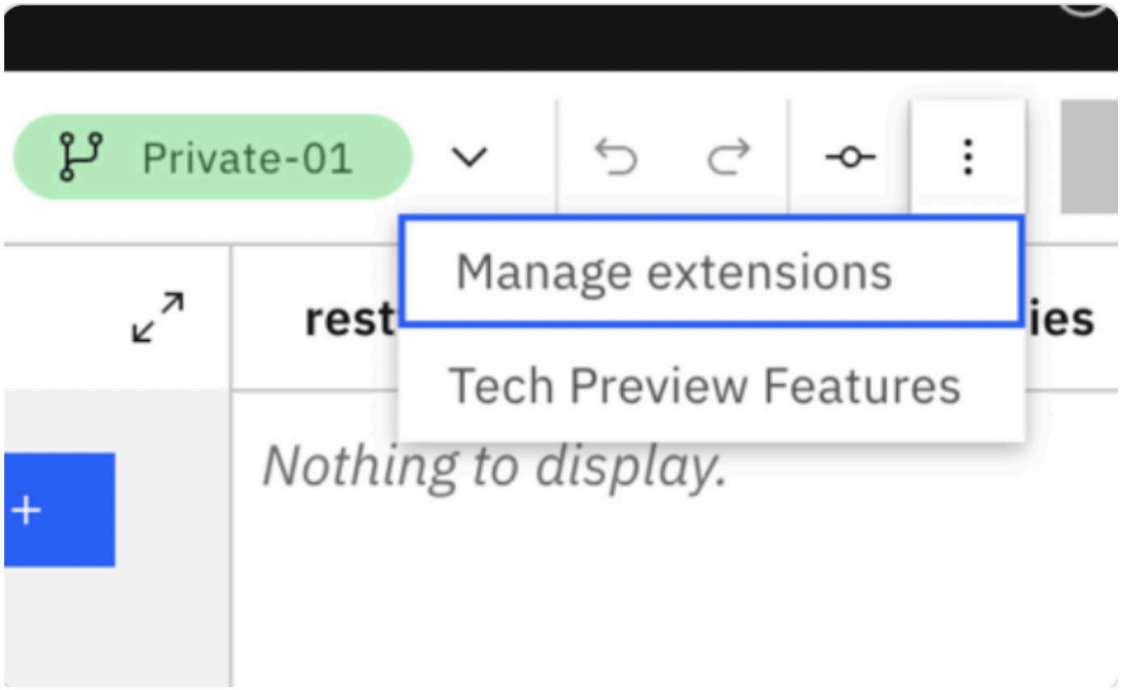
   a. Click **Commit to source** from the menu.

   b. In the Commit box, review your changes and check for conflicts.

   c. If there are conflicts, click **Resolve** to resolve all conflicted elements

      Note: This option only becomes available if you enable the Automatic diff merge under the Tech preview features.

i) Click **Tech Preview Features** from the overflow menu in the RhapsodySE application.



ii) Enable **Automatic diff merge**, and click **Apply**.

- If there are multiple elements that are shown as conflicted, you can select those elements, and then choose to click **Accept local changes** or **Keep server changes** from the toolbar.
- To resolve each conflicts separately, click on the row of conflicted element row. Choose whether to keep the changes from source branch or target branch.

d. To commit your changes, **Commit changes**.

4. If you are working on a private branch and want to sync it with the latest changes from its source branch (usually `main`), complete the following steps. The option to update from `main` appears only if `main` has new commits since your private branch was created.
   a. Click **Update from source**.
   b. Review your changes and check if there are any conflicts. If there are conflicts, follow the above procedure to resolve them.
   c. To update your changes with the latest changes from the source branch, click **Merge changes**.
5. To commit your changes to a selected public branch, complete the following substeps.
   a. Click **Commit**.
   b. In the **Commit** box, from the list of public branches, choose the public branch where you want to commit your changes.
   c. Review your changes and check if there are any conflicts.
   d. To commit your changes, click **Commit changes**.
6. To delete the changes made in the private or public branch, click **Discard all changes**. For public branches, this action deletes the branches derived from it.

Switching to other branches or tags

By using the configuration selector, you can switch between the available branches or tags of a project.

Procedure

1. In the project toolbar, click the **Current configuration** button.
2. To switch to the recently used configuration, select from one of the available branches or tags.

3. To select other configuration, click **Choose another**. In the **Select configuration** window, you get the following options:
   - If you want to switch to one of the available branches for the project, select **Branch**. Select the branch that you want to switch to.
   - If you want to switch to one of the available tags for the project, select **Tag**. Select the tag that you want to switch to.
4. Click **Apply**.

Switching to other branches or tags by using global configuration

Setting your current configuration context to a global configuration links to configurations in other Engineering Lifecycle Management project areas that are part of the same global configuration. This allows seamless integration and collaboration across interconnected project areas within the system.

About this task

To link across project areas in Engineering Lifecycle Management, team members must work in the context of a global configuration.

Procedure

1. In the project toolbar, click the **Current configuration** button.
2. To switch to the recently used configuration, select from one of the available configurations. The recently used configuration list can contain both global or local configuration.
3. To select other configuration, click **Choose another**.
4. In the **Select configuration context** window, you can either select global configuration or local configuration.

   Global configuration

   a. In the **Select configuration context** window, select the global configuration project area.
   b. Search for the global configuration that contains branch or tag of the given Rhapsody Systems Engineering project.
   c. Select the global configuration, and click **OK**.

   Note: If the selected global configuration does not contain any branch or tag of the given Rhapsody Systems Engineering project, then you get a warning with an option to switch to another configuration.

   Local configuration

   a. Select the local configuration by using the following options, and click **Apply**.
      - If you want to switch to one of the available branches for the project, select **Branch**. Select the branch that you want to switch to.
      - If you want to switch to one of the available tags for the project, select **Tag**. Select the tag that you want to switch to.

Related information

Using configurations with global configurations

Adding a branch or tag to the global configuration

You can add a Rhapsody Systems Engineering branch or tag to the global configuration and can manage different configurations or versions of the branch or tag depending on your requirements, improving flexibility in your development processes.

Procedure

1. Go to the Global Configurations project listing page by using the following URL. Replace the values within <> with actual values.

```
https://<elm_host_name>:<port_number>/gc/web
```

2. Open the Global Configuration project area.
3. Select the global stream where you want to add the Rhapsody Systems Engineering branch or tag to, and then click **Add configurations**.
4. To add Rhapsody Systems Engineering branch or tag, complete the following substeps:
   a) From the menu, select **Architecture Management (RSE)** application.
   b) From the **Project area** menu, select the project that you want to add.
   c) Under **Select context**, select **Branch** or **Tag**.
   d) From the **Select configuration** table, select the branch or tag.
   e) Click **Add**.

Project editor

The project editor provides a structured workspace, which is divided into three main sections: the browser panel for managing project elements, view area where diagrams and views are displayed and edited, and the Properties panel to modify the selected element details. You can efficiently organize and customize your projects by using intuitive controls within each section of the editor.

Browser view panel

In the **Browser view** panel, you can use the **Browser view** and **Library view** icons to switch between the browser view and the library view.

The Browser view panel displays a list of project elements organized into folders or packages in a hierarchical tree structure, where project is the root folder in the tree. You can organize large projects into package hierarchies and view them by nesting packages, components, and diagrams within other packages. This view enables you to add elements, update them, and perform different actions on the elements that are related to the tasks. The elements that are listed in the browser view have a specific icon representing its category, name, and certain actions to update the elements.

Note: To get a streamlined view of elements, and enable easy navigation of nested hierarchies, you can collapse all element groups back to the root level by clicking **Collapse All** from the overflow menu of the Browser view panel.

Switch to the library view by clicking the **Library view** icon. Library list displays the libraries that are currently included in the project. Libraries contain standard and reusable elements that you can use within your model. For example, IBMLibrary, SySMLLibrary, and other user-defined libraries contain the elements that you can reuse within your model. You can edit, update, or delete libraries or library elements by switching to the browser view. View the contents of the libraries by expanding the library name node. Use the library elements in your views or diagrams by dragging them on the view area.

View area

This is a middle section of the editor, where you can view and open the diagrams present in the project. Diagrams in Rhapsody Systems Engineering tool are called views. When you create a new project, the view area remains in an empty state, until you select an element that belongs to the category "view" in the browser view. You can work with multiple views and each view opens in a new tab. You can choose to expand the view area to span the entire screen and use the full screen option to perform tab or view level actions.

Properties panel

The Properties panel is located on the extreme side of the editor. It displays the properties of the selected model element. You can view and update the properties of the selected element. The panel is divided into the following sections:

Essential

Displays the basic properties of the selected element.

Other

Displays the additional properties for the selected element.

Owned relationships
Displays the relationships of a selected element with other model elements, where the element is the source or the owning entity of that relationship.

References
Displays other model elements that are referenced by the selected element.

Metadata
Displays the properties of metadata definition elements, which you can create and manage in the language extension view.

Links
Displays the external artifacts linked to the model element.

The following table displays the different actions that you can perform by using Browser view panel, view area, and **Properties** panel.

Actions	Editor area where the action can be performed	Description
Create elements	Browser view panel and view area	“Creating elements” on page 97
Manage elements	Browser view panel, view area, and Properties panel	“Managing elements” on page 98
Create relations	Browser view panel, view area, and Properties panel	“Creating relationships” on page 103
Create links	Browser view panel and Properties panel	“Creating links ” on page 138
Create views	Browser view panel and view area	See “Creating elements” on page 97
Manage views	View area	See “Creating visual models in views” on page 104

Elements

An element serves as the building block of a SysML V2 model. You can create new elements by using the browser view and the view area.

When you select an existing element in the browser view, the new element is always created as a child of the selected element. If no element is selected, the new element is added under the root project node.

Consider the following points when you select an element in the browser view:

- If the element belongs to another category other than views, then the **Properties** panel expands to display the properties of the selected element.
- If the active view contains that element on its canvas, it is highlighted within that view.

Creating elements

You can create elements from both browser view and view area.

Procedure

- In the Browser view panel, click the **Add element** (+) icon and complete the following substeps:
  a) From the **All** list, choose from one of the element categories, such as definitions, usages, relationships, views, or other. In the **Search** box, you can quickly search for an element type.

  Note: The Recently added section displays the recently created element types that you added. You can quickly create them from here.

b) In the **Select element** section, select the element type.

c) Based on the element type you selected in the previous step, specify the values in the mandatory fields that appear.

d) In the **Declared name** box, review the default declared name and specify the element name that you prefer.

e) Click **Add**.

- You can also add elements by using textual specifications.
  a) In the **Add element** box, enable **Add elements using textual specifications**.
  b) In the text area, type your SysML V2 code, and click **Add**.
- To create an element directly from the view area, complete the following steps:
  a) Select the node.
  b) Click the **Add element** icon (+), and then click one of the available elements that you want to create.

Adding elements by using drag-and-drop

You can add elements to your view by dragging them from the Browser view or diagram palette and dropping them into the diagram. You can also visualize the relations between different elements that reference other elements in a diagram by using drag-and-drop.

About this task

If you drag element from Browser view or diagram palette and bring it closer to a an element in the view then it also displays the preview of connection with that element.

Procedure

1. In the Browser view panel, select the elements that you want to add in the diagram.
2. Drag and drop the elements into the view.

Selecting and placing the elements

You can quickly add elements to your view by selecting and clicking them on the palette.

Procedure

- **Add an element once**: Choose the desired element in the palette and click it once, and then click in the diagram where you want to place the element.
- **Add an element multiple times:** Double-click the desired element in the palette. You can then click multiple times in the diagram to place multiple instances of that element.
- **Clear the selection:** Press Esc to exit placement mode and remove the current selection.

Managing elements

Managing elements in Rhapsody Systems Engineering include actions, such as renaming, reusing an existing usage structure for additional usages, sharing an existing package and its content as a library with other projects, validating, creating child elements, creating and copying links, moving elements, and more.

About this task

In the Browser view panel, hover over the element to find the following options:

Procedure

- To find the selected element in the views where it is used, and to open a specific diagram with the element highlighted, complete the following steps:

  a) Click the **Options** ( ) icon, and then click **Find in views**.

  b) From the **All** list, select the view, where you want to see the element highlighted.

  c) Optional: In the **Search** box, if you looking for a different view, type its name.

  d) Optional: You can specify a different element for search in views instead of the current one.
- To rename an element, complete the following substeps:

  a) Click the **Rename** icon next to the element that you want to rename.

  b) Type the new name of the element, and then click **Apply**.
- To create child elements directly under an element in the browser view, complete the following substep:

  – Click the **Options** ( ) icon, and then click **Create child element**. For more information, see “Creating elements” on page 97.
- To reuse an existing usage structure for additional usages, you can extract its structure as a new definition. Complete the following substeps:

  a) In the Browser view panel, locate and select the usage element whose attributes you want to reuse.

  b) Click **Options** ( ) icon, and then click **Create definition**. A new element is created. Once definition is extracted, the structure of usage is removed and a typing relationship is added between them.

  c) Depending on your requirements, you might want to update the new element such as name, attributes, and other properties. You can then use the newly created definition to type additional usages to fulfill the reuse.

  d) In the project toolbar, click **Save**.
- To share an existing package and its content as a library with other projects, complete the following substeps:

  a) In the Browser view panel, locate and select the package that you want to share as a library.

  b) Click **Options** ( ) icon, and then click **Change to library package**. The package is no longer visible in the browser view and appears in the list of libraries in your library view.

  To convert the library package to a package, in the library view, select the library package, click **Options** and then click **Change to package**.
- To import libraries from other projects, in the browser view panel, click ( ). Ensure that the project usage for the project that contains the libraries is created in All projects page. For more information, see “Creating project usages” on page 80.

  a) Click **Import library**.

  b) Select the libraries that you want to import, and click **Import**.
- To create links, see “Creating links ” on page 138.
- To share the link of an element with others, complete the following substeps:

  a) Locate the element, and click the **Options** ( ) icon.

  b) Click **Copy link**.

- To move an element under a different parent element, complete the following substeps.

  Note: You can also drag and drop an element directly under a new legal owner.

  a) Click the **Options** (⋮) icon, and then click **Move**.
  b) In the **Destination** field, select the appropriate element, and click **Move**.
- To move multiple elements under a different parent element at once, select the elements by using the `Shift` key on the keyboard, click the **More actions** (⋮) icon that appear on the blue toolbar, and click **Move**.
- To create a copy of a model element, so that you can quickly create a new element with the same content as in an existing element, click the **Options** (⋮) icon, and then click **Duplicate element**.
- To delete an element, click the **Options** (⋮) icon, and then click **Delete**. All references to that element are accordingly updated.
- To delete multiple elements at once, select the elements to be deleted, click **Delete** icon that appears on the blue toolbar. All references to that element are accordingly updated.
- To work with your view directly in the view area, select the element, and use the options in **Additional operations** menu.
  - Click **Ports**, select **Hide ports** or **Show ports** to hide or show the ports.
  - Click **Parameters** > **Show all parameters** or **Parameters** > **Hide all parameters** to show or hide the ports.
  - Click **Change type** to modify the element type.
  - Click **Create definition** to create a definition, which moves all features from usage to the new definition, creates a typing relationship between them. The usage inherits all features like attributes and ports from the new definition.

  Note: When an action or part has no parameters, you get the option to create only a definition.

Related tasks

“Creating project usages” on page 80
You can reuse the libraries from your existing projects by creating project usages.

Validating elements

Validate your elements to make sure that no inconsistencies are there, they adhere to the SysML V2 language specification, and there is no missing information.

About this task

You can run validation checks from both Browser view panel and by using the Validate project option in the global toolbar.

Procedure

1. To run the validation check from the Browser view panel, complete the following substeps:
   a) Select the elements that you want to validate, and click **Options** icon that appears next to the element.
   b) To validate the selected elements only, click **Validate element**.
   c) To validate the elements along with its children, click **Validate all**.
2. To run the validation check from the toolbar of project editor, complete the following substeps:
   a) Click the **Validate project** icon.

b) To validate the entire project, click **Validate project**.

3. Review the results. If there are errors, follow the instructions for resolution.
4. After making changes, re-run the validation checks to ensure that all issues are resolved.

Editing references of an element

To meet your design requirements, you might want to update the properties of an element that references to another element.

About this task

While you work in the browser view, you can quickly update the properties of an element that references to another element from the Properties panel. There are two types of references:

- Element reference identifies an element in the model hierarchy.
- Feature chain identifies a feature in the context of another feature or type.

Depending on the type of the reference property, the respective feature picker is invoked. Some reference properties can select multiple values.

Procedure

1. In the **Browser view** panel, select the element whose references you want to update.
2. In the **Properties** panel, locate the reference property.
   For example: for an element of type subsetting, navigate to the **Subsetted Feature** field.
3. Click , and then select the element or feature that you want to reference.
4. To create blank references, click the **Clear** icon.

Filtering the elements details

The Filter option in the Browser panel allows you to customize your view by showing or hiding model details based on your requirements.

Procedure

1. In the Browser view panel, click the **Filter** icon.
2. Select **Hide model details** to get specific elements and remove additional information.

Relationships

The SysML V2 standard introduces several categories of relationship elements that serve distinct purposes in system modeling. These categories are essential for organizing, defining interactions, mapping across domains, and enhancing documentation within complex system models.

Referential elements

In the SysML v2 standard, some usage elements can be visually represented with a graphical notation in the form of an edge, such as a satisfy requirement which connects a feature to a requirement. These elements can be referred to as the referential elements as in most cases they refer other elements.

The following is the list of referential elements.

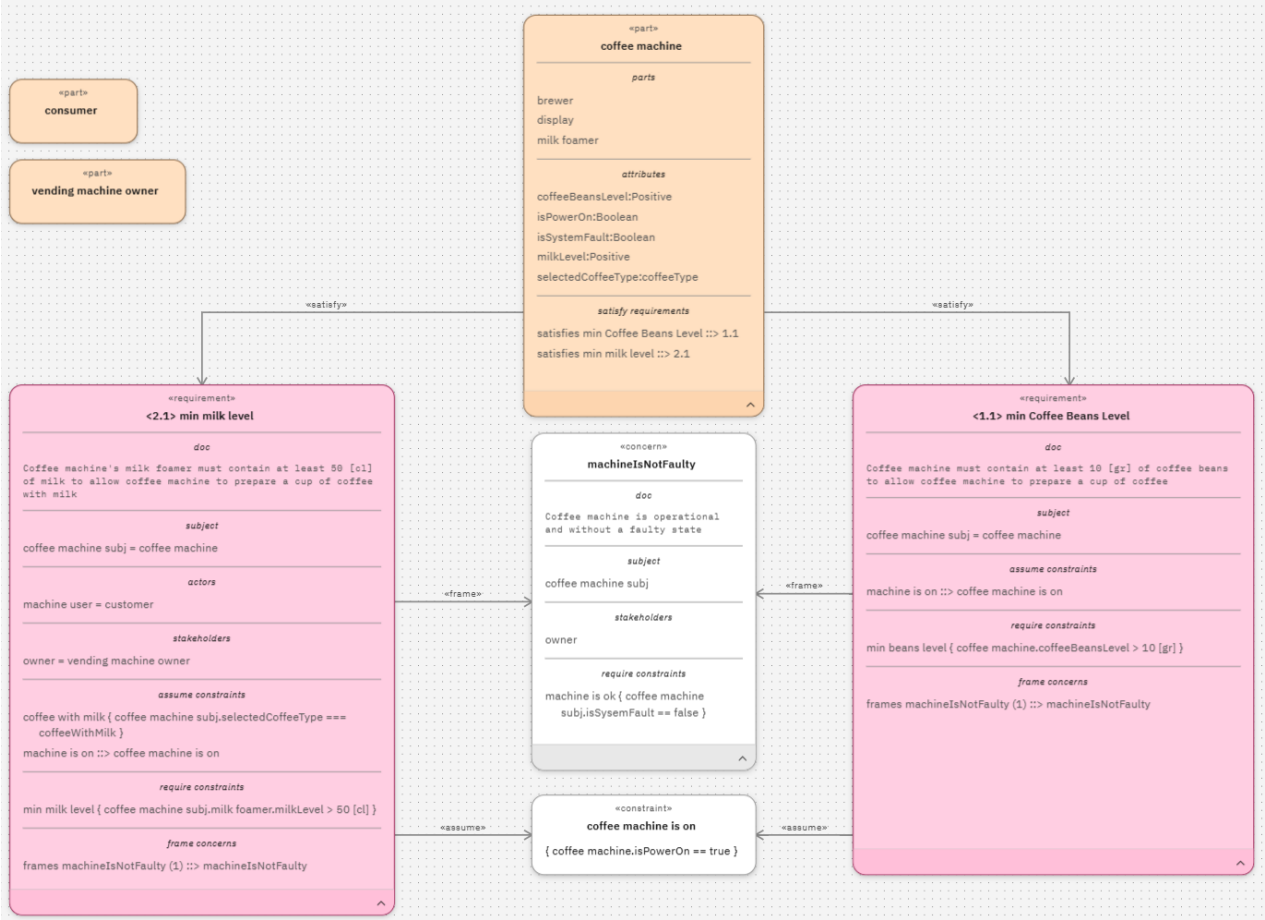
- Include use case connects one use case to another use case.
- Perform action connects a part or action to an action.
- Exhibit state connects a part or state to a state.
- Satisfy requirement connects a feature to a requirement.
- Assert constraint connects a feature to a constraint.
- Assume constraint connects a requirement to a constraint, which is relevant in context of the requirement.

- Require constraint connects a requirement to a constraint, which is relevant in context of the requirement.
- Frame concern connects a requirement to a concern, which is relevant in context of the requirement.

For example, when a part owns a perform action, it indicates that the part is performing the perform action referencing or when a part owns a satisfy requirement, it indicates that the part is fulfilling the satisfy requirement referencing. When you drop the source or target of a referential element to a view, Rhapsody Systems Engineering checks if the element of the other end of the referential element exists on the view and adds an edge representing the referential element.

Consider the following sample of a requirement view, where referential elements are used as edges. The following are the referential elements used in the following view:

- "Satisfy" between "coffee machine" (part) and "min milk level" (Requirement)
- "Satisfy" between "coffee machine" (part) and "min coffee beans level" (Requirement)
- "Assume" between "min milk level" (Requirement) and "coffee machine is on" (Constraint)
- "Assume" between "min coffee beans level" (Requirement) and "coffee machine is on" (Constraint)
- "Assume" between "min milk level" (Requirement) and "machineIsNotFaulty" (Concern)
- "Assume" between "min coffee beans level" (Requirement) and "machineIsNotFaulty" (Concern)



Creating relationships

Relations are the basic means by which you can reference other objects. You can either drag a relationship element from the Browser view panel to a view or can create relations manually from the Browser panel, Properties panel, or while you work on the view in the view area.

Procedure

- Drag a relationship element directly from the model navigator to a view. This creates an edge if both ends of the relationship are already represented as nodes in the view. For example, if your model includes elements with relationships like subsetting, you can easily place them on a view.
- To create relations manually from one of the following areas of project editor, complete the following procedure:

  - To create relations from the Browser panel, hover over an element, and click the **Options** (⋮) icon next to the element. Click **Create child elements**.
    1. In the Add element box, from the **All** list, select **Relationships**. You can also search for them in the Search field.
    2. Select the appropriate relationship that you want to add.
    3. Specify the information in the required fields that appear for different relationship types.
    4. Click **Add**.
  - To create relations from the **Properties** panel, locate **Owned relationships** and expand it.
    1. In the Properties panel, click the **Add Relationship** (+) icon.
    2. Complete the above procedure to add the relationship.
  - To create relations while you work on a view, select the appropriate option from the diagram palette, and connect two nodes.
    1. Expand the palette.
    2. In the Relationships section, select the relation that you want to establish.
    3. To create a relationship, click the boundary of the first node to set the source, and then click the boundary of the second node to set the target.

Viewing relationships

Relationships are the elements that relates two or more elements. You can view them in the browser view or Properties panel.

- In the Browser view panel, relationships are visible under the parent element.
- In the **Properties** panel, you can view relations under the Owned relationships section of its target and source element properties. Clicking on the hyperlinked names of elements displays the details of each relationship. This section displays the relationships in a tabular format and supports searching within the listed relationships. By default, it displays the categories of relation types for an element. You can expand them to view the links to the connected related elements.

  Under the References section, you can view the incoming relationships for a selected element. These relationships show which elements depend on the selected element. For a target element, the direct relationship where the selected element is the target is shown. For connectors, you can view the relationships of both source and target elements. By clicking on the hyperlinked names of elements you can view the details of each relationship.

Creating visual models in views

All diagrams in Rhapsody Systems Engineering are called views. You can add different kinds of elements as nodes on the view and create relationships between those elements by adding specific arrows between the nodes.

About this task

If you created your projects by using simple template or Harmony, the All views tab is displayed by default in the view area. Once you start working on views, both Recently opened and All views tabs appear.

- The Recently opened tab shows the last six views you have worked on. To locate a view displayed in the Recently opened tab in the Browser view panel, click its link. To view its location, hover over the path at the bottom of the tile.
- The All views tab displays the views that you can use in your projects.

Procedure

1. To visually differentiate between the different views, refer the following icons:
   - Interconnection view: 
   - Use case view: 
   - General view: 
   - Action flow view: 
   - State transition view: 
   - Requirements view: 
   - Table view: 
2. Add views from the view area.
   a) Click **Add view** in the view area, or click **Add view** (+) icon in the view toolbar.
   b) In the **Select view** area, select the view that you want to add.
   c) In the **Declared name** field, specify a name.
   d) Click **Add**.
3. Create new elements in the view.
   a) Drag the icon of the element that you want to add from the palette to anywhere in the view and click **Save**. For example, drag part definition, part usage, connectors, or nodes that are specific to views from the diagram palette to anywhere in the view. The elements are represented as nodes in the views, and relationships or connections as lines or edges.
   b) To create edges, select the tool from palette, click start and end connection points on nodes, and click **Save**.
   For more information, see “Adding elements by using drag-and-drop” on page 98.
4. Add elements to your view quickly while you work on the view.
   a) Select a node. The **Add element** (+) icons appear around the node.
   b) Click the icon in the direction, where you want to create new elements. A menu including a list of allowed actions appears.

c) From the menu, choose and click the action that you want to perform. This action creates a new element in the model and a new node on the view with an edge connecting to the origin node.

d) Click **Save**.

5. Align your elements with each other by using the snapping lines in the view area.
6. Associate an actor or a subject to a use case by using proximity connect. Select the appropriate element from the palette and drop it on the use case node, or in the proximity of the use case node and click **Save**.
7. Use the context-specific elements like usage, definitions, ports, and relationships that can be added to a view by using the diagram toolbar.
8. Copy or cut nodes and edges from one diagram and paste in another diagram.
   a) In the source diagram, select the nodes or edges that you want to copy.
   b) Click the **Copy** or **Cut** icon from the right toolbar.
   c) In the target diagram, click **Paste** from the right toolbar.
9. Refer to the visual feedback while you work on a diagram in the view area to quickly determine whether a drawing action is valid or invalid before completing it.
10. View and edit multiple parts of any model simultaneously on different monitors or areas of the screen by clicking the pop out window icon ( ) on the tab. You can navigate, resize, and manage multiple windows.
11. Preview your changes.
    a) To hide your preview, click the **Hide preview** icon.
    b) To fit the preview to your screen, click the **Fit to view** icon.

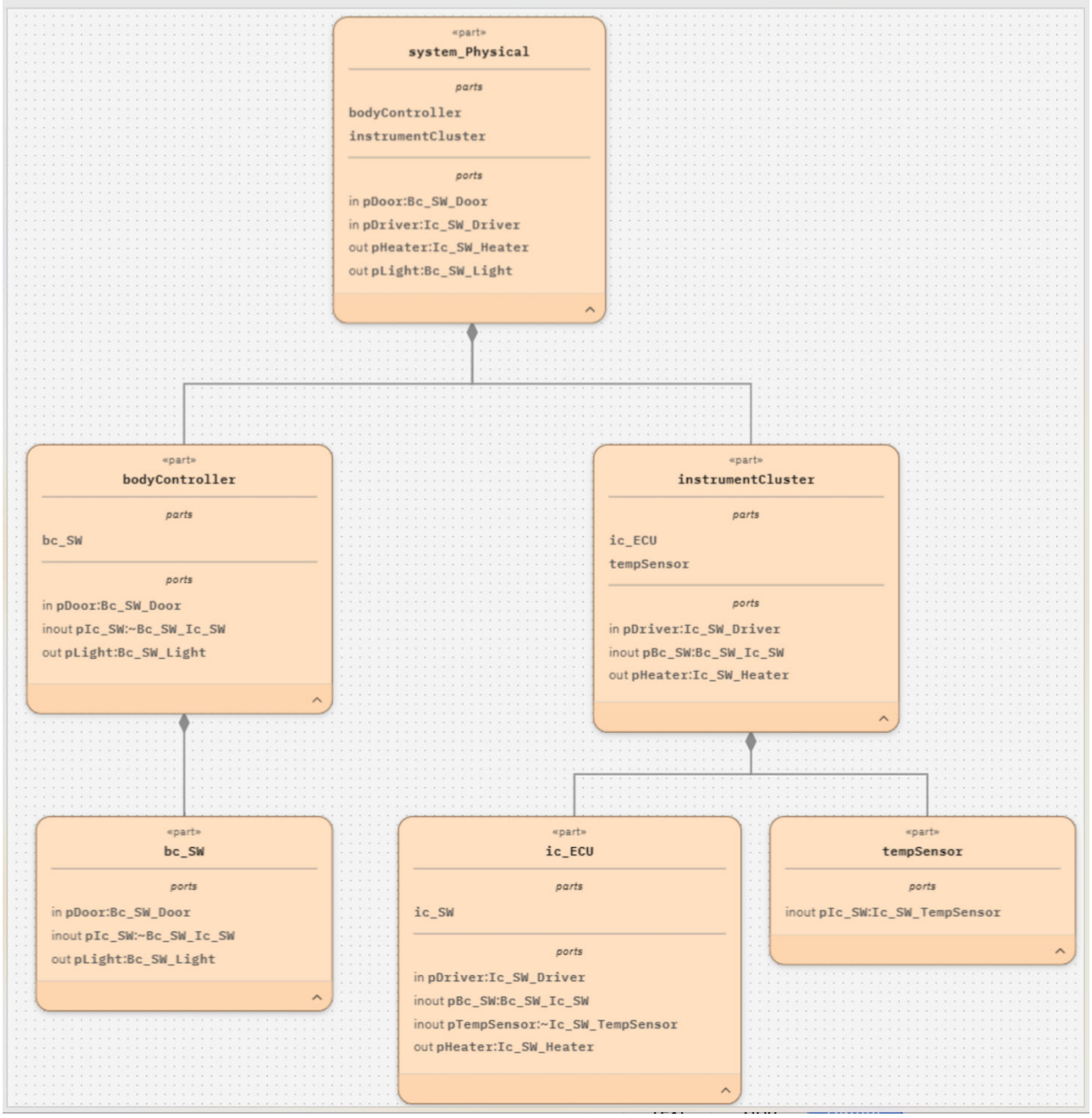
Types of views

Rhapsody Systems Engineering offers you a number of views to capture different perspectives of your system.

General view

In a general view, create diagrams by using any model element. Currently, the palette contains a subset of all elements.

Consider the following sample of a general view.


«part»
system_Physical
parts
bodyController
instrumentCluster
ports
in pDoor:Bc_SW_Door
in pDriver:Ic_SW_Driver
out pHeater:Ic_SW_Heater
out pLight:Bc_SW_Light
«part»
bodyController
parts
bc_SW
ports
in pDoor:Bc_SW_Door
inout pIc_SW:~Bc_SW_Ic_SW
out pLight:Bc_SW_Light
«part»
instrumentCluster
parts
ic_ECU
tempSensor
ports
in pDriver:Ic_SW_Driver
inout pBc_SW:Bc_SW_Ic_SW
out pHeater:Ic_SW_Heater
«part»
bc_SW
ports
in pDoor:Bc_SW_Door
inout pIc_SW:~Bc_SW_Ic_SW
out pLight:Bc_SW_Light
«part»
ic_ECU
parts
ic_SW
ports
in pDriver:Ic_SW_Driver
inout pBc_SW:Bc_SW_Ic_SW
inout pTempSensor:~Ic_SW_TempSensor
out pHeater:Ic_SW_Heater
«part»
tempSensor
ports
inout pIc_SW:Ic_SW_TempSensor

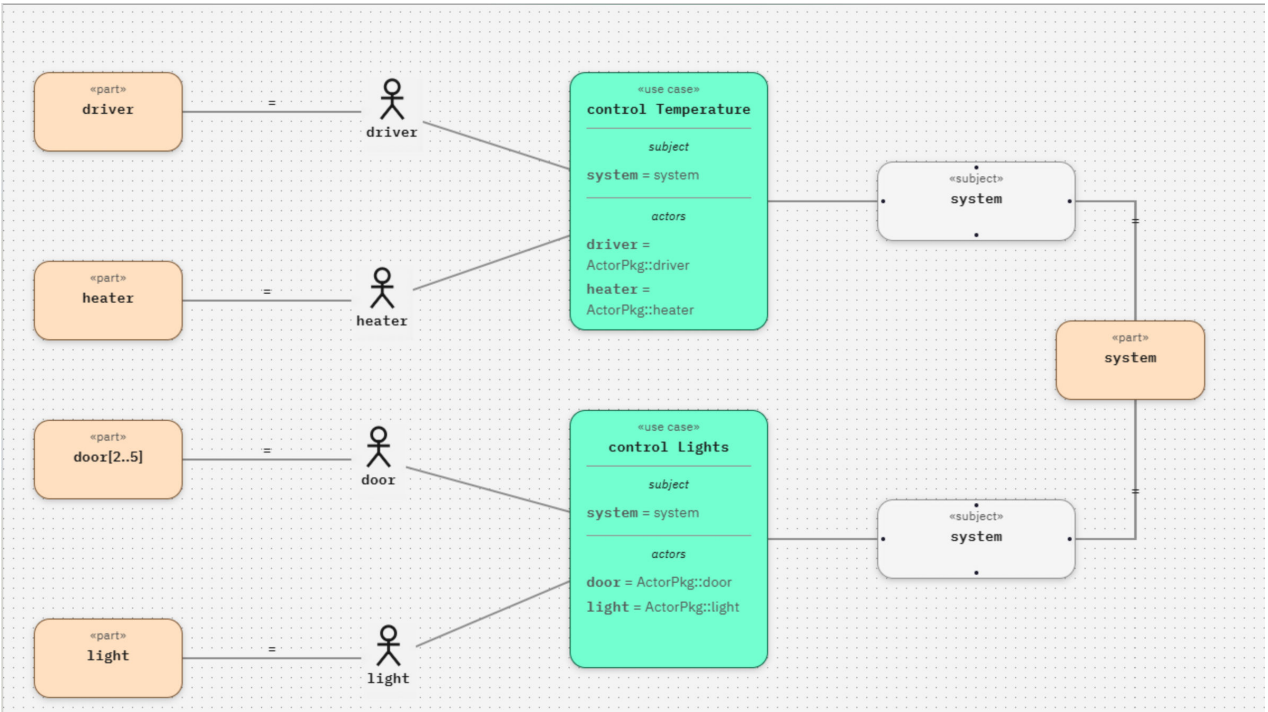
Use case view

In an use case view, create diagrams that show communication between use cases and external users or actors in the context of a system of interest, which is known as a subject.

Defining relationships between the subject and the actors is an effective informal way to define the system scope. Use case view diagrams identify the interactions between the system and its actors. The use cases and actors in use case diagrams describe what the system does and how the actors use it, but not how the system operates internally. You can model a complex system with a single use case diagram, or create many use case diagrams to model the components of the system. You would typically develop use case diagrams in the early phases of a project and refer to them throughout the development process.

When you drag and drop a use case from the Browser view panel on a view, any actor or subject parameters owned by that use case are automatically added to the view.

Consider the following sample of a use case view.



Related concepts

“Referential elements” on page 101

In the SysML v2 standard, some usage elements can be visually represented with a graphical notation in the form of an edge, such as a satisfy requirement which connects a feature to a requirement. These elements can be referred to as the referential elements as in most cases they refer other elements.

Simplified properties
In a use case view, you can bind an actor, stakeholder, or a subject by using "=" inside the textual compartment, where they are visible under their owner.

Consider the following format: `<actor> = <bound-element>`. To create a binding between the parameter and the element, type the name of a resolvable element, after a "=" sign. To clear the binding remove the `= <bound element>` from the textual notation. The following simplified properties are added to CaseDefinition, CaseUsage, and any derived element types from them:

Bound subject
Views or updates the element bound to a subject.

Bound actor
Adds or removes actors and set their bound element.

Bound stakeholder
Adds or removes stakeholders and set their bound element. CaseDefinition, CaseUsage, RequirementDefinition, RequirementUsage, and the derived element types

Action flow view
In an action flow view, model the sequence of actions that must occur in a system or application, or describe what happens in a business process workflow. When you create action flow diagrams, you can add and modify nodes and edges, and organize them by using partitions.

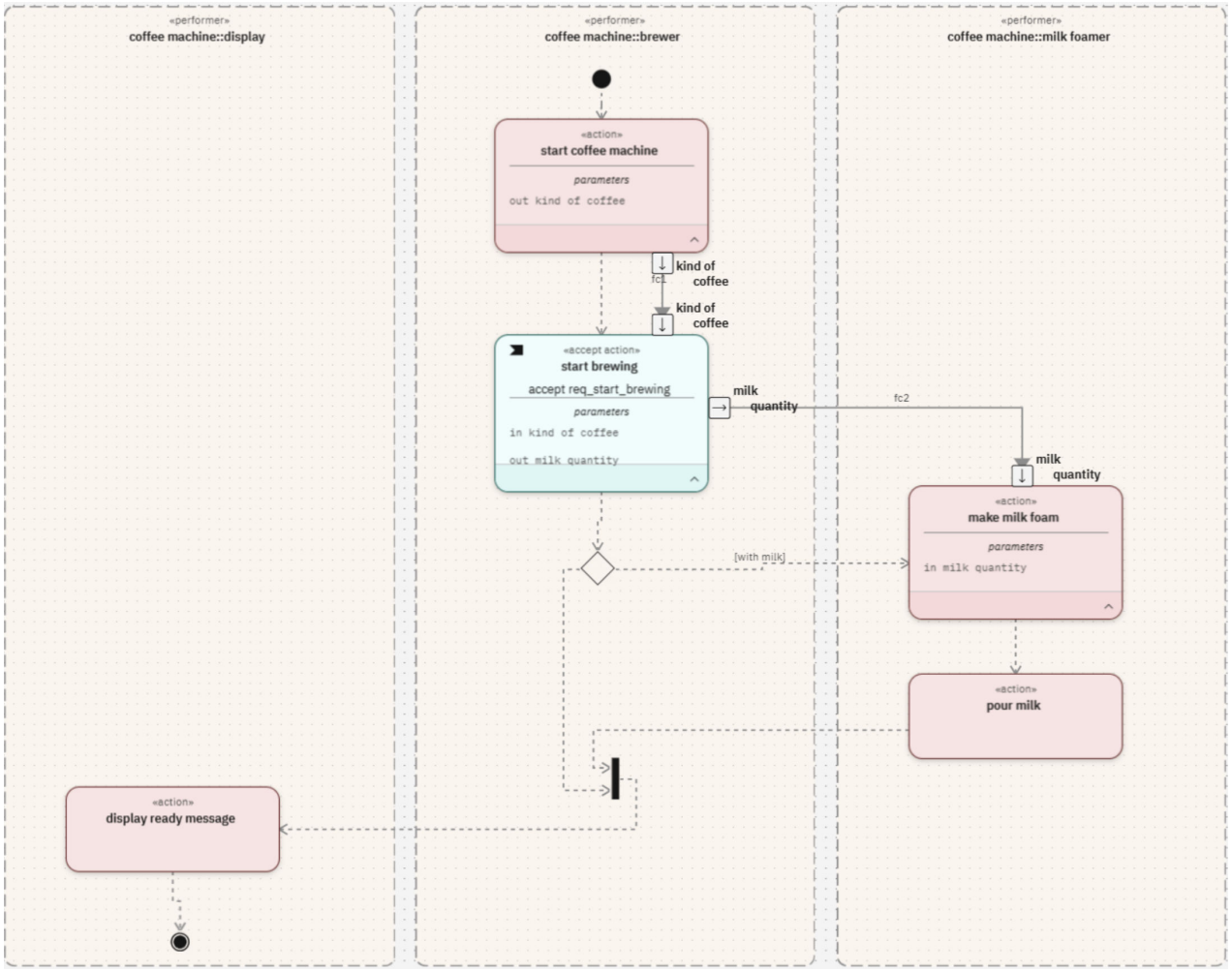
Actions can have parameters, which are directional features owned by the action. To convert a feature owned by an action into a parameter, set its **Direction** property under the Essential section in the Property panel to a value other than **Unassigned**. You can then drag and drop the parameter from the Browser view panel to a view, over a node of its owning action to show it as a parameter node.

You can also add parameters by using the (+) button available for an action node. The contextual menu allows you to choose from one of the three options: in, out, or inout parameter. Dragging a parameter from the palette creates a parameter with a default direction: in. When you add a parameter to an action, the parameters of the owning action, as well as the parameters of any subsetting actions, are also updated automatically.

Dividing action flow diagrams by using swimlanes

Swimlanes divide and structure the flow of actions performed by different elements in the model. They divide an action flow diagram into distinct sections, with each swimlane representing a specific part of the system model. Each swimlane indicates which performer is responsible for which part of the action flow. Placing an action in a swimlane designated for a performer creates a `PerformAction` for that action under that performer. Conversely, moving an action out of a swimlane removes the `PerformAction` for that performer.

Consider the following example of an action flow diagram that uses swimlanes. The coffee machine has three parts: `display`, `brewer`, and `milk foamer`.


«performer»
coffee machine::display
«performer»
coffee machine::brewer
«performer»
coffee machine::milk foamer
«action»
start coffee machine
parameters
out kind of coffee
kind of coffee
fc1
kind of coffee
«accept action»
start brewing
accept req_start_brewing
parameters
in kind of coffee
out milk quantity
milk quantity
fc2
milk quantity
«action»
make milk foam
parameters
in milk quantity
[with milk]
«action»
pour milk
«action»
display ready message

Display swimlane shows messages or instructions.
It can have actions like displaying brewing progress or error messages, indicating when the machine is ready, and more.

Brewer swimlane handles the brewing process.
It can have actions, such as heat water, extract coffee grounds, brew the coffee, and more.

Milk foamer swimlane manages the milk frothing process.
It can have actions, such as preparing drinks like lattes or cappuccinos.

Related concepts

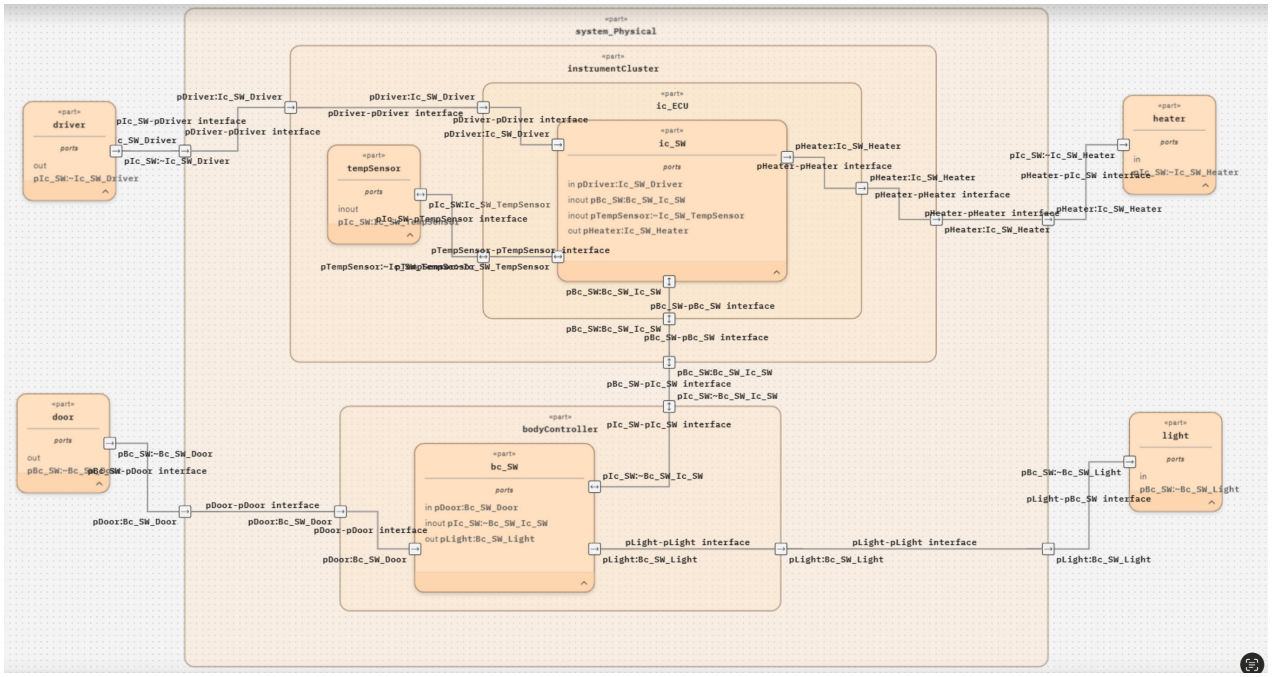
“Referential elements” on page 101
In the SysML v2 standard, some usage elements can be visually represented with a graphical notation in the form of an edge, such as a satisfy requirement which connects a feature to a requirement. These elements can be referred to as the referential elements as in most cases they refer other elements.

Interconnection view

In a interconnection view, create diagrams that present features as nodes, nested features as nested nodes, and connections between features as edges between nested nodes.

Nested nodes might represent boundary features, such as ports. It describes the internal structure and connection of a specific part.

Consider the following example of an interconnection view.


«part»
system_Physical
«part»
instrumentCluster
«part»
ic_ECU
«part»
ic_SW
ports
in pDriver:Ic_SW_Driver
inout pBc_SW:Bc_SW_Ic_SW
inout pTempSensor:~Ic_SW_TempSensor
out pHeater:Ic_SW_Heater
«part»
driver
ports
out
pIc_SW:~Ic_SW_Driver
«part»
tempSensor
ports
inout
pIc_SW:Ic_SW_TempSensor
«part»
heater
ports
in
pIc_SW:~Ic_SW_Heater
«part»
door
ports
out
pBc_SW:~Bc_SW_Door
«part»
bodyController
«part»
bc_SW
ports
in pDoor:Bc_SW_Door
inout pIc_SW:~Bc_SW_Ic_SW
out pLight:Bc_SW_Light
«part»
light
ports
in
pBc_SW:~Bc_SW_Light
pDriver:Ic_SW_Driver
pDriver-pDriver interface
pIc_SW-pDriver interface
pIc_SW:~Ic_SW_Driver
pIc_SW:Ic_SW_TempSensor
pTempSensor-pTempSensor interface
pHeater:Ic_SW_Heater
pHeater-pHeater interface
pIc_SW:~Ic_SW_Heater
pHeater-pIc_SW interface
pBc_SW:Bc_SW_Ic_SW
pBc_SW-pBc_SW interface
pBc_SW-pIc_SW interface
pIc_SW:~Bc_SW_Ic_SW
pBc_SW:~Bc_SW_Door
pDoor-pDoor interface
pDoor:Bc_SW_Door
pLight:Bc_SW_Light
pLight-pLight interface
pLight-pBc_SW interface
pBc_SW:~Bc_SW_Light

State transition view

A state transition diagram is a visual representation of the states and transitions of a system. It helps to model the dynamic behavior of a system by showing the different states it can be in and the triggers that cause it to transition between those states.

Each state can have a label or name, and the transitions might be labeled with the events or actions that cause the transition to occur. When you select a transition, the Properties panel displays details about its source and target states.

Consider the following example of a state transition view. Transition has an optional label that may have: trigger, guard, and effect.

State

allDoorsclosed, checkingDoors, and doorOpen are states.

Transition

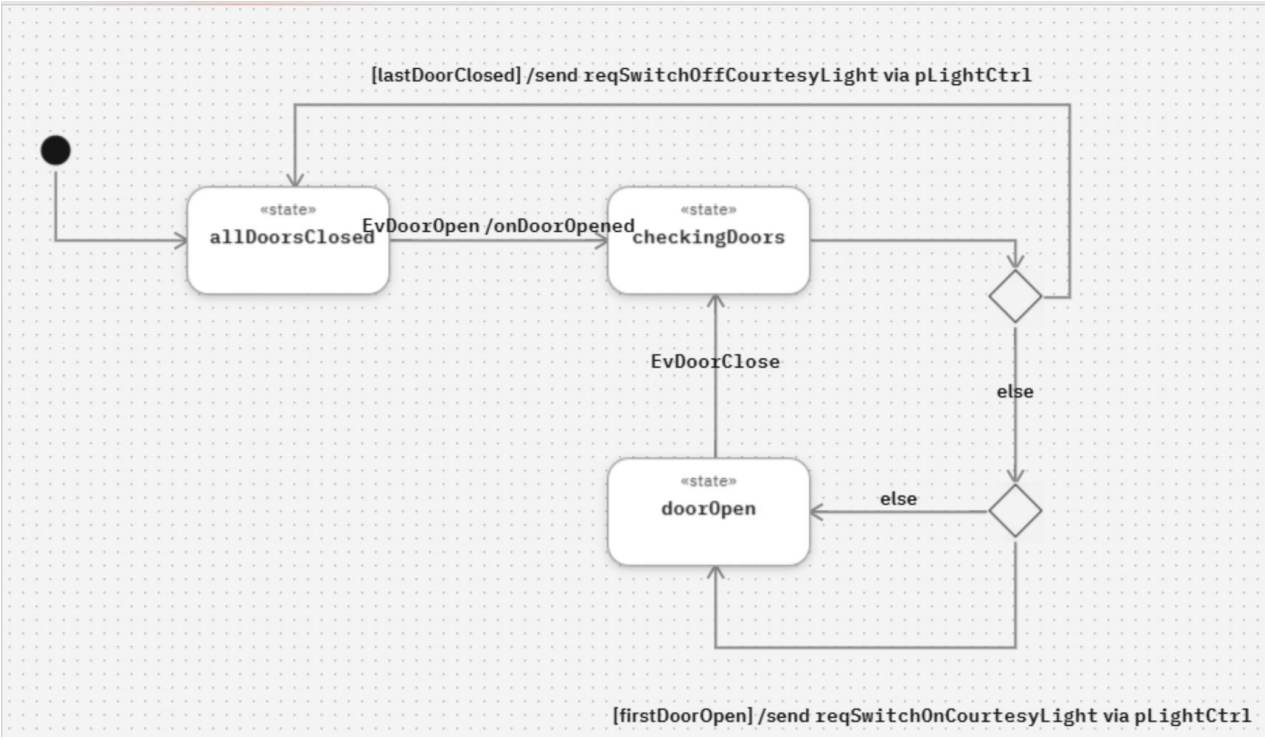
EvDoorOpen is the `<trigger>` part of the transition.

Format: `<trigger>[<guard>]/<effect>`

guard is a boolean expression, for example: [`vehicle.speed = max_speed`], or [`firstDoorOpen`]

effect is an action (perform, accept, send, assign), for example: `/send reqSwitchOnCourtesyLight by pLightCtrl`

Each state has a declaration text as its label.


[lastDoorClosed] /send reqSwitchOffCourtesyLight via pLightCtrl
«state»
allDoorsClosed
EvDoorOpen /onDoorOpened
«state»
checkingDoors
EvDoorClose
else
«state»
doorOpen
else
[firstDoorOpen] /send reqSwitchOnCourtesyLight via pLightCtrl

About states
Each state in a state transition diagram can have a label or name, and the transitions might be labeled with the events or actions that cause the transition to occur.

A state can also have one of the state actions: entry, do, or exit. You can add them by using the (+) button on the state node. After they are added, they appear in state actions compartment, and can be updated by editing the text by following a standard textual notation to provide the action associated with each sub-action. The actions can include: perform, send, accept, or assign actions. The inherited sub-actions in the state actions compartment are marked with a ^ symbol. You can redefine them by editing the text directly in the compartment.

A state transition view can also use flow control nodes, for example, decision, merge, fork, or join.

An exhibit state is an element that references another state. The semantics changes depending on where it is used.

Owned by part
The part exhibits the behavior associated with the referenced state.

Owned by another state
The referenced state extends the structure of the referencing state.

Parallel states are the states whose immediate sub-states are considered as concurrent. Transitions cannot cross between these concurrent states under the same parallel state.

Related concepts
“Referential elements” on page 101
In the SysML v2 standard, some usage elements can be visually represented with a graphical notation in the form of an edge, such as a satisfy requirement which connects a feature to a requirement. These elements can be referred to as the referential elements as in most cases they refer other elements.

Requirements view
The requirements view manages and traces the requirements, creates, and updates requirements with simplified workflows to set their properties, structure and relations with other elements.

Consider the following sample of a requirements view. This diagram includes two requirements: `min milk level` and `min coffee bean level`. These requirements are satisfied by the part `coffee machine`, represented by the `satisfy` edges.

The `min milk level` requirement has a requirement ID: <2.1>. It includes the following parameters:

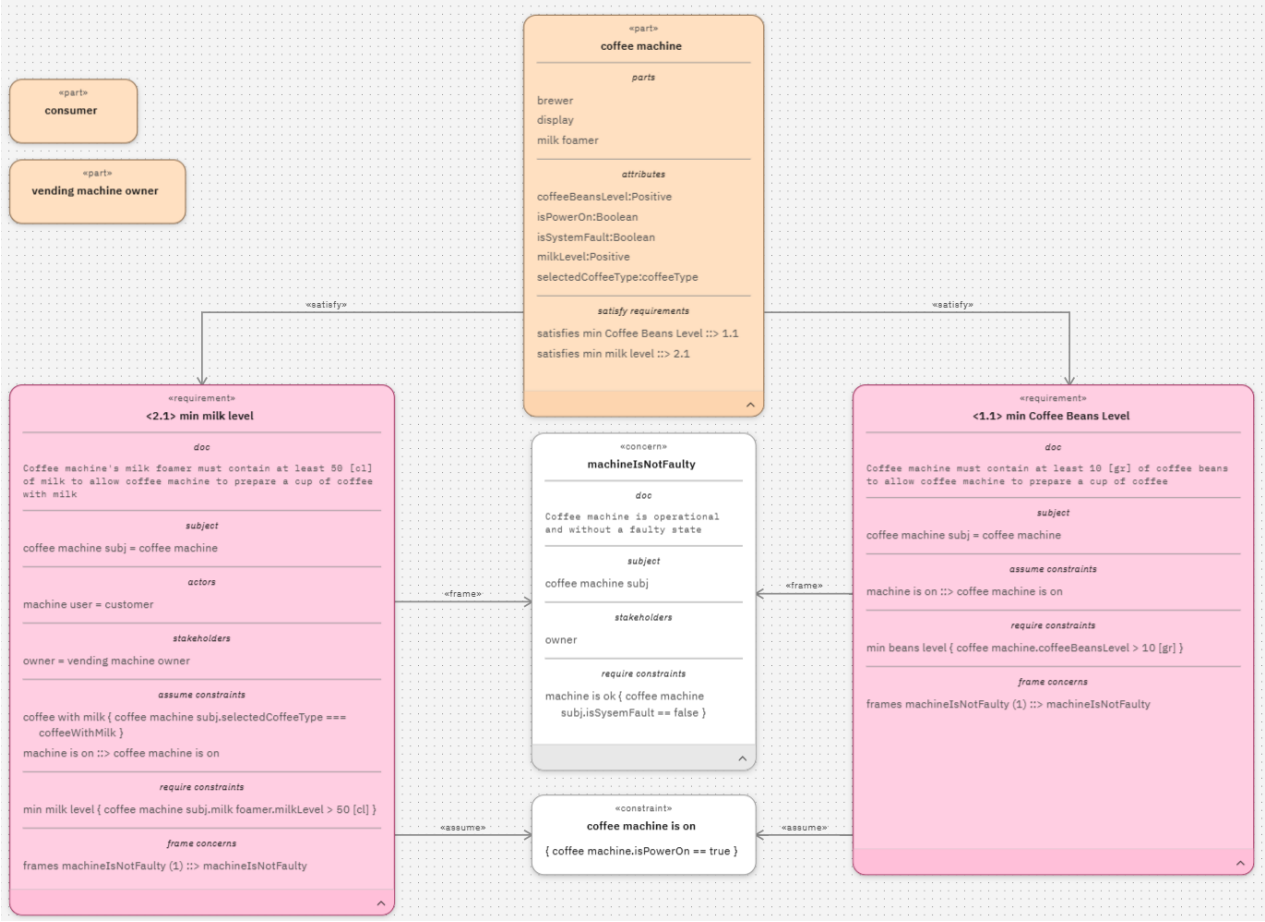
Subject
`coffee machine subj` is bound to the part `coffee machine`.

Actor
`machine user` is bound to `consumer`.

Stakeholder
`owner` is bound to `vending machine owner`.

The following are the additional elements associated with this requirement:

- An assume constraint `coffee with milk` defined to be { `coffee machine subj.selectedCoffeeType === coffeeWithMilk` }
- An assume constraint `machine is on`, which references the constraint `coffee machine is on`, visualized by the «`assume`» edge.
- A frame concern `frames machineIsNotFaulty`, which references the concern `machineIsNotFaulty`, visualized by the «`frame`» edge.



The property "Description" has the label "Requirement doc" for the following elements:

- RequirementUsage
- RequirementDefinition
- SatisfyRequirementUsage
- ConcernDefinition
- ConcernUsage

Consider the following format: `<actor> = <bound-element>`. To create a binding between the parameter and the element, type the name of a resolvable element, after a "=" sign. To clear the binding remove the `= <bound element>` from the textual notation.

Related concepts

“Referential elements” on page 101
In the SysML v2 standard, some usage elements can be visually represented with a graphical notation in the form of an edge, such as a satisfy requirement which connects a feature to a requirement. These elements can be referred to as the referential elements as in most cases they refer other elements.

Simplified properties
In a requirements view, you can bind an actor, stakeholder, or a subject by using "=" inside the textual compartment, where they are visible under their owner.

The following simplified properties are available in the Properties panel and are added to RequirementDefinition, RequirementUsage, and any derived element types from them:

Bound subject
Views or updates the element bound to a subject.

Bound actor
Adds or removes actors and set their bound element.

Bound stakeholder

Adds or removes stakeholders and set their bound element. CaseDefinition, CaseUsage, RequirementDefinition, RequirementUsage, and the derived element types

Table view

Table views provides capabilities for managing and modeling complex systems. A table view displays model elements and their attributes in a tabular format.

The following are the benefits of table view:

Traceability

Makes it easier to track model elements and their attributes, ensuring traceability in system design.

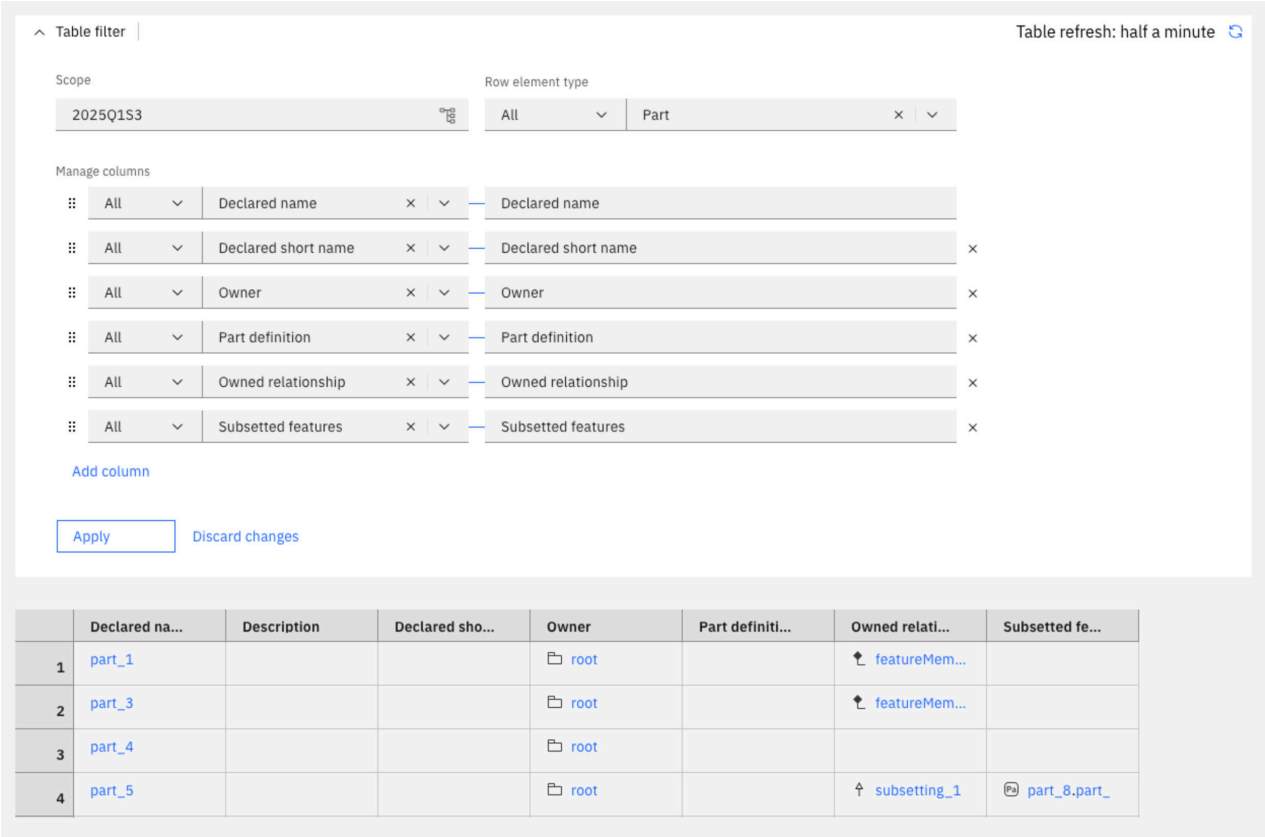
Requirement coverage

Helps to track relationships between elements to ensure required traceability in the model.

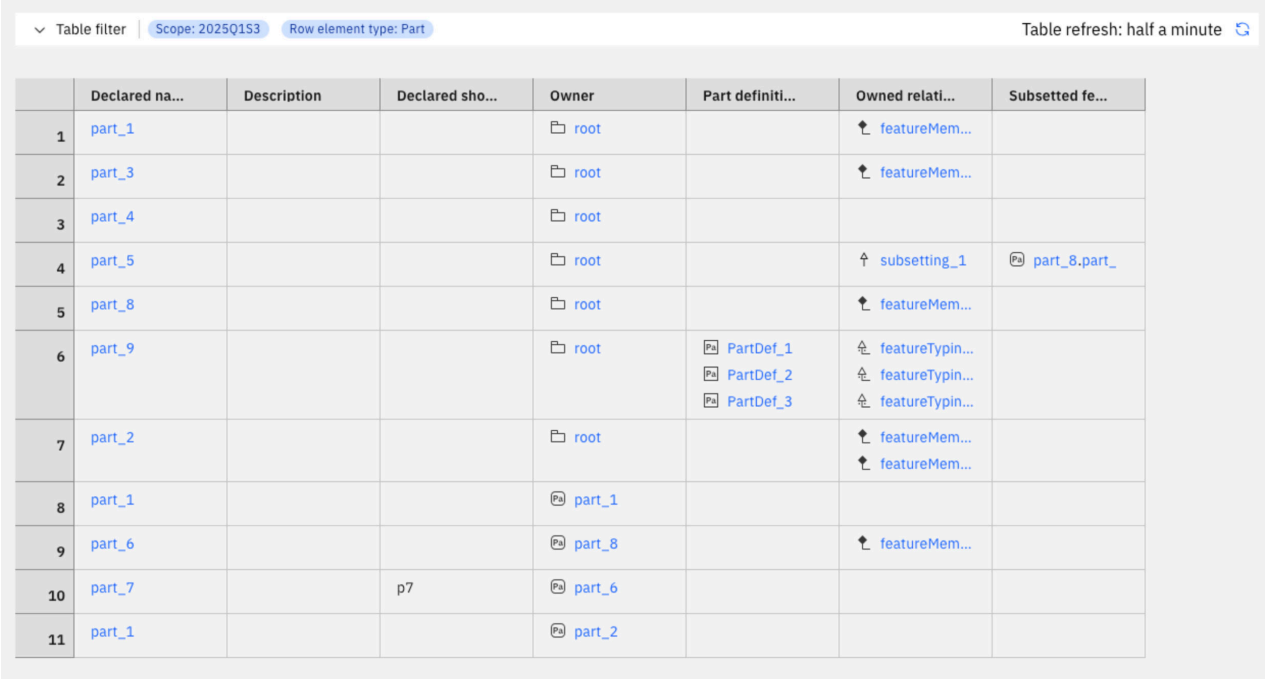
Bulk editing

Supports efficient data entry and editing, allowing you to quickly update multiple attributes for various elements, ensuring consistency and accuracy across your system model.

Consider the following sample data added under the Table filter section.



The following is the table view generated based on the above data given under the Table filter.



	Declared na...	Description	Declared sho...	Owner	Part definiti...	Owned relati...	Subsetted fe...
1	part_1			root		featureMem...	
2	part_3			root		featureMem...	
3	part_4			root			
4	part_5			root		subsetting_1	part_8.part_
5	part_8			root		featureMem...	
6	part_9			root	PartDef_1 PartDef_2 PartDef_3	featureTypin... featureTypin... featureTypin...	
7	part_2			root		featureMem... featureMem...	
8	part_1			part_1			
9	part_6			part_8		featureMem...	
10	part_7		p7	part_6			
11	part_1			part_2			

Managing data in table views
Rhapsody Systems Engineering allows you to add, manage, and customize table views.

Procedure

1. Add a table view.
   a) In the **Add element** box, select the table view.
   b) In the **Scope** list, select an appropriate scope for the table view.
   c) In the **Row element type** list, select the element type that you want to show in your view.
   d) Click **Add**. The table view is created.
2. Add elements directly from your table view.
   a) In the table view, click the **Add element** (+) icon.

      Note: The Add element button gets disabled if the element type that you chose in the row filter is not supported for creation.

   b) To add elements, follow one of the procedures mentioned in Steps 1 and 2.
3. Manage the data of your table view. Under the **Table filter**, you get various options to customize your view. Once you apply your changes, click **Apply** to get the updated table view.
   - From the **Scope** list, you can choose a new scope for your view.
   - From the **Row element type** list, you can choose a new element type.
   - Under **Manage columns**, you can manage the columns that you want to appear in your table view. You can perform the following actions:
     – Click **Add column**, and choose the appropriate column type. You can also edit the name of your column as per your requirement.
     – Remove a column by using the **Remove** icon located near each column.
     – To manage the order of your columns, use the drag handles located near each column.
   - To display the hierarchy of elements in the table view, enable **Hierarchy mode**.
4. Use the following options to add or edit the existing information displayed in the table, and click **Save** to apply the changes.
   - Edit the text fields.

a. Hover over the text fields, such as the **Declared Name** or **Description** columns.
b. Click the **Edit** icon that appears, and enter the new text.
c. To save the changes, click the **Apply** icon.

- Add or delete the information.
  a. Hover over the field, such as **Port Definition**.
  b. Click the **Add** (+) icon to get the list of available options, and select the appropriate option to add new information.
  c. To remove the existing information, click the **Delete** icon.
- Edit the information in the fields that have a menu of available options.
  a. Hover over the field, such as **Direction** column.
  b. Click ↓ to select the appropriate option from the list of options.
- Edit the information in the fields that have a checkbox by selecting the checkbox, such as `Is Abstract`.
- Adjust the width of the columns.
  a. Move your cursor to the column divider.
  b. Click and drag the handle to increase or decrease the width of the column.

5. To assign element references in the table view, drag elements from the Browser panel and drop them into the reference field.
6. To display the hierarchy of elements in the table view, enable **Hierarchy mode**.
7. Sort the columns alphabetically or numerically depending on their content. Sorting is only available in non-hierarchical mode.
   a) Hover over the column header, and click the up or down arrow.

Managing links in table views
In table views, you can view OSLC links based on their link types. You can filter, add and remove links from the table view.

Procedure

1. View the existing OSLC links by adding the related OSLC link type columns in the table view.
   a) Expand the **Table filter**.
   b) Under **Manage columns**, click **Add column**.
   c) Select **Links** as column type.
   d) From the **Select property** list, select an appropriate OSLC link type
   e) Optional: You can edit the name of your column as per your requirement.
   f) Click **Apply** to get the updated table view.
2. Add OSLC links from the table view.
   a) Hover over the appropriate OSLC link type column.
   b) Click the **Add** icon that appears.
   c) In the **Create links** box, specify the appropriate link information and click **OK**.
3. Remove the existing OSLC links from the table view.
   a) Hover over the appropriate OSLC link type column.
   b) In the **Confirm remove** box, click **Remove**.

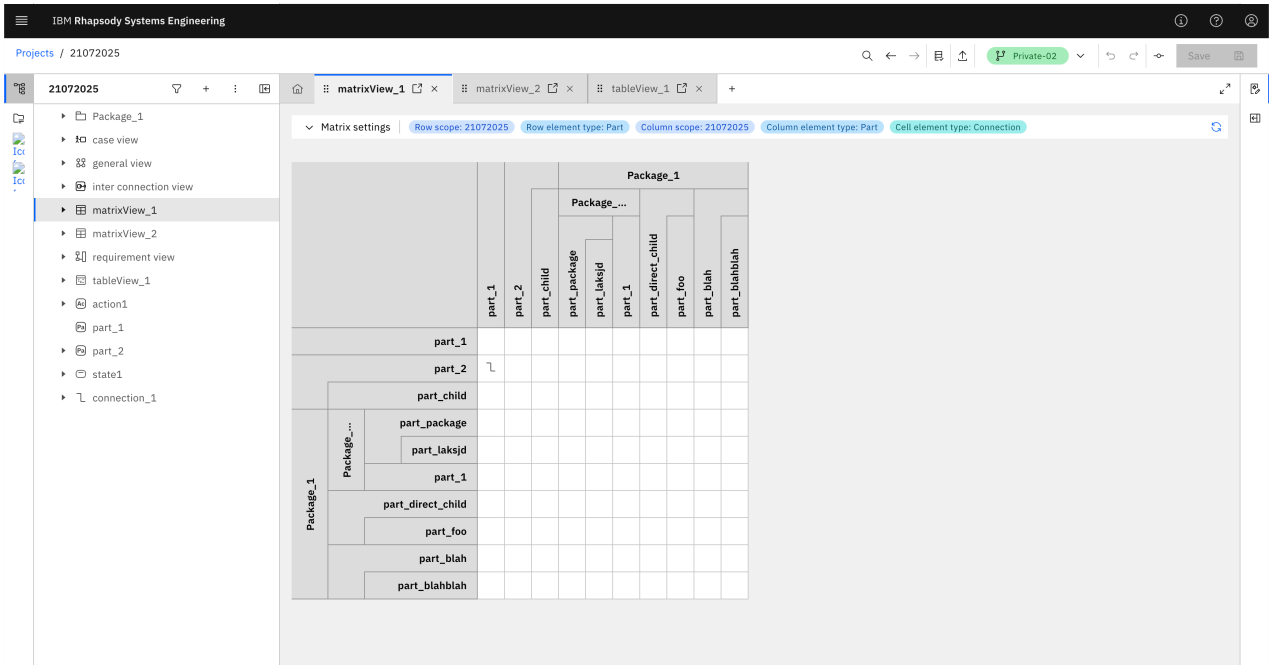
Related concepts
“Managing links” on page 138
Managing links in Rhapsody Systems Engineering involves several key actions: creating new links between model elements and artifacts of friend applications, viewing detailed information about existing links, and

removing links when necessary. This functionality allows for comprehensive management of relationships between Rhapsody Systems Engineering model elements and artifacts in friend application, ensuring effective traceability and alignment throughout the development process.

Matrix view

Use a matrix view to display the relationships between different model elements.

Consider the following sample matrix view.



Managing data in matrix views

Rhapsody Systems Engineering allows you to add, manage, and customize matrix views.

Procedure

1. Create a project, add elements, and add relationships between them.
2. Add a matrix view.
   a) In the **Add element** box, select the matrix view.
   b) In the **Row scope** and **Column scope** lists, select an appropriate scope.
   c) In the **Row element type** and **Column element type** lists, select the element types that you want to show in your view.
   d) In the **Cell element type** list, select the relationship type that you want to show between the rows and columns.
   e) Click **Add**. The matrix view is created.
3. Manage the data of your matrix view. Under the **Matrix settings**, you get various options to customize your view. Once you apply your changes, click **Apply** to get the updated matrix view.
4. To display the hierarchy of elements in the matrix view, enable **Hierarchy mode**.
5. Create relationships directly within the matrix view.
   a) Hover over the cell where you want to create a relation and click the (+) icon.
   b) Review the populated **Source feature** and **Target feature** fields, and click **Add**. The relationship appears in the matrix view.
   c) Click the relation icon to view more details about the created relationship. To create another relationship of the same type, click the **Add** (+) icon and to remove a relation click the **Delete** icon.
6. Click **Refresh** if your model is updated with new elements or relations.

Using the right toolbar options

In the right toolbar, you can pan, select, copy, cut, paste, or remove elements from the view, and use the auto-layout icon to automatically update the layout of selected nodes and edges in top-down or left-right formats.

Procedure

- You can use the following options in the right toolbar.
  - To pan around within the view, click the **Move mode** ( ) icon.
  - To click and drag to select multiple nodes and edges on the view, click the **Select mode** ( ) icon.
  - To copy or cut a single or multiple nodes or edges, select the nodes or edges, and click the **Copy** ( ) or the **Cut** ( ) icon.
  - To paste the copied nodes or edges, click the **Paste** ( ) I con.
  - To remove the selected nodes and edges from the view, click the **Remove from view** ( ) icon.
  - To focus on an element in a view, click **Auto Fit to View on Selection** ( ) icon to enable it, and then select the element from the Browser panel. The diagram automatically pans and centers on the selected element.

Arranging the diagram elements automatically

To meet your visualization requirements, you can apply different layout options for arranging the elements of a diagram.

Procedure

1. In the right toolbar of your view area, click on the **Auto-layout** ( ) icon.
2. You can choose from one of the available layouts and edit them.
   - Click **Configure layout settings**. This allows you to modify your own layouts by using the advanced settings, such as element spacing, direction, alignment, and more.
3. In the left panel of the **Configure layout settings** window, you can reorder the layout algorithms by using the drag handles available at the end of each layout. This order gets reflected in the auto-layout menu options.
4. Once you have finalized your changes, you can preview the layout in the **Preview** area.
5. Select **Apply**.

Exporting diagrams

Download your diagrams in the SVG format by using **Download as SVG** option available in the right toolbar of the view area.

Procedure

1. In the view area, locate and select the diagram that you want to export.
2. In the right toolbar, click the **Download as SVG** ( ) icon. The SVG file is generated and downloaded into your computer.

Adding visual comments

You can add visual comments to your views to enrich the views of the system model and contribute to more effective system development and understanding,

About this task

Procedure

From the diagram palette, use **Comment**, **Annotation**, and **Documentation** tools to add visual comments to your views. The following workflows are supported:

- A node with an `Annotation` relation with another node displays **about** label for the node in the diagram.
- When you drag **comment** from model navigator o an element's node in a view, the following results can take place:

Element	Results
Textual compartment	– A new node is created for the dropped comment. – A new **Annotation** element is created to connect the dropped comment and the target element.
Graphical compartment	– A new node is created for the dropped comment. – The comment is moved to be owned by the target element.

Editing edges

You can improve the appearance of a diagram by moving around the edges, and adjusting them around the existing components of a diagram.

Procedure

1. Select the edge that you want to edit. The control points appear.
2. Click on the control points. A draggable point appears.
3. To delete the draggable point, right-click on it.
4. To finalize your changes, click **Save**.

Editing labels

You can edit the existing labels of edges or specify labels for the edges, where no labels are set.

Procedure

1. To edit an existing label, double-click the label, and type a new name.
2. If there is no label set to an edge, select that edge. A placeholder label appears. Double-click the label to specify a new name.
3. To finalize your changes, click **Save**.
4. To improve diagram readability, you can reposition the edge labels by dragging them to the desired location. The new position of the label is relative to its original position, and if the edge moves, its label also follows accordingly.

5. To identify which label belongs to a specific edge, select the edge, and the text of the associated label is also underlined for easy identification.

Customizing the color of nodes or edges

You can customize the color of your nodes or edges to meet the requirements of your organization.

Procedure

1. Select the node or edge that you want to customize in the view area.
2. In the contextual menu, for a node click **Customize node** (), and for an edge click **Customize edge**.

   Node
   Choose a color for background. The colors for footer and border are set automatically based on the color chosen for the backgroound. However, you can also override the colors for footer or border.

   Edge
   Choose a color for stroke.
3. To apply the changes, click **Apply**.

Using diagram palette

There are two types of palette views: collapsed and expanded. You can switch between them, search for the diagram elements, and add them directly to the view area. You can also quickly add elements by selecting them on the diagram palette.

About this task

Collapsed view
By default, the palette is in the collapsed view. In this view, the palette items are grouped into categories. When you hover over a group, you can see the related items for that group. When you click on a specific palette item, for example, action definition, the selected item replaces the group icon.

Expanded view
In the expanded view, all items in the group are shown individually, providing a more detailed view of the available palette items.

Procedure

1. Expand the palette by clicking (>), and type the element name in the **Search palette** box.
2. Click the element type to add it your view area.
3. To quickly add elements by selecting them on the diagram palette, complete the following substeps:
   a) In the diagram palette, select the element that you want to add in the diagram.
   b) Click in the view area. The element is added to the view.
   c) To disable the selection, either press ESC on the keyboard or click the same element on the palette.

Adaptive ports and connections

Ports act as adaptive and intelligent endpoints within the diagram. Instead of remaining fixed, they respond to layout changes such as movement or resizing of elements, allowing connections to adjust naturally. Intelligent routing applies whenever the ports are unlocked.

Shared ports

When a port is connected to multiple edges, the system determines the most appropriate side of the parent element by considering all connected edges. Ports move to the side that best fits the overall layout.

Locking ports

- You can lock or unlock the ports even without any connected edges.
- Hovering over a port clearly indicates its lock state
- A single lock icon represents the entire shared port, even with multiple connections
- Locking or unlocking applies to all connected edges at once

Creating ports

Ports are essential elements specific to interconnection and general views, allowing connections between them and interface elements from the palette.

About this task

Procedure

- Create ports automatically by drawing interfaces. The direction of the ports is set as per the following rules:

  Both ports are created
  The direction of source port is set to out, and the target port is set to in.

  One port exists and has a direction
  The new port will have the opposite direction, unless the interface is between parent or child parts.

  One port exists without a direction
  The new port will also be without direction. The direction of source port is set to out and target port to in.
- To create ports by using the Browser view, complete the following substeps:
  1. Select the element for which you want to create ports.
  2. In the **Options** menu, click **Create child element**.
  3. From the **Category** list, select **Any**.
  4. From the **Types** list, choose the port type, and click **Add**.
- Drag and drop a port from the palette on a legal owner like part or part definition.
- In the diagram, drag and drop the port from the Browser view over its owner.
- To create ports by using the diagram contextual menu, complete the following substeps:
  1. In a diagram, select a node that can owns a port, like a part.
  2. Click the **Create element** (+) button that appears around the node, and click **Port**.

  Note: Ports are automatically added to all nodes of an owner of a port or to any element that inherits from the owner. This means that when a port is defined on a specific element referred to as the owner of the port, it is automatically included in all instances or derived elements of that owner throughout the model. Automatic synchronization of ports take place in the following conditions:

  – When you establish a specialization relationship between elements, Rhapsody Systems Engineering automatically synchronizes ports across the specialized or derived elements.
  – When you delete a specialization relationship between elements, Rhapsody Systems Engineering updates the synchronization of ports accordingly.

  Note: Missing ports are automatically added to the end of a newly created interface, with views updated to reflect these new elements. This automation simplifies the process and ensures an accurate model. Ports are added when the interface is not connected to a port on one or more ends.

  – When ports are created on both sides, the source port is assigned a new port definition, while the target port uses the conjugate of the source's port definition.

- If a port exists on one side and the other side is created, the new port uses the conjugate port definition of the existing port. If the existing port is not assigned a port definition, the newly added port also remains untyped.
- If the connected elements have a parent-child relationship, the port definition is not conjugated.

What to do next

To control and visualize the direction of ports, complete the following substeps:

1. Select the port that you want to visualize.
2. In the Properties panel, expand the **Essential** section.
3. In the **Direction** list, choose **in**, **inout**, or **out**.

Creating ports

Ports are essential elements specific to interconnection and general views, allowing connections between them and interface elements from the palette.

About this task

Procedure

- Create ports automatically by drawing interfaces. The direction of the ports is set as per the following rules:

  Both ports are created
  The direction of source port is set to out, and the target port is set to in.

  One port exists and has a direction
  The new port will have the opposite direction, unless the interface is between parent or child parts.

  One port exists without a direction
  The new port will also be without direction. The direction of source port is set to out and target port to in.
- To create ports by using the Browser view, complete the following substeps:
  1. Select the element for which you want to create ports.
  2. In the **Options** menu, click **Create child element**.
  3. From the **Category** list, select **Any**.
  4. From the **Types** list, choose the port type, and click **Add**.
- Drag and drop a port from the palette on a legal owner like part or part definition.
- In the diagram, drag and drop the port from the Browser view over its owner.
- To create ports, and set its direction by using the diagram contextual menu complete the following substeps.
  1. In a diagram, select a node that can owns a port, such as a part.
  2. Click the **Create element** (+) button that appears around the node, and click **In Port** or **Out Port**.

  Note: Ports are automatically added to all nodes of an owner of a port or to any element that inherits from the owner. This means that when a port is defined on a specific element referred to as the owner of the port, it is automatically included in all instances or derived elements of that owner throughout the model. Automatic synchronization of ports take place in the following conditions:
  - When you establish a specialization relationship between elements, Rhapsody Systems Engineering automatically synchronizes ports across the specialized or derived elements.
  - When you delete a specialization relationship between elements, Rhapsody Systems Engineering updates the synchronization of ports accordingly.

Note: Missing ports are automatically added to the end of a newly created interface, with views updated to reflect these new elements. This automation simplifies the process and ensures an accurate model. Ports are added when the interface is not connected to a port on one or more ends.

- When ports are created on both sides, the source port is assigned a new port definition, while the target port uses the conjugate of the source's port definition.
- If a port exists on one side and the other side is created, the new port uses the conjugate port definition of the existing port. If the existing port is not assigned a port definition, the newly added port also remains untyped.
- If the connected elements have a parent-child relationship, the port definition is not conjugated.

Controlling the visibility of ports or parameters

You can customize the visibility of ports or parameters.

Procedure

1. Select the node, whose appearance settings you want to review and update.
2. To review the appearance of ports, complete the following steps.
   a) From the contextual menu, click the **Additional operations** icon, and then click **Ports**.
   b) To show all ports, click **Show all ports**.
   c) To hide all ports, click **Hide ports**.
3. To review the appearance of parameters, complete the following steps.
   a) From the contextual menu, click the **Additional operations** icon, and then click **Parameters**.
   b) To show all parameters, click **Show all parameters**.
   c) To hide all parameters, click **Hide parameters**.

Additional operations of ports or parameters

You can customize the visibility of ports or parameters and create definitions for them.

Procedure

1. Select the node, whose appearance settings you want to review and update.
2. To review the appearance of ports or create definitions, complete the following steps.
   a) From the contextual menu, click the **Additional operations** icon, and then click **Ports**.
   b) Choose one of the following options.
      - **Show all ports**: Displays all ports.
      - **Show only connected ports**: Displays only connected ports.
      - **Hide all ports**: Hides all ports.
   c) To create port definitions, select **Create definition** from the **Additional operations** menu.
3. To review the appearance of parameters, complete the following steps.
   a) From the contextual menu, click the **Additional operations** icon, and then click **Parameters**.
   b) Choose one of the following options.
      - **Show all parameters**: Displays all parameters.
      - **Show only connected parameters**: Displays only connected parameters.
      - **Hide all parameters**: Hides all parameters.
   c) To create parameter definitions, select **Create definition** from the **Additional operations** menu.

Customizing the appearance of ports

Customize the appearance of ports by customizing their color, label content, and label positioning. Use the Appearance Settings window and Live preview section to adjust how port information is displayed.

Procedure

1. Select the port and then click **Customize mode**.
2. In the Appearance Settings window, you can modify the port color and its label settings. Use the Live Preview pane to view the changes as you make them. To get more viewing space, you can also resize it.
3. Use the Color and Formatting section to customize the appearance of the port, including its color.
4. Use the Labels section to customize both the content and position of port labels.
   - Under **Label Position**, select **Custom** to manually control the placement of the port label.
   - Under **Label Mode**, choose how the label content is displayed.
     - Under **Name** you can choose to display the short display name of port.
     - Under **Custom** you can choose to select which port properties are shown in the label, such as the name, definition, cardinality, and other available attributes.

Generating views using the Browser view panel

You can automatically generate a view such as a state transition view, interconnection view, use case view, or action flow view with content from the corresponding elements in the Browser view panel.

Procedure

1. Select the corresponding element of a view that you want to use as scope, and right-click it. For example: to automatically create a state transition view, select a state element.
2. Click **Populate to a new view**.

   In the Populate to a new view box, you get various options to customize your new view.
   - Under the **View name** and **Owner** fields, specify the view name and select the package or element where the view will be created.
   - Under **View type**, choose the type of view you want to generate.
   - From the **Element** list, click the icon and select the model-specific elements.
   - To include the recursive members of selected elements, select **Take recursive members of selected elements** checkbox.
   - Use the **Element type** list to choose the type of elements to be used for your selected view.
     - All: Includes all elements.
     - By palette: Includes only elements that are available in the palette of the selected view.
     - Custom: Allows you to choose the element types manually from the **Element types** list.
   - Use the **Relationship type** list to define the relationship types to be added.
     - All: Includes all relationship types.
     - By palette: Includes only relationships that are available in the palette of the selected view.
     - None: No relationship is added.
     - Custom: Allows you to choose the relation types manually from the **Relationship elements** list.
   - Under Choose which relationships are displayed, you get the following options.
     - Among selected: The relationships between those elements that came from the scope and the filter will only be displayed

– From selected, To selected, Both directions: Defines the direction of displayed relationships. Based on the selected direction, relationships are added, and any missing elements at the ends of those relationships are included along with the defined scope.

Click **Populate** to generate your view

Creating views with content from packages

You can automatically create a view with content from packages in the **Browser view** panel. A general view is created. A new view is created corresponding to the type of the selected element. For example: if you want to create a state transition view, choose a package that contain state elements.

Procedure

1. Choose the package from where you want to populate the content in the view area.
2. Right-click the package. This action opens a contextual menu with available options.
   - If you want to populate the content from the selected package only, click **Exclude sub-packages of the package**.
   - If you want to populate the content from all packages within the selected package, click **All package elements**.

Reviewing the history of selected elements

You can review the history of your selected elements by using the "Go backward" and "Go forward" buttons available on the project toolbar. This allows you to keep a track of your selected elements and make changes on them. The selected element is highlighted in the Browser view panel, while the Properties panel displays the relevant details of the element.

Procedure

- To go back and see your selections in the history, click **Go backward** (←) in the project toolbar.
- To move forward and see your selections in the history, click **Go forward** (→) in the project toolbar.
- To view the history of your selections, press the **Go backward** or **Go forward** buttons for a longer period of time.

Removing elements

You can either choose to remove elements from the view or permanently delete them from the model.

Procedure

1. Select the node, and click the **Delete** icon from the contextual menu. To remove multiple elements at one time, select the elements and click the **Delete** icon.
2. To permanently delete the element from the model and from all views, select **Delete from model**.
3. To remove the selected node from the view or diagram, and leave it on the model, select **Remove from view**. You can also click the **Delete** icon in the right toolbar of the view.

   You can choose to remove the relationships, and a member of compartment from the diagram or model by using the above procedure.
4. Click **Save**.

Creating connections between nodes

Use the diagram palette in the view to create connections between nodes.

Procedure

- In the diagram palette, select an element. Click on the edge of the node from where you want to create connections or relations and drag it over the other node.

  Note: You can create a connection from any point of the node, and it automatically snaps into place near the connection points of other nodes, aligning and locking into the available positions.
- For some elements, you can create connections by using the **Create element** (+) option.
- For some edges, you can create sub-flows.

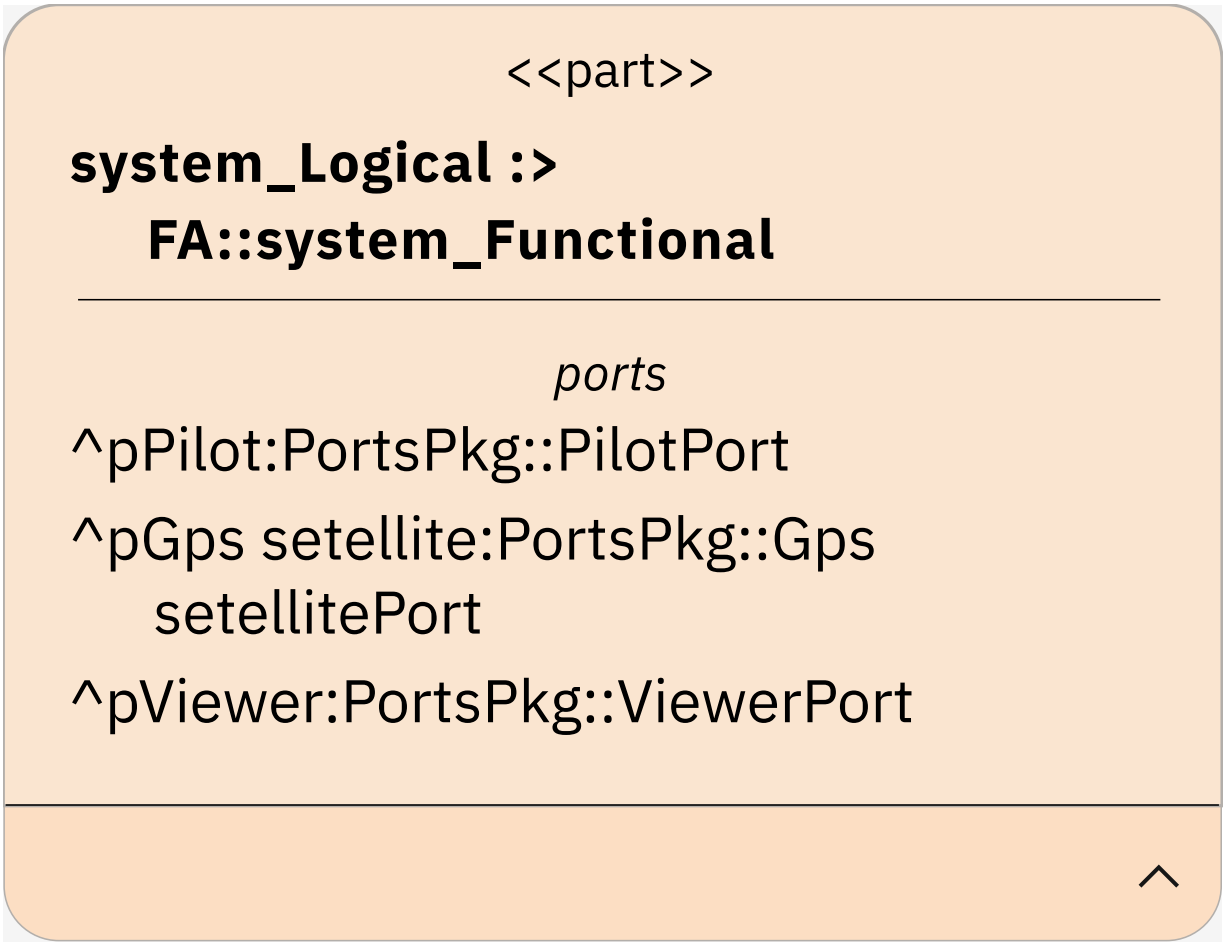
  1. Click on the edges. A contextual menu appears.
  2. Click **Manage sub-connectors** ( ) icon.
  3. Select the connector type.
  4. Choose the sub-elements from the **Source** and **Target** lists that you want to connect.
  5. Review the direction of flow. If you want the connection to go from target to source, select the **Reverse** checkbox.
  6. In the **Existing sub-connections** section, review the connection that you added. You can create multiple connections by repeating Step “4” on page 129. You can also remove the connections by clicking the **Remove** icon, which is available next to the target attribute.

Working with compartments

Compartments are specific to nodes and their availability depends on the type of model element that the node represents.

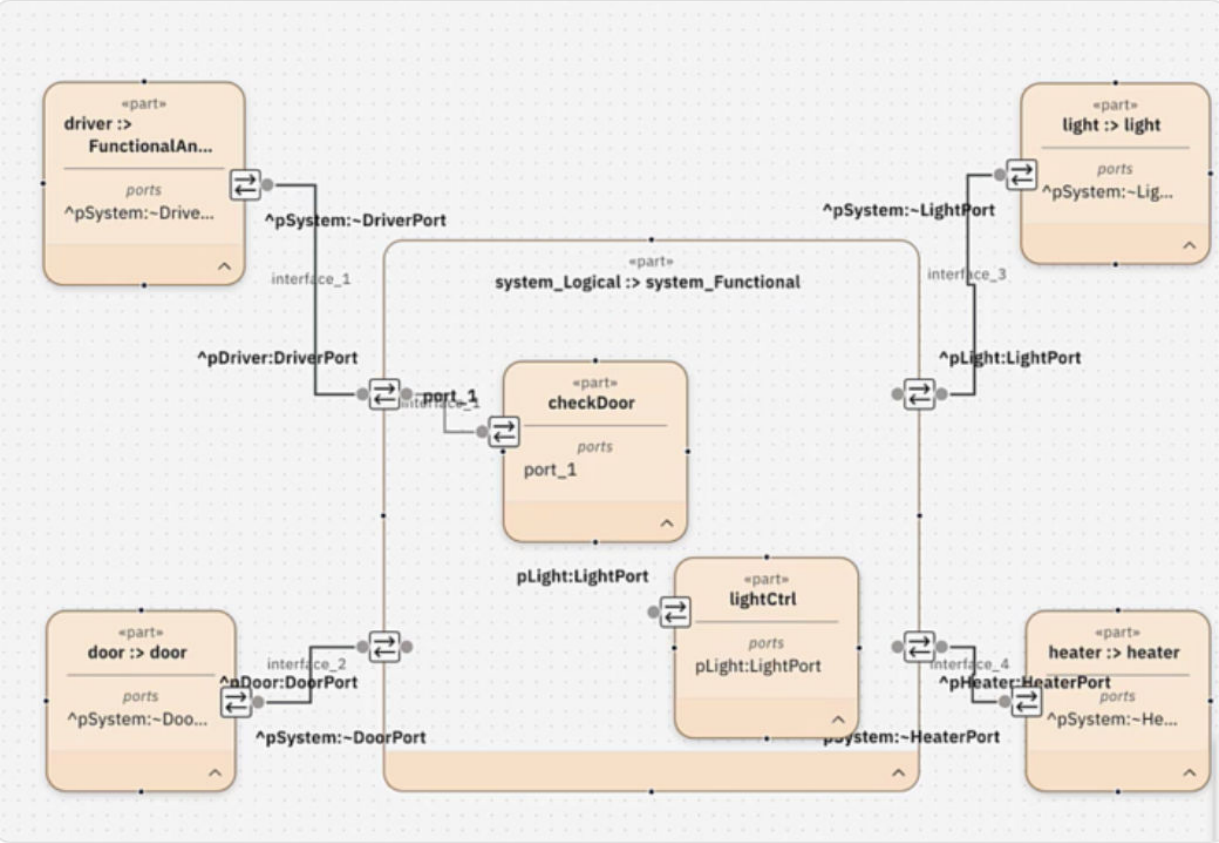
There are two types of compartments:

Textual
: Shows owned elements grouped by element type, in sections with separators and title. It appears only when there are relevant elements to display for the group's type. The following is an example of a node with a textual compartment.



Graphical

Allows placing sub-nodes to express sub-structure or sub-elements. To move a node out of a graphical compartment, select the node, click the **Detach from view** ( ) icon from the contextual toolbar, and move the mouse pointer to the new position where you want to drop the detached node. The following is an example of a node with a graphical compartment.



You can switch between textual and graphical mode by using the context toolbar. For example: in a textual compartment by adding attributes to a node that represents a part automatically includes the «`attributes`» compartment and displays the newly added attribute. Similarly, if you add port, then a new compartment «`ports`» appear that shows the newly added port.

Searching elements in project editor

You can search, locate, and validate elements by applying different filters in the project editor.

Procedure

1. In the project editor, on the global toolbar, click the **Search elements** box.
2. From the **All category** list, choose the appropriate category related to your search.
3. In the **Search elements** box, type the name of the element that you are searching for. A list of matching elements appear.
4. To locate and view an element from the results in the browser view, select it. The Properties panel is also updated with the selected element's properties.
5. To view the complete list of matching elements, click **View all results**.
6. In the Search results page, you can select a different category under the **Filters** column.
7. You can directly validate, locate an element in views, and delete the element from the table of filtered results. In the element row, click the **More options** ⋮ icon.
8. To return to your initial filter settings, click **Reset** .

Using visual indicators to identify invalid graphical elements

To ensure that your views remain logically correct, Rhapsody Systems Engineering provides visual indicators for graphical elements, such as nodes or edges. These indicators provide immediate feedback whenever semantic inconsistencies occur, allowing you to quickly identify and resolve issues during modeling.

Procedure

1. Observe the view for highlighted elements, which indicate semantic inconsistencies. If your graphical elements, such as nodes or edges are misplaced or do not confirm to their logical definitions, the invalid element gets highlighted in the view.
2. To identify the reason for the inconsistency, hover over the highlighted element in the view. A message describing the reason for inconsistency appears.

Syntax

Syntax specifies how the SysML language appears to you. The SysML syntax includes both textual notation and graphical notation. Various views of a SysML model can be rendered using the textual notation, graphical notation, or by using a combination of the two. Currently, only the subset of textual and graphical notations is supported.

Textual notation

Textual notation defines how lexical tokens for an input text are grouped in order to construct an abstract syntax representation of a model.

The concrete syntax grammar definition uses standard notation that describes how the syntax maps to the abstract syntax. For example, in name compartment, for a part, you can specify if it is a subsetting, redefinition expression, multiplicity, and more. In a textual compartment, you can do similar operations by including redefinition. In action view, you can specify accept, send, or perform expressions, and loops definition.

Graphical notation

Graphical notation is used to define the SysML textual notation.

A graphical production contains a two-dimensional layout of graphical and textual elements including graphical shapes and lines. Shapes might contain other elements nested within these shapes. In this notation, you are aligned with the standard rendering definition. The graphical compartments are allowed to express sub-members of an element and their relations.

Language extension

The SysML V2 language capabilities enable you to introduce new concepts or element types based on existing element types. For example, creating a new concept called `Aircraft` based on `PartUsage`.

About this task

You require server administrator privileges to establish a project usage from the project that uses the domain-specific concepts to the extension library where the concepts are defined.

Procedure

1. Define the extension library in a project, containing the extended modeling concepts and views. For example, to define an extension library Aviary, complete the following substeps:
   a) Prepare a library project.
      i) Create a new project.
      ii) Add a new library package, such as Aviary Library.
      iii) Under **Aviary Library**, add a new language extension view, aviary DSL Definition View.
   b) Define a new concept, such as `Aircraft`, with base type `PartUsage`.
      i) In the new view, draw a semantic metadata definition element to the canvas.
      ii) Rename the new element to be: `Aircraft :> Semantic Metadata`.
      iii) Set the second line text to be: `:>> baseType = aircrafts meta SysML::PartUsage`

      ```
      metadata def
      Aircraft :>SemanticMetadata
      :>> baseType = aircraftsmetaSysML::PartUsage
      ```

      The following elements are automatically created under the Aviary Library package.
      - PartDefinition: Aircrafts
      - PartUsage: aircrafts of type (PartDefinition) Aircraft

      iv) Save the project. You can start adding new elements.
      v) To share the library with other projects, commit the changes back to the main branch.
2. Create a project usage from your project to the "Aviary Library" project.
3. Import the Aviary Library to you project.

   For more information, see “Customizing view definitions” on page 136.
4. Save and reopen your project.
5. Use extended types and views in your project.
   a) To create elements based on the extended library, in the Browser view panel, click the **Add element** (+) icon and follow the same procedure in “Creating elements” on page 97. In case you defined a custom view, you can also use the new palette to add elements to your custom view.
6. Create a new extension element from the user defined custom view.
   a) Create a new view from one of the custom views, which contains extension elements in its palette, for example: AircraftView.

b) Drag and drop an extension element from the palette to canvas, for example: `Usages > #Ac` for `#Aircraft`.

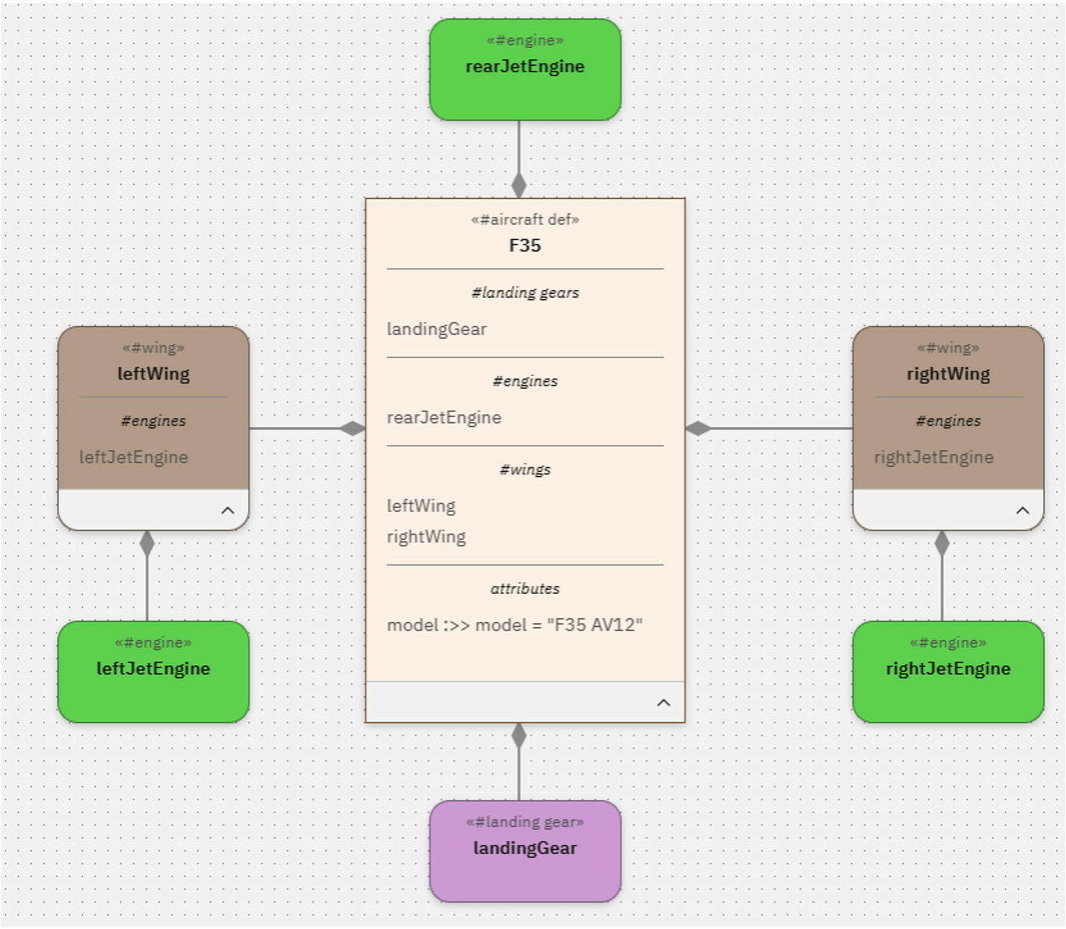
c) Add extension element from the node menu, for example: #Wing. The new `#Wing` element gets added in its own compartment `#wings`. The type of an extension element, in the Properties pane, is shown as `#<concept name>`.

For example: for the element `aircraft_1` it shows : `#Aircraft`

7. You can change the type of element.
   - From a regular element to any concept of the same base type.
   - From a concept element to its base type.

Results

The following is a sample model created by using concepts from the Aviary library.



Metadata support

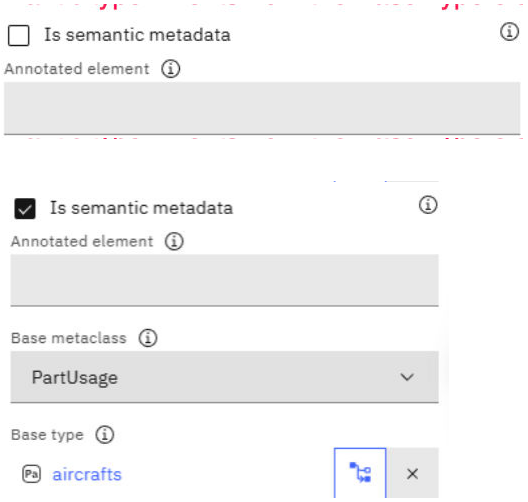
Metadata in SysML v2 models allows you to add information to elements without changing their core semantics or structure. This is done by using metadata definition elements, which you can create and manage in the language extension view.

Metadata definitions allow you to augment elements with additional information using attributes. This is useful for adding non-model data to support tool extensions, process definition, and organization specific compliance.

To simplify working with metadata definition, a number of simplified properties are available.

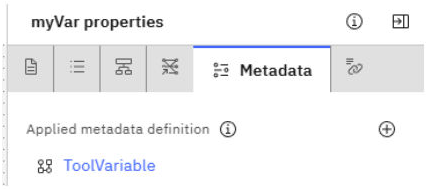
- Annotated elements: Defines which element types the metadata definition applies to. This is a comma separated list of basic element types, such as PartUsage, ActionUsage, and more.

- Is semantic metadata: Marks the metadata definition as a language extension element, enabling semantic behavior. When enabled, additional properties appear:
  - Base metaclass: Specifies the metaclass, such as `PartUsage` that the metadata definition is evaluated to.
  - Base Type: Specifies the a base element whose type complies with the Base metaclass. Elements of this semantic type inherit from the Base Type thus inheriting its features.

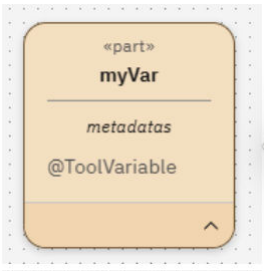


You can apply metadata to elements in several ways.

- Properties panel: In the Metadata property pane, use the Applied metadata definition.

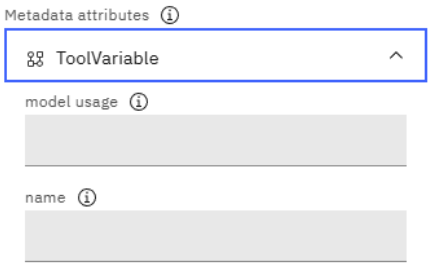


- Drag and drop: Drag a metadata definition from the Browser panel and drop it over the model elements in a view.
- Context toolbar: Create metadata usage directly from the context menu of the node. Update the text to refer to the metadata definition by typing "`@<metadata-def-name>`", for example: "`@ToolExtension`"



Note: While applying metadata on an element the system checks whether metadata is valid to the element type. If it is not valid, the operation is blocked.

You can override the values for metadata attributes with element's specific values. Navigate to the Metadata property pane to see the available attributes per applied metadata definition.



The model validator checks for inconsistency in metadata usage, such as duplicate metadata definitions, type incompatibility between the element and the metadata definition, and incorrect annotation element types. If a metadata definition is incompatible, the model validator notifies you with an option to remove it from list of applied metadata definitions.

Customizing view definitions

In Rhapsody Systems Engineering, you can define custom views definitions based on any of the predefined view definitions. These custom views allow you to create views specific to the extended domain, redefine the view's palette to include the new concepts or elements.

Procedure

1. Define custom views, such as AircraftView based on GeneralView.
   a) Add a Subclassification relation between the new AircraftView view definition and the GeneralView view definition by opening a language extension view and drawing a new view definition and setting the label as `AircraftView :> GeneralView`.
2. You can change the palette definition of a user defined custom view.
   a. Add a child element: `TextualRepresentation` , named `paletteJson`, under the view definition: AircraftView.
   b. Under the properties of `paletteJson`, set the property `Language` as `JSON`.
   c. From `IBMViews::View Definitions::GeneralView::paletteJson` copy the value of the property Body to the property Body of Body of `AircraftView`.
   d. Update the json file defined in Body to match the palette content you prefer, for example, in the Usages section to add `#AirCraft`, add the following snippet.

```
{
    "type": "PaletteItem",
    "paletteItemMetaclass": "#Aircraft",
    "paletteItemLabel": "# Air craft",
    "paletteItemToolTip": "# Air craft",
    "paletteItemIcon": "Air craft-icon",
    "renderingName": "asNodeUsage",
    "viewDefinitionName": "NodeView”
}
```

```
{
"type": "PaletteItemsGroup",
"label": "Usages",
"paletteItemIcon": "Interconnection-icon-url",
"items": [
{
"type": "PaletteItem",
"paletteItemMetaclass": "#Aircraft",
"paletteItemLabel": "# Air craft",
"paletteItemToolTip": "# Air craft",
"paletteItemIcon": "Air craft-icon",
"renderingName": "asNodeUsage",
"viewDefinitionName": "NodeView"
},
```

e. In the Definitions section, to add the `#AirCraft` definition, add the following snippet:

```
{
    "type": "PaletteItem",
    "paletteItemMetaclass": "#Aircraft definition",
    "paletteItemLabel": "# Air craft definition",
    "paletteItemToolTip": "# Air craft definition",
    "paletteItemIcon": "Aircraft definition-icon",
    "renderingName": "asNodeDefinition",
    "viewDefinitionName": "NodeView"
}
```

```
{
"type": "PaletteItemsGroup",
"label": "Definitions",
"paletteItemIcon": "Definitions-icon-url",
"items": [
{
"type": "PaletteItem",
"paletteItemMetaclass": "#Aircraft definition",
"paletteItemLabel": "# Air craft definition",
"paletteItemToolTip": "# Air craft definition",
"paletteItemIcon": "Aircraft definition-icon",
"renderingName": "asNodeDefinition",
"viewDefinitionName": "NodeView"
},
```

f. Save the project. To share the library with other projects, commit the changes back to the main branch.

Related tasks

“Language extension” on page 133

The SysML V2 language capabilities enable you to introduce new concepts or element types based on existing element types. For example, creating a new concept called `Aircraft` based on `PartUsage`.

Enabling extensions

In Rhapsody Systems Engineering to improve the efficiency of your projects, you can directly enable extensions from its user interface.

Procedure

1. In the project editor page, from the toolbar, click the **More actions** (⋮) icon, and click **Manage extensions**.
2. Review the list of available extensions, and select the extensions that you want to enable for your project.
3. To apply the changes, click **Save**.

Integrating

Rhapsody Systems Engineering integrates with other products to enhance its capabilities and extend its functionality across different aspects of systems engineering and software development. These integrations promote collaboration, improve efficiency, and ensure alignment between different stages of development.

Links

In Rhapsody Systems Engineering, you can manage links between its model elements and artifacts of friend applications. You can create, view, and delete links to artifacts of friend applications from Rhapsody Systems Engineering.

Rhapsody Systems Engineering supports the following OSLC link types to Requirements Management applications:

- Derives From
- Satisfies
- Refines
- Trace

Rhapsody Systems Engineering supports the following OSLC link types to Architecture Management applications:

- Tracks
- Allocates
- Realizes

Rhapsody Systems Engineering supports the following OSLC link types to Change Management applications:

- Elaborates

Rhapsody Systems Engineering supports the following OSLC link types to Quality Management applications:

- Validated By

Related concepts

“Managing open services for lifecycle collaboration integrations” on page 81
Managing open services for lifecycle collaboration (OSLC) integrations for Rhapsody Systems Engineering includes integrating other products with Rhapsody Systems Engineering, integrating Rhapsody Systems Engineering with IBM Engineering Lifecycle Management products, and creating associations between the projects of Rhapsody Systems Engineering and the projects of other friend applications.

Managing links

Managing links in Rhapsody Systems Engineering involves several key actions: creating new links between model elements and artifacts of friend applications, viewing detailed information about existing links, and removing links when necessary. This functionality allows for comprehensive management of relationships between Rhapsody Systems Engineering model elements and artifacts in friend application, ensuring effective traceability and alignment throughout the development process.

Note: To manage links between the artifacts of Rhapsody Systems Engineering and other products, open Rhapsody Systems Engineering in the context of global configuration.

Creating links

In Rhapsody Systems Engineering, you can create links from both browser view and the Properties panel. This functionality enables seamless integration and traceability between model elements and artifacts managed in friend applications. Whether you are navigating through the browser view or editing specific properties in the Properties panel, you can establish new links efficiently, enhancing the connectivity and coherence of your development workflows.

About this task

You can create links from both browser view or Properties panel.

Procedure

1. To create links from the browser view, hover over an element, and click the **Options** icon next to the element. Click **Create links**.
2. To create links from **Properties** panel, expand **Links**, and click the **Add** icon.
3. In the **Link type** list, select an appropriate type of link that you want to create.
4. In the **Artifact container** list, select the associated project.
5. Choose from one of the artifact types:

   Existing artifact
   Selects the existing artifacts to link to model elements.

   New artifact
   Creates new artifacts to link to model elements.
6. Click **OK**.

Related information

Linking to development, design, test, and requirement artifacts

Linking artifacts by drag and drop

You can establish linking between the artifacts of Rhapsody Systems Engineering, DOORS® Next, and Rhapsody Model Manager by using the drag and drop method.

When you link artifacts, OSLC link types are created between the applications. Ensure that the project areas in Rhapsody Systems Engineering, DOORS Next, and Rhapsody Model Manager are associated with each other for link creation.

Note: You can only link artifacts between similar global configurations.

Linking Rhapsody Systems Engineering artifacts

You can link the elements of Rhapsody Systems Engineering with the artifacts of DOORS Next and Rhapsody Model Manager by using the drag and drop method.

Procedure

1. In Rhapsody Systems Engineering, select the elements from the Browser view panel that you to use for link creation.
2. Drag the selected elements into another window or tab that displays the corresponding elements in DOORS Next or Rhapsody Model Manager. Ensure that the applications are opened within the same browser instance.
3. Drop the Rhapsody Systems Engineering elements into the **Drop artifacts** window within the target artifact editor in DOORS Next or Rhapsody Model Manager.
4. In the **Select the link type** list, choose the link type that you prefer.
5. Click **Create links**. In the Links section of the target artifact editor, you can view the linked artifact and the link type.

Linking DOORS Next and Rhapsody Model Manager artifacts

You can link the artifacts of DOORS Next and Rhapsody Model Manager with the elements of Rhapsody Systems Engineering by using the drag and drop method.

Procedure

1. In DOORS Next and Rhapsody Model Manager, select the artifacts that you want to use for link creation.
2. Drag the selected artifacts into another window or tab that displays the corresponding elements in Rhapsody Systems Engineering. Ensure that the applications are opened within the same browser instance.

3. Drop the artifacts of DOORS Next and Rhapsody Model Manager into the **Create links** window within the Rhapsody Systems Engineering.
4. In the **Owner** list, you can select the target element that you want to use for the link.
5. In the **Link type** list, choose the link type that you prefer.
6. Click **Create links**. In the Links section of the target element, you can view the linked artifact and the link type.

Viewing links

You can explore and inspect the links that are associated with any element within your system model in IBM Rhapsody Systems Engineering. This capability helps in understanding the relationships and dependencies between different elements, ensuring comprehensive analysis of your system design.

Procedure

1. In the browser view, select the element for which you want to view the links. The properties are displayed in the Properties panel.
2. In the **Properties** panel, expand the **Links** section and then expand the link type.

   For example, you might have links categorized by type such as Satisfies, Derives From, Allocated By, or more.
3. To view the details of the link, click the link.

Related information

Linking to development, design, test, and requirement artifacts

Removing links

You can efficiently manage and remove links associated with elements in IBM Rhapsody Systems Engineering. This process helps to maintain the accuracy of your system models by removing unnecessary or outdated links between elements.

Procedure

1. In the browser view, select the element for which you want to remove the links. The properties are displayed in the Properties panel.
2. In the **Properties** panel, expand the **Links** section and then expand the link type.
3. Locate the specific link that you want to remove. Click the **Remove** icon located next to the link.

Related information

Linking to development, design, test, and requirement artifacts

Requirements coverage

After you create links from Rhapsody Systems Engineering elements to DOORS Next 7.1 requirements, you can generate tabular or graphical requirements coverage reports by using a Python script or Jupyter Notebook. This determines the DOORS Next 7.1 requirements in a DOORS Next 7.1 module that are covered by Rhapsody Systems Engineering elements.

Requirement coverage requires one or more links of any type between Rhapsody Systems Engineering elements and the DOORS Next 7.1 requirement. Requirements coverage reports contain the following metrics:

- Percentage of requirements covered or uncovered by one or more elements for each link type.
- Total number of requirements covered or uncovered by one or more elements for each link type.

To generate a tabular requirements coverage report, run the requirements_coverage_report.py Python script. To generate a graphical requirements coverage report, run the requirements_coverage_notebook.ipynb Jupyter Notebook.

Integrating with IBM Engineering Lifecycle Management

Rhapsody Systems Engineering and Engineering Lifecycle Management are compliant with the OSLC specification, enabling seamless integration to link artifacts and ensure traceability throughout the engineering process.

Note: Configure Rhapsody Systems Engineering to use OIDC provider that generates access token less than 250 characters. For more information, see “Authenticating with OIDC” on page 32.

Consider the following videos on how to integrate Rhapsody Systems Engineering with Doors Next using Rhapsody Systems Engineering and Doors Next applications.

https://mediacenter.ibm.com/playlist/dedicated/1_3facrih1/.

Registering IBM Rhapsody Systems Engineering as a client

Registering Rhapsody Systems Engineering as a client includes submitting the JAS client registration request by using an appropriate REST client, such as postman.

For more information, see “Registering Rhapsody Systems Engineering as a client” on page 75.

Note: This procedure is only applicable if you are using JAS as an OIDC provider for Rhapsody Systems Engineering.

Updating the configuration file

To start using JAS as an OIDC provider for Rhapsody Systems Engineering, update or add attributes to the operator configuration file.

For more information, see “Updating the configuration file” on page 76.

Establishing friendship from Rhapsody Systems Engineering

Learn how to establish friendship connection from Rhapsody Systems Engineering .

For more information, see the following video on how to integrate Rhapsody Systems Engineering with IBM Doors Next.

Establishing friendship with DOORS Next

Establishing friendship with DOORS Next includes using the correct rootservices URL and approving the consumer key through the Jazz Team Server Administration page.

Procedure

1. Add DOORS Next base URL to the allowlist. For more information, see “Managing the URL allowlist” on page 82.
2. Create friendship with DOORS Next. For more information, see “Integrating other products with Rhapsody Systems Engineering” on page 81.
3. Use the following format for the rootservices URL.

   ```
   https://<elm_host_name>:<port_number>/rm/rootservices
   ```

4. If you are unable to grant access to the provisional key by using DOORS Next user interface, see Approving consumer keys to approve consumer key.
5. Go to the Jazz Team Server Administration page to approve DOORS Next consumer key by using the following URL.

   ```
   https://<elm_host_name>:<port_number>/jts/admin
   ```

6. To create links, add the Rhapsody Systems Engineering branch to the Global Configuration . For more information, see “Adding a branch or tag to the global configuration” on page 95.

Establishing friendship with Rhapsody Model Manager or Engineering Workflow Management

Establishing friendship with Rhapsody Model Manager or Engineering Workflow Management includes using the correct rootservices URL and approving the consumer key through the Engineering Workflow Management Administration page.

Procedure

1. Add Rhapsody Model Manager or Engineering Workflow Management base URL to the allowlist. For more information, see “Managing the URL allowlist” on page 82.
2. Create friendship with Rhapsody Model Manager or Engineering Workflow Management. For more information, see “Integrating other products with Rhapsody Systems Engineering” on page 81.
3. Use the following format for the rootservices URL.

```
https://<elm_host_name>:<port_number>/ccm/rootservices
```

4. If you are unable to grant access to the provisional key by using Engineering Workflow Management or Rhapsody Model Manager user interface, see Approving consumer keys to approve consumer key.
5. Go to the Engineering Workflow Management Administration page to approve Rhapsody Model Manager or Engineering Workflow Management consumer key by using the following URL.

```
https://<elm_host_name>:<port_number>/ccm/admin
```

6. To create links, add the Rhapsody Systems Engineering branch to the Global Configuration . For more information, see “Adding a branch or tag to the global configuration” on page 95.

Establishing friendship with Engineering Test Management

Establishing friendship with Engineering Test Management includes using the correct rootservices URL and approving the consumer key through the Engineering Test Management Administration page.

Procedure

1. Add Engineering Test Management base URL to the allowlist. For more information, see “Managing the URL allowlist” on page 82.
2. Create friendship with Engineering Test Management. For more information, see “Integrating other products with Rhapsody Systems Engineering” on page 81.
3. Use the following format for the rootservices URL.

```
https://<elm_host_name>:<port_number>/qm/rootservices
```

4. If you are unable to grant access to the provisional key by using Engineering Test Management user interface, see Approving consumer keys to approve consumer key.
5. Go to the Engineering Test Management Administration page to approve Engineering Test Management consumer key by using the following URL.

```
https://<elm_host_name>:<port_number>/qm/admin
```

6. To create links, add the Rhapsody Systems Engineering branch to the Global Configuration . For more information, see “Adding a branch or tag to the global configuration” on page 95.

Establishing friendship with Global Configuration

Establishing friendship with Global Configuration includes using the correct rootservices URL and approving the consumer key through the Jazz Team Server Administration page.

Procedure

1. Add Global Configuration base URL to the allowlist. For more information, see “Managing the URL allowlist” on page 82.
2. Create friendship with Global Configuration. For more information, see “Integrating other products with Rhapsody Systems Engineering” on page 81.
3. Use the following format for the rootservices URL.

```
https://<elm_host_name>:<port_number>/gc/rootservices
```

4. If you are unable to grant access to the provisional key by using Global Configuration user interface, see Approving consumer keys to approve consumer key.
5. Go to the Jazz Team Server Administration page to approve Global Configuration consumer key by using the following URL.

```
https://<elm_host_name>:<port_number>/jts/admin
```

6. Provide functional user for the consumer key to perform background tasks.

Establishing friendship from Engineering Lifecycle Management

Learn how to establish friendship connection from Engineering Lifecycle Management.

For more information, see the following video on how to integrate Rhapsody Systems Engineering with IBM Doors Next.

Establishing friendship from Jazz Team Server

Establishing friendship with Jazz Team Server includes using the correct rootservices URL and approving the consumer key through the Rhapsody Systems Engineering Administration page.

About this task

The following procedure is applicable if Doors Next, Rhapsody Model Manager, Engineering Workflow Management, Engineering Test Management, and Global Configuration applications are registered on same Jazz Team Server application.

Procedure

1. Add Rhapsody Systems Engineering base URL to the allowlist. For more information, see Managing the URL allowlist.
2. Go to the Jazz Team Server Administration page by using the following URL.

```
https://<elm_host_name>:<port_number>/jts/admin
```

3. Create friendship with Rhapsody Systems Engineering. For more information, see Establishing friend relationships.
4. Follow the following format of Rhapsody Systems Engineering rootservices URL.

```
https://<rse_host_name>:<port_number>/<custom_path>/api/rootservices
```

5. If you are unable to grant access to the provisional key by using Rhapsody Systems Engineering user interface, see “Approving consumer keys” on page 83 to approve consumer key.

6. Go to the Rhapsody Systems Engineering Administration page to approve the consumer key by using the following URL.

```
https://<rse_host_name>:<port_number>/admin
```

Establishing friendship from DOORS Next

Establishing friendship with DOORS Next includes using the correct rootservices URL and approving the consumer key through the Rhapsody Systems Engineering Administration page.

About this task

The following procedure is applicable Doors Next, Rhapsody Model Manager, Engineering Workflow Management, Engineering Test Management, and Global Configuration applications are not registered on same Jazz Team Server application.

Procedure

1. Add Rhapsody Systems Engineering base URL to the allowlist. For more information, see Managing the URL allowlist.
2. Go to the DOORS Next Administration page by using the following URL.

```
https://<elm_host_name>:<port_number>/rm/admin
```

3. Create friendship with Rhapsody Systems Engineering. For more information, see Establishing friend relationships.
4. Follow the following format of Rhapsody Systems Engineering rootservices URL.

```
https://<rse_host_name>:<port_number>/<custom_path>/api/rootservices
```

5. If you are unable to grant access to the provisional key by using Rhapsody Systems Engineering user interface, see “Approving consumer keys” on page 83 to approve consumer key.
6. Go to the Rhapsody Systems Engineering Administration page to approve the consumer key by using the following URL.

```
https://<rse_host_name>:<port_number>/admin
```

Establishing friendship from Rhapsody Model Manager or Engineering Workflow Management

Establishing friendship with Rhapsody Model Manager or Engineering Workflow Management includes using the correct rootservices URL and approving the consumer key through the Rhapsody Systems Engineering Administration page.

About this task

The following procedure is applicable Doors Next, Rhapsody Model Manager, Engineering Workflow Management, Engineering Test Management, and Global Configuration applications are not registered on same Jazz Team Server application.

Procedure

1. Add Rhapsody Systems Engineering base URL to the allowlist. For more information, see Managing the URL allowlist.
2. Go to the Rhapsody Model Manager or Engineering Workflow Management Administration page by using the following URL.

```
https://<elm_host_name>:<port_number>/ccm/admin
```

3. Create friendship with Rhapsody Systems Engineering. For more information, see Establishing friend relationships.
4. Follow the following format of Rhapsody Systems Engineering rootservices URL.

```
https://<rse_host_name>:<port_number>/<custom_path>/api/rootservices
```

5. If you are unable to grant access to the provisional key by using Rhapsody Systems Engineering user interface, see “Approving consumer keys” on page 83 to approve consumer key.
6. Go to the Rhapsody Systems Engineering Administration page to approve the consumer key by using the following URL.

```
https://<rse_host_name>:<port_number>/admin
```

Establishing friendship from Engineering Test Management

Establishing friendship with Engineering Test Management includes using the correct rootservices URL and approving the consumer key through the Rhapsody Systems Engineering Administration page.

About this task

The following procedure is applicable if Doors Next, Rhapsody Model Manager, Engineering Workflow Management, Engineering Test Management, and Global Configuration applications are not registered on same Jazz Team Server application.

Procedure

1. Go to the Engineering Test Management Administration page by using the following URL.

```
https://<elm_host_name>:<port_number>/qm/admin
```

2. Create friendship with Rhapsody Systems Engineering. For more information, see Establishing friend relationships.
3. Follow the following format of Rhapsody Systems Engineering rootservices URL.

```
https://<rse_host_name>:<port_number>/<custom_path>/api/rootservices
```

4. If you are unable to grant access to the provisional key by using Rhapsody Systems Engineering user interface, see “Approving consumer keys” on page 83 to approve consumer key.
5. Go to the Rhapsody Systems Engineering Administration page to approve the consumer key by using the following URL.

```
https://<rse_host_name>:<port_number>/admin
```

Establishing friendship from Global Configuration

Establishing friendship with Global Configuration includes using the correct rootservices URL and approving the consumer key through the Rhapsody Systems Engineering Administration page.

About this task

The following procedure is applicable Doors Next, Rhapsody Model Manager, Engineering Workflow Management, Engineering Test Management, and Global Configuration applications are not registered on same Jazz Team Server application.

Procedure

1. Add Rhapsody Systems Engineering base URL to the allowlist. For more information, see Managing the URL allowlist.
2. Go to the Global Configuration Administration page by using the following URL.

```
https://<elm_host_name>:<port_number>/gc/admin
```

3. Create friendship with Rhapsody Systems Engineering. For more information, see Establishing friend relationships.
4. Follow the following format of Rhapsody Systems Engineering rootservices URL.

```
https://<rse_host_name>:<port_number>/<custom_path>/api/rootservices
```

5. If you are unable to grant access to the provisional key by using Rhapsody Systems Engineering user interface, see “Approving consumer keys” on page 83 to approve consumer key.
6. Go to the Rhapsody Systems Engineering Administration page to approve the consumer key by using the following URL.

```
https://<rse_host_name>:<port_number>/admin
```

Re-establishing the existing friendship between Rhapsody Systems Engineering and Engineering Lifecycle Management

If you want to re-establish the friendship connection between Rhapsody Systems Engineering and Engineering Lifecycle Management applications, begin by clearing the existing friendship connections. If you are establishing friendship between them for the first time, you can skip this process.

Removing friendship from Rhapsody Systems Engineering

Deleting the friendship connections from Rhapsody Systems Engineering includes going to the Administration page and then deleting the friendship cinnections.

Procedure

1. Go to the Rhapsody Systems Engineering Administration page by using the following URL

```
https://<rse_host_name>:<port_number>/admin
```

2. On the Administration page, click **Connections**, and then go to the **Friends (Outbound)** section.
3. Click the **Delete** icon to delete the friendship connections with DOORS Next, Rhapsody Model Manager, and Global Configuration applications.
4. Delete the Engineering Lifecycle Management (Jazz Team Server, DOORS Next, Rhapsody Model Manager, Engineering Test Management, Engineering Workflow Management, Global Configuration) registered with Rhapsody Systems Engineering) consumers. For more information, see “Deleting consumer keys” on page 83.

Removing friendship from Jazz Team Server

Deleting the friendship from Jazz Team Server includes going to the Jazz Team Server Administration page, deleting the friendship with Rhapsody Systems Engineering, and deleting the consumer keys from Rhapsody Systems Engineering.

Procedure

1. Go to the Jazz Team Server Administration page by using the following URL.

```
https://<elm_host_name>:<port_number>/jts/admin
```

2. Delete the friendship with Rhapsody Systems Engineering. For more information, see Deleting servers from the friends list.
3. Delete consumer keys from Rhapsody Systems Engineering. For more information, see Deleting consumer keys.

Note: Jazz Team Server contains consumer key from Rhapsody Systems Engineering for DOORS Next, and Global Configuration.

Removing friendship from DOORS Next

Deleting the friendship from DOORS Next includes going to the DOORS Next Administration page, and deleting the friendship with Rhapsody Systems Engineering.

Procedure

1. Go to the DOORS Next Administration page by using the following URL

   ```
   https://<elm_host_name>:<port_number>/rm/admin
   ```

2. Delete the friendship with Rhapsody Systems Engineering. For more information, see Deleting servers from the friends list.

Removing friendship from Rhapsody Model Manager or Engineering Workflow Management

Deleting the friendship from Rhapsody Model Manager or Engineering Workflow Management includes going to the Rhapsody Model Manager Administration page, and deleting the friendship with Rhapsody Systems Engineering.

Procedure

1. Go to the Rhapsody Model Manager or Engineering Workflow Management Administration page by using the following URL.

   ```
   https://<elm_host_name>:<port_number>/ccm/admin
   ```

2. Delete the friendship with Rhapsody Systems Engineering. For more information, see Deleting servers from the friends list.
3. Delete consumer keys from Rhapsody Systems Engineering. For more information, see Deleting consumer keys.

Removing friendship from Engineering Test Management

Deleting the friendship from Engineering Test Management includes going to the Engineering Test Management Administration page, and deleting the friendship with Rhapsody Systems Engineering.

Procedure

1. Go to the Engineering Test Management by using the following URL.

   ```
   https://<elm_host_name>:<port_number>/qm/admin
   ```

2. Delete the friendship with Rhapsody Systems Engineering. For more information, see Deleting servers from the friends list.
3. Delete consumer keys from Rhapsody Systems Engineering. For more information, see Deleting consumer keys.

Removing friendship from Global Configuration

Deleting the friendship from Global Configuration includes going to the Global Configuration Administration page, and deleting the friendship with Rhapsody Systems Engineering.

Procedure

1. Go to the Global Configuration Administration page by using the following URL

```
https://<elm_host_name>:<port_number>/gc/admin
```

2. Delete the friendship with Rhapsody Systems Engineering. For more information, see Deleting servers from the friends list.

Integrating with link index provider

Rhapsody Systems Engineering supports querying links from the link index provider (LDX) application.

Procedure

LDX without JAS

If the LDX application does not use Jazz Authorization Server (JAS) as an OIDC provider, then create friendship with LDX application.

Procedure

1. Update the operator configuration file by using the following attributes.

```
linkIndexProvider:
    ldxEndpoint: https://<elm_host_name>:<port_number>/ldx
    isJASEnabled: false
```

2. Create friendship with LDX. For more information, see “Integrating other products with Rhapsody Systems Engineering” on page 81.
3. Consider the following format of rootservices URL.

```
https://<elm_host_name>:<port_number>/ldx/rootservices
```

4. If you are unable to provide access to provisional key by using LDX UI, then follow the steps mentioned in Approving consumer keys to approve consumer key.
5. Go to the Jazz Team Server Administration page to approve the LDX consumer key by using the following URL.

```
https://<elm_host_name>:<port_number>/jts/admin
```

6. Provide functional user for the consumer key to perform background tasks.

LDX with JAS

If the LDX application uses Jazz Authorization Server (JAS) as an OIDC provider, then use the same JAS as an OIDC provider for Rhapsody Systems Engineering.

Procedure

- Do not create friendship with LDX application.
- Update operator configuration file by using the following attributes

```
linkIndexProvider:
    ldxEndpoint: https://<elm_host_name>:<port_number>/ldx
    isJASEnabled: true
```

Adding Rhapsody Systems Engineering branch or tag in Global Configuration

Add a Rhapsody Systems Engineering branch or tag to the global configuration to manage different configurations or versions of the branch or tag depending on your requirements, improving flexibility in your development processes.

About this task

For more information, see “Adding a branch or tag to the global configuration” on page 95.

Creating project associations

Learn how to establish an association between the projects of Rhapsody Systems Engineering and the projects of other Engineering Lifecycle Management applications.

Procedure

- Create project association between Rhapsody Systems Engineering project, DOORS Next, and Rhapsody Model Manager project areas. For more information, see “Creating associations between projects” on page 79.

Troubleshooting Rhapsody Systems Engineering

Troubleshoot and find support for Rhapsody Systems Engineering.

Fixing the issues in preparing and configuring the catalog sources by using Red Hat OpenShift console

If you experience issues while preparing and configuring the catalog source for Rhapsody Systems Engineeringby using the command line inputs in Red Hat OpenShift console, the catalog source is not set up properly.

Problem

You get the following error while configuring the catalog source for Rhapsody Systems Engineering by using command line inputs in Red Hat OpenShift console.

```
Error from server (NotFound): catalogsources.operators.coreos.com "ibm-rse-catalog" not
        found
```

Solution

- Run the following command to check the pods and CatalogSource resources in the *<catalog_source_namespace>* namespace:

  ```
  oc get catalogsource,pods -n <catalog_source_namespace>
  ```

  The output should provide information about your CatalogSource resources and status of the pods.

Fixing the issues in preparing and configuring the catalog sources by using the Kubernetes cluster

If you experience issues while preparing and configuring the catalog sources by using the Kubernetes cluster, the catalog source is not set up properly.

Problem

You get the following error while configuring the catalog source for Rhapsody Systems Engineering by using Kubernetes cluster..

```
Error from server (NotFound): catalogsources.operators.coreos.com "ibm-rse-catalog" not found
```

Solution

- Run the following command to check the pods and CatalogSource resources in the *<catalog_source_namespace>* namespace:

  ```
  kubectl get catalogsource,pods -n <catalog_source_namespace>
  ```

  The output should provide information about your CatalogSource resources and status of the pods.

Reference

Learn about the APIs that are required to interact with Rhapsody Systems Engineering.

APIs

Rhapsody Systems Engineering APIs allow you to interact with the application. You can query and alter your projects, elements, and configurations by using these APIs.

The open API documentation for Rhapsody Systems Engineering is available on IBM API Hub.